Sistemi operativi

Thread, core, multicore

Un **thread** è l'unità base d'uso della CPU. Comprende un TID (identificatore), un program counter, un insieme di registri e uno stack.
I thread **condividono** il **codice**, la sezione **dati** e le **risorse** allocate al processo originale.
Un **singolo thread** si dice "**lightweight process**".
Un **processo single-thread** è detto "**heavyweight process"**.
Un processo **multi-thread** è in grado di eseguire più operazioni concorrentemente.

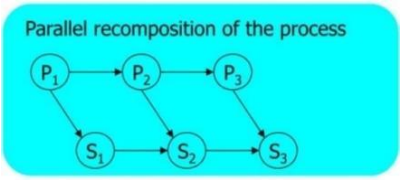
Il termine **parallelismo** indica che stanno venendo compiute più azioni in contemporanea, non necessariamente legate allo stesso problema.
Il termine **concorrenza** indica invece un lavoro di più thread per la risoluzione di un problema più grande, condividendo dati e risorse.

Esistono tipi diversi di parallelismo:
Data parallelism: una stessa operazione viene eseguita contemporaneamente da più thread su insiemi ristretti del pool di dati originale. (Far analizzare un file audio da 24 ore a 24 thread, uno per ogni ora).
Task parallelism: thread diversi eseguono compiti diversi in contemporanea per risolvere in fretta un problema.

In molte CPU moderne è presente l'**hyper-threading**, una tecnologia che permette ad un singolo core di gestire più thread contemporaneamente, mantenendo una singola ALU e sdoppiando i registri. Questa funzionalità è molto utile in scenari in cui spesso un thread è in attesa (di dati, di input...) e l'altro può sfruttare l'unità di calcolo nel mentre. È invece svantaggioso a causa dell'overhead nei carichi di lavoro compute-bound, in cui le unità di calcolo sono in uso in modo intenso e continuativo.

Grafi di precedenza

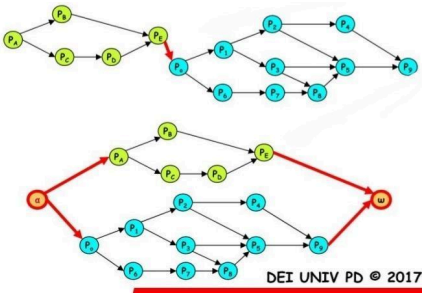
Il **grafo di precedenza** è un **grafo diretto aciclico** dove ogni arco rappresenta una relazione di precedenza, possibile dedurre un ordinamento parziale dei nodi, se Px precede Py nel grafo allora Px precede Py nell'ordinamento.



Composizione di sistemi

Un **sistema** si dice **chiuso** se presenta un solo processo iniziale e un solo processo finale. Si dice **aperto** altrimenti.

La notazione per la combinazione di sistemi di processi è la seguente:
Ca+Cb se i due sistemi sono in parallelo.
Ca*Cb se i due sistemi sono in serie.



Il **massimo grado di parallelismo** è la cardinalità del più grande sottoinsieme di processi (nodi del grafo) tali per cui, presi due qualsiasi nodi del sottoinsieme, non esiste un percorso orientato che li connette.
Concretamente, se si suddivide il grafo in livelli, è il numero di elementi del livello più popolato.
Questo è importante per l'ottimizzazione, e si cerca di ridurlo nel caso in cui il grado sia più alto del numero di core disponibili.

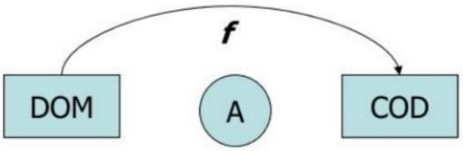
Interferenza e determinatezza

Se in un sistema il risultato finale dipende dalla **velocità** e dall'**ordine di esecuzione** dei processi, si dice che tale sistema è **indeterminato**. Si dice **determinato** altrimenti.

Ad ogni processo si assegna un **input**, un **output** e una **funzione di mappa** che trasforma i dati di input in output.
Due processi si dicono **non interferenti** se rispettano una di queste due proprietà:

- Uno è il successore dell'altro
- Non si intersecano i condomini e nemmeno i domini con i condomini.

Altrimenti si dicono **interferenti.**



Un sistema composto esclusivamente da processi non interferenti è sempre determinato.

Risorse

Con il termine **risorsa** si intende **qualsiasi cosa possa essere richiesta** da un processo per svolgere il proprio lavoro. Se **non è disponibile**, il processo dovrà **attendere**.
Le risorse possono essere condivisibili o non condivisibili.
Il termine **preemption** indica la rimozione forzata di una risorsa ad un processo.

Deadlock

Il **deadlock** o **stallo** è una situazione in cui un insieme di thread è bloccato, in cui ciascuno possiede almeno una risorsa ma è **in attesa** di acquisirne un'altra, senza procedere.
Se si raggiunge uno stallo, bisogna **eliminare i processi bloccati**, e quelli da loro dipendenti fino al ritorno ad una situazione normale.

Elenco di possibili condizioni per uno stallo:

- **mutua esclusione:** un solo processo alla volta usa le risorse.
- **allocazione parziale:** il processo richiede diverse risorse durante la sua evoluzione.
- **non detraibilità delle risorse:** solo chi possiede la risorsa può rilasciarla.
- **attesa circolare:** p1 attende p2, ... , pn-1 attende pn.

Il protocollo di accesso alle risorse da parte di un thread è
RICHIESTA → UTILIZZO → RILASCIO

Livelock

Il **livelock**, anche detto **stallo attivo**, è una situazione simile al deadlock in cui, senza che ci sia un effettivo blocco, i thread non riescono a progredire.
ESEMPIO: due thread continuano ad effettuare un polling reciproco per verificare l'uno lo stato dell'altro.

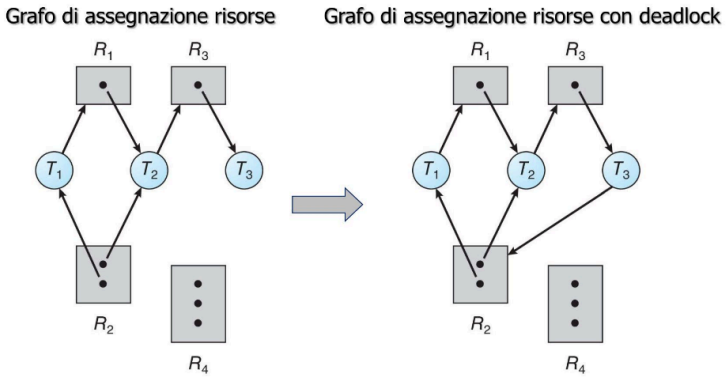
Grafo delle risorse

E' un grafo G = {V,E} in cui l'insieme dei vertici V è composto da due tipologie di nodo:

- **T** = {T1, T2, ... , Tn} è l'**insieme dei thread.**
- **R** = {R1, R2, ... , Rn} è l'**insieme delle risorse**.

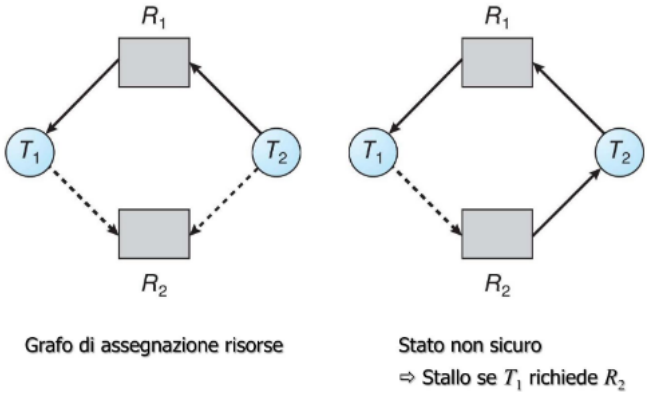
Gli archi si dividono invece in

- Ti → Rj **arco di richiesta.**
- Rj → Ti **arco di assegnazione**.
- Ti → Rj **arco di reclamo** (si indica con una freccia tratteggiata e indica che il thread può richiedere la risorsa in futuro).



Se il grafo è **aciclico** allora **non ci sono deadlock**.
Se **contiene un ciclo** e c'è **solo un'istanza per ogni** tipo di **risorsa**, allora si ha un **deadlock**. Se ci sono più istanze, allora il deadlock è possibile ma non certo.

Le **risorse** vanno **reclamate a priori**: prima che il primo thread inizi l'esecuzione, tutti i suoi archi di reclamo devono essere tracciati.

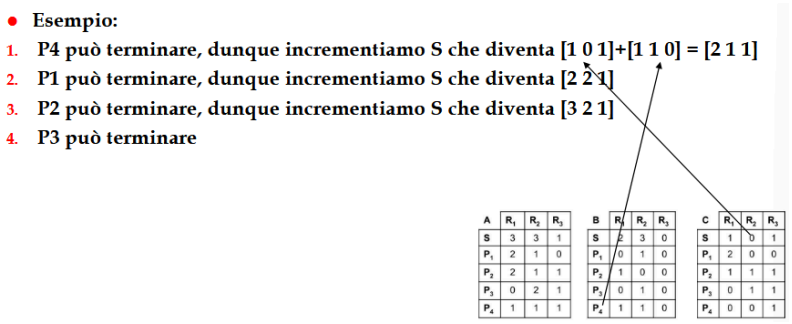


Individuazione di uno stallo

A partire da un grafo delle risorse (**grafo di Holt**) è possibile capire se un insieme di thread può terminare la sua esecuzione senza incorrere in un deadlock.
Questo metodo è detto **individuazione del blocco critico**.
Si effettuano una serie di passi di **riduzione**. Se è possibile eliminare ognuno dei thread, allora il sistema giunge al termine senza stalli.

Si costruiscono le matrici A, B, C rispettivamente per **dichiarazioni** (gli archi di reclamo), **allocazioni** (gli attuali archi di assegnazione) e **richieste** (gli attuali archi di richiesta).

Se una riga della tabella C è minore o uguale di una riga della riga S (le risorse disponibili) della tabella C, si può **eliminare il thread,** in quanto le sue richieste possono essere soddisfatte.



Prevenzione dello stallo

Un metodo possibile per evitare gli stalli è l'**allocazione globale**, ovvero la richiesta da parte di ogni thread di **tutte le risorse di cui avrà bisogno** durante la sua esecuzione. Ovviamente, seguendo questo metodo si utilizzano molte risorse in modo poco efficiente.

Alternativamente, si usa l'**allocazione gerarchica**, in cui si assegna un **valore di importanza** a ogni risorsa. Se un thread vuole accedere ad una risorsa meno importante, deve prima rilasciare quelle di importanza maggiore, rendendole disponibili agli altri processi.

Una terza soluzione possibile è l'**algoritmo del banchiere**, secondo cui le risorse vengono **allocate solo se** la **sequenza** è **sicura** (ovvero non può avvenire un deadlock).
Per verificarlo si costruiscono sempre **3 matrici**: una per le richieste **totali**, una per le risorse **allocate** e una (da una sola riga) per quelle **disponibili**.
Un processo può terminare se il suo **vettore need** (max - allocate) è minore o uguale alle risorse al momento disponibili. Può essere quindi eliminato.
Se tutti i thread vengono eliminati, il sistema si trova in uno **stato sicuro**.

Ripristino di un deadlock

Si può procedere **terminando** ogni **thread** in deadlock, ma un metodo più efficiente è quello di eliminarne uno alla volta fino a che il ciclo di deadlock non è scomparso.
Per stabilire quali thread eliminare si tengono in considerazione:

- **Priorità**: i thread meno prioritari vengono sacrificati per primi.
- **Tempo di computazione:** quanto un thread ha già lavorato e quanto tempo gli manca alla fine dell'esecuzione.
- **Risorse:** la quantità e il tipo di risorse che il processo sta utilizzando.
- Il **numero di thread** da terminare per interrompere il ciclo di stallo.

Si stabilisce quindi un **ordine** di prelazione (preemption) per minimizzare i costi.

Un thread deve **tornare ad uno stato sicuro** da cui ripartire (**rollback**).
Per effettuare un rollback si deve tenere conto di due fattori:

- **Costo:** è più efficiente terminare il thread che riportarlo ad uno stato sicuro.
- **Starvation:** è possibile che un thread venga sempre selezionato tra quelli da eliminare, di fatto non riuscendo mai a terminare la sua esecuzione. Va quindi incluso il numero di rollback che ha già subito nel calcolo del costo.

Organizzazioni hash

Utilizzano una **funzione** che trasforma una **chiave** in un **indirizzo**.



Ogni indirizzo individua un **bucket** (generalmente implementato tramite una lista) dove vengono salvati gli elementi che hanno causato una **collisione** in inserimento. Il numero di elementi che possono essere inseriti in un bucket è la sua **capacità (C)**.
Se si inserisce un elemento in un bucket contenente già C elementi, si ha un **overflow**.
L'area di memoria costituita dai bucket indirizzabili dalla funzione è detta **area primaria**.
Una funzione di hash **si applica ai numeri**. Se le chiavi utilizzate non appartengono a un insieme di numeri, è necessario **associare** ad ogni chiave un **valore numerico** (solitamente tramite una funzione biiettiva che associa ad ogni simbolo dell'alfabeto della stringa A un numero intero) e poi applicare ad esso la funzione di hash.

Il principale problema della funzioni hash è individuarne una che **riduca al minimo il numero delle collisioni** (solitamente si usa una struttura molto sparsa → tanta memoria allocata).

Linux utilizza le funzioni di hash per l'autenticazione tramite username e password.
Una volta inserito lo username, si applica un hash per trovare lo UID, che indicizza un array (o una tabella) contenente l'hash della password.

Evoluzione SO

Mentre una volta i diversi ambienti di elaborazione avevano confini ben definiti (un pc in casa connesso alla rete, una rete di pc in un ufficio), ora si parla di **sistemi distribuiti**, cioè di insiemi di calcolatori connessi attraverso una **rete di comunicazione**.

Le diverse reti sono WAN (wide area network), MAN (metropolitan area network), LAN (local area network) e PAN (personal area network).
L'accesso alle risorse condivise nella rete consente di **accelerare l'elaborazione**, **aumentare la disponibilità** di dati e **migliorare l'affidabilità**.

I due modelli di rete sono

- **Client - server:** rete in cui i vari elaboratori richiedono servizi ad un server centrale.
- **Peer to peer (P2P):** rete costituita da nodi equivalenti in cui ogni elaboratore si comporta come client e come server per ogni altro nodo nella rete.

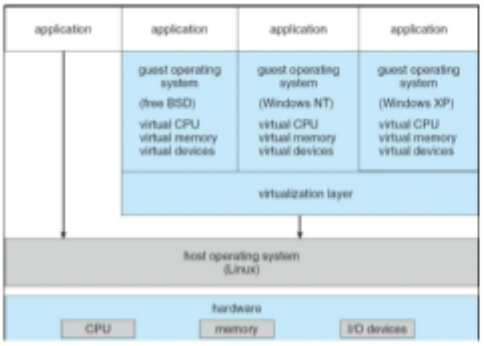
Virtualizzazione

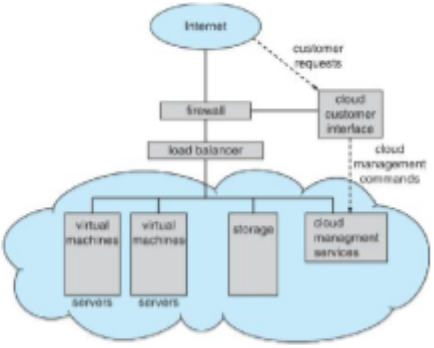
La **virtualizzazione** è una tecnica che permette di eseguire un sistema operativo come applicazione all'interno di un altro sistema operativo.
I programmi di virtualizzazione sono software di **emulazione**.
L'emulazione si usa nel caso in cui l'architettura del sistema da virtualizzare sia diversa e non compatibile con quella che fa da host.
L'emulazione permette a sistemi compilati per una piattaforma di essere eseguiti su **macchine virtuali** all'interno di piattaforme diverse.
Le macchine virtuali sono completamente **isolate** dalle altre. Possono però comunicare tra loro tramite una rete virtuale privata, modellata come una rete fisica ma realizzata interamente tramite software.
L'utilizzo di macchine virtuali consente lo sviluppo di sistemi operativi e la messa a punto di applicazioni per piattaforme diverse da quella che fa da host.

In una macchina virtuale ogni processo usufruisce di una **copia virtuale del calcolatore**.
Le risorse del computer fisico vengono condivise dall'**hypervisor** o **Virtual machine manager** (**VMM**), in modo che ogni macchina virtuale risulti avere il suo processore, la sua memoria, i propri dispositivi. Lo spazio del disco può essere diviso per creare dischi virtuali.

Cloud computing

Si tratta di una tecnica che permette di **usufruire di risorse computazionali**, di **storage** e di **applicazioni** come **servizi di rete**.
I file non sono conservati su hard disk ma su server remoti. I programmi non necessitano di essere installati, ma possono essere usati direttamente online.





Gli ambienti di cloud computing fanno uso di **macchine virtuali** e di strumenti specifici per la gestione del cloud.
La connettività impone l'uso di dispositivi di sicurezza (**firewall**).

Sistemi embedded

Sono la **tipologia di elaboratore più diffusa** (elettrodomestici, motori delle auto, robot industriali...).
Sono generalmente installati su **piattaforme rudimentali**, hanno **compiti** molto **precisi** e offrono **funzionalità limitate**.

Esistono **svariati tipi** di sistema embedded (elaboratori general-purpose, microprocessori, application specific integrated circuit (ASIC) senza nemmeno un sistema operativo).
La **tipologia più diffusa** sono i sistemi embedded **real time**, in cui si impone un limite ai tempi di esecuzione. Se questi vincoli temporali non vengono rispettati, non si considera corretta l'esecuzione. (ad esempio robot industriali).

Race condition

Situazione in cui più **thread** accedono **contemporaneamente** e modificano **dati condivisi**. In questo modo l'**esito** dell'esecuzione **dipende** esclusivamente **dall'ordine** nel quale sono avvenuti gli accessi ai dati.
Per evitare le **corse critiche** è necessario **sincronizzare** i thread.

NOTA: Un'operazione atomica è un'istruzione che viene completata senza subire interruzioni. Se è possibile usarle, possono essere una soluzione per evitare problemi di race condition, anche senza sincronizzazione.

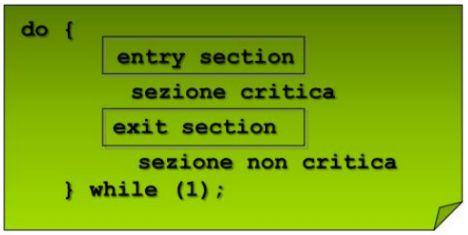
Ogni thread è costituito da una **sezione critica** nella quale effettua l'accesso e la **modifica** ai **dati condivisi**.
Ci si deve assicurare che, quando un processo è nella sua sezione critica, **nessun altro** abbia accesso ai dati.
Si seguono 3 principi:

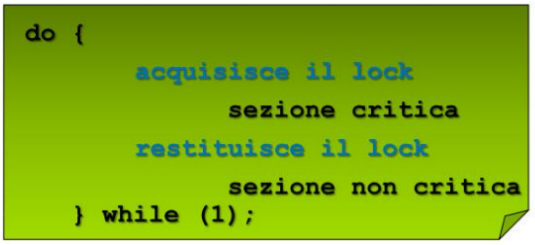
- **Mutua esclusione:** se un processo è nella sua regione critica, nessun altro processo può entrare nella propria regione critica.
- **Progresso:** se la sezione critica è libera, e qualche processo vuole entrarci, la decisione su quale processo entrerà nella sezione critica non può essere rimandata indefinitamente.
- **Attesa limitata:** se un processo ha richiesto l'ingresso in sezione critica, si pone un limite al numero di processi che possono entrarci prima di esso (per evitare la starvation).

In un dato istante, **più processi** possono **richiedere servizi** al SO. E' compito del **kernel** regolare gli accessi ai dati condivisi.

- **kernel senza diritto di prelazione:** i processi vengono eseguiti fino a che non escono dalla modalità kernel, si bloccano o resitutiscono la cpu.
- **kernel con diritto di prelazione:** si consente la prelazione di processi in esecuzione in modalità kernel. Questo consente di gestire più processi in modalità kernel su processori distinti.

In generale, una soluzione può essere l'utilizzo di un **lock**, che non permetta ad un processo di entrare nella propria sezione critica senza prima aver preso possesso del lock, che viene restituito al momento dell'uscita dalla sezione critica.





Lock mutex

Una delle soluzioni al problema della sezione critica è il **lock mutex**.
Si proteggono le regioni critiche prima **acquisendo** e poi **rilasciando** il lock. Si definisce una variabile **booleana** per stabilire se il lock è disponibile.
Le due operazioni **acquire()** e **release()** devono essere **atomiche**, generalmente implementate tramite istruzioni hardware.
La soluzione con lock mutex impone l'**attesa attiva** (busy waiting): mentre un processo si trova in attesa deve continuamente iterare effettuando una chiamata ad acquire().

Semafori

I **semafori** sono strumenti di sincronizzazione realizzati tramite una variabile intera **S.**
Si accede al semaforo tramite le due **operazioni atomiche wait()** e **signal()**.

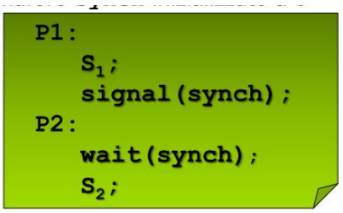
- **signal** aumenta il contatore di 1.
- **wait** diminuisce il contatore di 1, se è già 0, rimane in attesa fino a quando il contatore non risale.

Esistono due tipologie di semafori:

- **Semafori contatori:** la variabile S può assumere valori in un dominio non limitato. E' utile nel controllo dell'accesso alle risorse presenti in un numero

finito di esemplari. Un processo che desidera accedere ad una risorsa deve invocare una wait(). Se il semaforo è a 0, tutte le istanze della risorsa sono allocate e il processo deve attendere.

- **Semafori binari:** accetta come valori solo 0 e 1. Sono simili in funzionamento ad un lock mutex. Utili in caso di coppie di processi che devono essere eseguiti in sequenza.



Anche utilizzando i semafori si incorre nel problema del **busy waiting**. Quando un processo è nella sua fase critica, tutti gli altri sono bloccati nel loop della loro **entry section** (wait del semaforo). Questo spreca cicli della cpu che potrebbero essere utilizzati produttivamente.

Per evitare questo problema si può ricorrere a una **definizione alternativa** di semaforo:

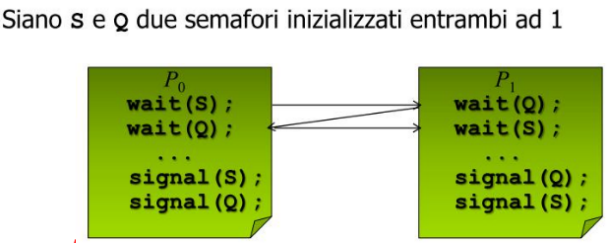
- Associata ad ogni semaforo, c'è una **coda di processi** in attesa.
- La struttura del semaforo contiene un **valore intero** (il numero di processi in attesa) e un **puntatore alla testa della lista** dei processi.

Si assumono disponibili **due operazioni** (disponibili come **system call**):

- **block:** posiziona il processo chiamante nell'opportuna coda di attesa, interrompendolo.
- **wakeup:** rimuove un processo dalla coda di attesa e lo sposta nella r**eady queue**.

Questa tipologia di semaforo ha un contatore che può avere valori negativi. Se il valore è negativo, il suo modulo è il numero di processi che stanno attendendo il semaforo.

La realizzazione di semafori con coda di attesa può condurre a **deadlock**, in cui ogni processo attende l'esecuzione di un'istruzione signal() o di un qualsiasi altro evento che può essere eseguito soltanto da uno dei processi in coda.



Un processo che attende per un tempo indefinito un semaforo è in **starvation**.

Per assicurarsi un'**attesa limitata** si devono aggiungere i processi alla coda del semaforo in modalità **LIFO**.

Si definisce **semaforo privato** un semaforo in cui **solo un processo** è autorizzato a chiamare **wait()**, mentre **chiunque** può chiamare **signal()**.

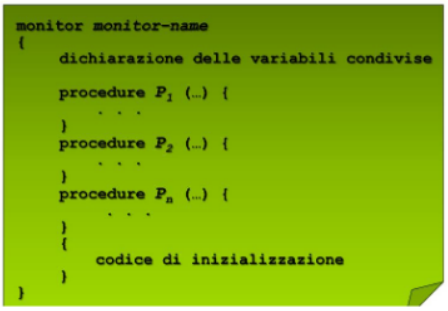
Monitor

I monitor **semplificano** la gestione della **mutua esclusione**, riducendo quindi le possibilità di commettere errori rispetto all'uso dei semafori.
Come i semafori, però, **richiedono** di avere una **memoria condivisa**.
Un solo processo alla volta può accedere al monitor.

I monitor si gestiscono tramite le operazioni **wait()** e **signal()**, ma sono gestite diversamente dal semaforo:

- **wait**: il processo rimane sospeso fino alla chiamata di signal() da parte di un altro processo.
- **signal**: attiva un processo sospeso dal wait().

Per **allocare risorse**, un processo deve chiamare una **routine interna** al monitor.



Si possono usare delle **variabili condition** per mettere in attesa un processo fino alla realizzazione di una **specifica condizione**.
Ogni variabile condition ha una sua coda di processi associata.
Quando un processo è in attesa di una **condizione x**, invoca una wait() sulla variabile condition legata ad essa (**wait(x)**). A questo punto viene inserito nella coda di x, da cui uscirà quando un altro processo avvererà la condizione richiesta e lo segnalerà tramite un **signal(x)**.

Un'altra utilità delle **variabili condition** è la gestione della **priorità** dei processi. Si associa ad ogni processo un **indice di priorità** e si fa in modo di schedulare prima quelli con elevata priorità.

Si possono usare i monitor per gestire l'**allocazione di risorse** a singola istanza tra processi in competizione, che ne specificano il **tempo di utilizzo** massimo.

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;
   void acquire(int time) {
           if (busy)
              x.wait(time);
           busy = TRUE;
   }
   void release() {
           busy = FALSE;
           x.signal();
   }
    initialization code() {
          busy = FALSE;
   }
}
```

Si possono usare i **monitor** per gestire la **comunicazione tra due processi**, sfruttando un **buffer comune** per inviare messaggi. Mentre un processo **invia** il messaggio tramite una chiamata ad una funzione **send()**, solo esso avrà accesso al buffer. Lo stesso vale mentre l'altro lo **legge** dal buffer chiamando la funzione **recieve()**.

```
monitor buffer_circolare
{
  messaggio buffer[N];
  int contatore=0; int testa=0; int coda=0;
  condition pieno;
  condition vuoto;

  void send(messaggio m)
  {
     if (contatore==N) pieno.wait;
     buffer[coda] = m;
     coda = (coda + 1)%N;
     ++contatore;
     vuoto.signal;
}
```

funzione send()

```
messaggio receive()
  {
     messaggio m;
     if (contatore == 0) vuoto.wait;
     m = buffer[testa];
     testa = (testa + 1)%N;
     --contatore;
     pieno.signal;
     return m;
  }
}/* fine monitor */
```

funzione recieve()

Linux e sincronizzazione

Attualmente, Linux fornisce fornisce diversi strumenti per la gestione della sincronizzazione dei processi, come

- **Lock mutex:** gestiti tramite valori booleani impostati tramite le funzioni mutex_lock() e mutex_unlock(). Un processo in attesa del lock è sospeso.
- **Spinlock:** simili ad un mutex, con la differenza che i processi in attesa sono in busy waiting e consumano quindi cicli di cpu per attendere il rilascio della risorsa.
- **Semafori:** consentono di sincronizzare i processi come descritto sopra.
- **Interi atomici:** sono numeri interi in cui ogni operazione matematica viene eseguita senza interruzioni. Particolarmente utili nei contatori.

Lo standard **POSIX** fornisce diversi degli strumenti di sincronizzazione usati dai thread a livello utente tramite la **libreria pthread**.

Gestione di un lock mutex con libreria pthread

```c
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

```c
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

Lock mutex con variabili condition

```c
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

```c
pthread_mutex_lock(&mutex);
while (a != b)
      pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

```c
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```

I **semafori** nello standard POSIX possono essere di **due tipi**: con nome o senza nome. Quelli **con nome** possono essere **usati da processi non correlati**, quelli **senza nome** solo dal **processo che li ha creati**.

Gestione di un semaforo con nome con libreria pthread

```c
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

```c
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

Gestione di un semaforo senza nome con libreria pthread

```c
#include<semaphore.h>
sem_t sem;
…
/* crea il semaforo e lo inizializza a 1 */
sem_init(&sem,0,1);
…
```

```c
/* acquisisci il semaforo */
sem_wait(&sem);

/* sezione critica */

/* rilascia il semaforo */
sem_post(&sem);
```

La libreria **pthread** permette anche di creare e mandare in esecuzione thread a partire da un processo.

Un **esempio di codice** in cui si vede questa procedura:

```c
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```

```c
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

Usando **più lock mutex insieme** si può causare un **deadlock.**
Si prendano come esempio questi due thread:

```c
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}
```

```c
/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

Nel caso in cui *one* prendesse il primo lock e *two* il secondo, nessuno dei due potrebbe liberare la risorsa necessaria all'altro per terminare l'esecuzione.
In questo caso, è sufficiente fare in modo che l'**acquisizione dei lock** sia effettuata **nello stesso ordine** nei due processi.

In **Linux**, si utilizza lo strumento **lockdep** per individuare possibili deadlock nel codice.
Ovviamente, l'esecuzione con lockdep **consuma molte risorse** e tende ad essere significativamente più lenta del normale.
E' quindi uno strumento da usare solamente in **fase di debug**.

Memory barrier

Normalmente, **il compilatore può cambiare l'ordine** delle operazioni di lettura/scrittura in memoria rispetto a come appaiono nel codice per ottimizzare alcuni segmenti.

Le **memory barrier** sono particolari istruzioni che impongono al **compilatore** di eseguire prima tutte le operazioni in memoria che sono state invocate prima della barriera, e poi di eseguire quelle invocate dopo.

Le memory barrier **sono implementate all'interno dei semafori** per fare in modo che l'acquisizione e il rilascio della risorsa non crei problemi.
Non sempre, però, è necessario usare i semafori.
Se bisogna solo assicurarsi che i thread vedano i dati nello stesso ordine, una semplice memory barrier è più che sufficiente.

Scheduling CPU

Meccanismo di interruzione

Si tratta di un **meccanismo** a disposizione dell'elaboratore che **salva il program counter** del processo interrotto e **trasferisce il controllo** ad un'altra locazione di memoria che conterrà la **procedura di interruzione**, che determinerà la causa della sua attivazione e agirà di conseguenza.

Il sistema operativo ha accesso esclusivo ad alcune funzioni come

- **Abilitare/disabilitare interruzioni.**
- **Accedere ai registri di protezione della memoria.**
- **Gestione delle eccezioni.**
- **Arresto della CPU.**

Kernel

E' il **nucleo** del sistema operativo. Viene implementato in parte in linguaggio assembly.
Si divide in varie sezioni

- **First interrupt handler (FLIH):** gestione iniziale di ogni interrupt. Individua l'origine e attiva la specifica funzione di gestione per quell'interrupt.
- **Dispatcher:** assegnazione dei vari core ai processi.
- **Funzioni di sincronizzazione:** implementa semafori, lock, etc...

Processi

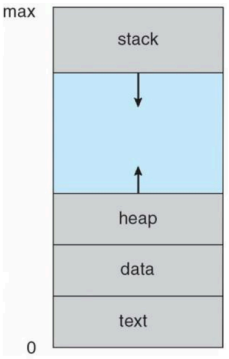
I processi sono **rappresentati** da un **descrittore** denominato **struttura di controllo** o **vettore di stato** che può trovarsi in 3 stati:

- **Running:** il processo è in esecuzione.
- **Ready:** il processo sta aspettando di accedere a un core.
- **Waiting:** il processo non può essere eseguito (ad esempio per mancanza di risorse).

Il **dispatcher** del kernel usa la funzione **assign** per **assegnare una cpu** al processo, **resign** per **rimuoverla**, **suspend** per metterlo in **stato di waiting**, **resume** per **risvegliarlo**.

Lo **stato del processo** è l'**insieme di tutte le informazioni nei registri** condivisi del sistema che sono accessibili al processo.
Un processo include:

- **Sezione di testo:** il codice da eseguire.
- **Registri:** il valore del program counter e di tutti i registri della cpu.
- **Stack:** dati temporanei, variabili locali, parametri delle subroutine.
- **Sezione dati:** variabili globali.
- **Heap:** memoria allocata dinamicamente.



Il **programma** è l'**entità passiva** salvata su un disco. Quando il programma viene **caricato in memoria** diventa un **processo**.
Un programma può corrispondere a **vari processi** (detti **istanze** del programma).

Scheduler

Lo scheduler prende una decisione quando lo stato cambia:

- **Running → Wait:** richiesta di I/O, attesa di un processo figlio, etc...
- **Running → Ready:** interrupt.
- **Wait → Ready:** completamento di I/O.
- **Termina il processo**

Se lo scheduling avviene solo nel primo e l'ultimo caso, allora il sistema è **non-preemptive** (senza diritto di prelazione), altrimenti è **preemptive**.

Nel caso di uno scheduler preemptive, vanno considerati i **rischi dovuti alla prelazione**, come prelazione durante la modifica di un file, prelazione di processi del kernel, l'interruzione di operazioni cruciali del SO.

Dispatcher

Il **dispatcher** passa il **controllo della cpu** ai processi a seconda dell'output dello scheduler.
Svolge 3 operazioni: context switch → passaggio in user mode → salto alla posizione iniziale del nuovo programma.

Il **tempo impiegato** tra l'interruzione del processo precedente e l'inizio di quello nuovo è detto **latenza di dispatch**.

Primitive di sincronizzazione

La **sincronizzazione** dei processi è generalmente gestita tramite **semafori**.
Deve essere implementata all'interno del kernel in quanto l'invocazione di una wait() deve avere **accesso diretto al dispatcher**.

I semafori sono implementati tramite una **coda di descrittori di stato** dei propri processi sospesi.
Generalmente è una coda FIFO, ma può essere LIFO o divisa per priorità.
Wait e **signal** devono essere **operazioni indivisibili**.

Scheduling della cpu

I **processi** sono di solito **ordinati** anche in base ad una **priorità**.
L'assegnazione della priorità è dinamica, e dipende da fattori come le risorse richieste, il tempo trascorso dall'ultimo accesso alla cpu, etc...

I processi si dividono in

- **I/O bound:** lunghe fasi di attesa, esecuzioni brevi (ad esempio editor di testo).
- **CPU bound:** uso intensivo del processore.

I fattori da considerare sono

- **Utilizzo della CPU:** deve essere il più attiva possibile.
- **Throughput:** devono terminare quanti più processi possibile.
- **Tempo di turnaround:** tempo totale per l'esecuzione di un determinato processo.
- **Tempo di attesa:** tempo passato nella ready queue.
- **Tempo di risposta:** tempo tra una richiesta e la risposta prodotta.

Thread

La definizione di processo si basa su due aspetti: il possesso delle risorse e l'esecuzione.

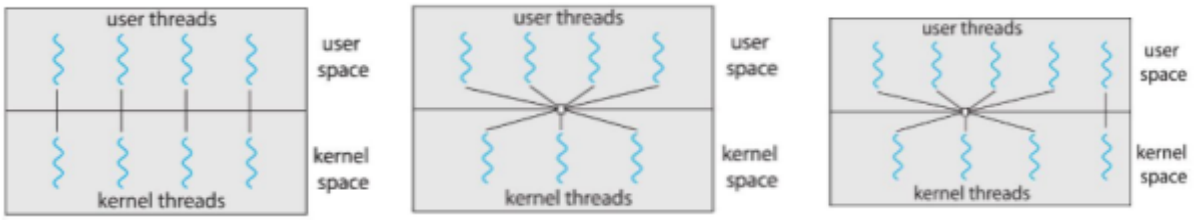
- Il **thread** viene **eseguito**.
- Il **processo** possiede le **risorse**.

Si dice **multithreading** per descrivere la situazione in cui ad un processo siano assegnati più thread.

Tutti i **thread condividono le risorse** del processo e hanno accesso agli **stessi dati**. Ogni thread ha il suo descrittore (program counter, registri, stack, ...).

I thread a livello utente e quelli a livello kernel possono essere legati tra loro secondo tre modalità:

- **One-to-One:** Ad ogni processo utente è associato un processo kernel. Si genera molta concorrenza. Usato da Windows e Linux.
- **Many-to-one:** Un numero arbitrario di processi utente legato ad un numero di processi kernel. Non molto comune.
- **Two-level:** Può funzionare sia da one-to-one sia da many-to-many.



Processi in Linux

I Processi sono gestiti dal **PCB** (process control block) anche detto **TCB** (task control block) in Linux.

Una volta creato un nuovo processo, il parent ha due opzioni:

- Proseguire l'esecuzione in parallelo
- Aspettare che il nuovo processo termini.

Il nuovo processo può essere una **copia del genitore** oppure contenere un **nuovo programma**.

I nuovi processi sono **generati gerarchicamente**. Ogni processo può fare da genitore e generare processi figli.
In Linux, il primo processo è sempre **systemd** (che ha PID = 1).

Linux usa la system call **clone()** per generare processi. Vengono passate come argomenti alcune flag per determinare se il processo figlio condivide i dati o altri elementi con il genitore.

La **ready queue** è realizzata tramite una **lista** concatenata e un **puntatore current** al processo attuale.
Per duplicare un processo si usa la system call **fork()**, che restituisce due valori diversi al genitore (il suo PID) e al figlio (il valore 0).

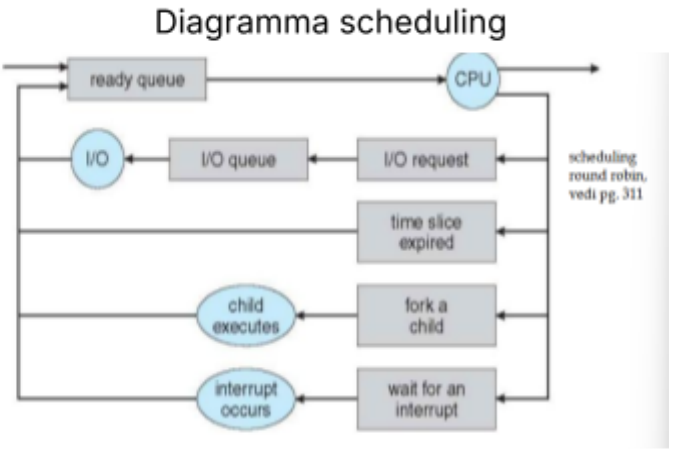
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      execlp("/bin/ls","ls",NULL);
   }
   else { /* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
   }

   return 0;
```



IOS

Alcuni **SO per mobile**. incluse le prime versioni di iOS, prevedevano **un solo processo utente in esecuzione** alla volta (tutti gli altri sospesi).

Nelle versioni **più recenti**, esiste **un processo in foreground** controllabile tramite GUI e **altri processi in background**, che sono in esecuzione ma non possono accedere al display.
Un'applicazione può essere in background solo se

- Realizza un unico task di lunghezza finita.
- Riceve notifiche sul verificarsi di un evento (email, messaggi, etc...).
- Impone attività di lunga durata (lettore audio).

Android

Android **non pone particolari limiti** alle applicazioni eseguite in **background**.
Un'applicazione in background usa un **servizio** separato in esecuzione per conto dell'app.
Il servizio **continua a funzionare** anche senza che l'app venga sospesa.
I servizi **non hanno interfaccia** utente.

I processi su Android seguono una **gerarchia**:

- **Processo foreground:** il processo con cui l'utente sta interagendo attivamente.
- **Processo visibile:** esegue un'attività a cui il processo foreground fa riferimento.
- **Processo di servizio:** esegue in background ma svolge un'attività evidente per l'utente.
- **Processo in background:** svolge attività non evidente per l'utente.
- **Processo vuoto:** non contiene componenti attive associate ad un'app.

Questo **ordine di priorità** è sfruttato dall'**OOM Killer** (Out Of Memory Killer): quando non c'è abbastanza ram a disposizione, vengono terminati i processi di importanza minore.
Per non rimuovere accidentalmente processi in background utili (es. antivirus), si usa un **foreground service**, ovvero si mostra una notifica persistente che alza la priorità del processo.

Browser Chrome

I **processi** dei browser moderni sono **suddivisi** in modo che, in caso di problemi, non si blocchi l'intera applicazione.

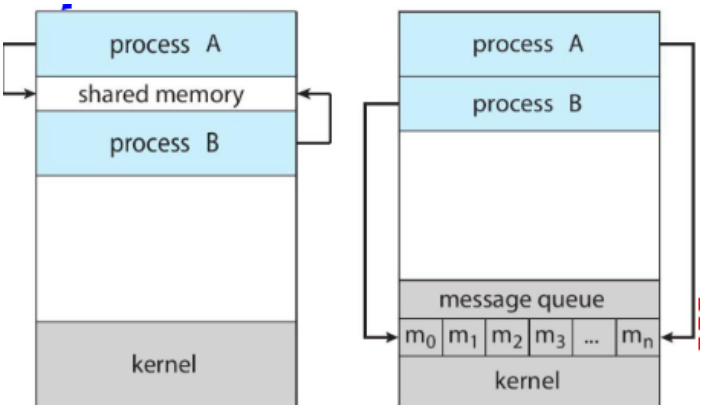
- Il **browser** ha un processo indipendente che si occupa di gestire l'interfaccia utente e l'I/O dal disco o dalla rete.
- I **renderer** contengono la logica per il rendering delle pagine web, l'esecuzione del Javascript, le immagini, etc...
  Ad ogni pagina è associato un renderer.
- I **plug-in** del browser sono associati a processi indipendenti.

Comunicazione tra processi

La **comunicazione** tra diversi processi è necessaria per

- **Condivisione** di informazioni.
- **Velocità** computazionale, nel caso di task suddivise in sotto-task.
- **Modularità** del codice.

E' quindi richiesto un meccanismo di interprocess communication (**IPC**).



Il modo più veloce per comunicare è tramite **memoria condivisa**, che non usa system calls.
Un altro metodo è lo **scambio di messaggi**. Questo metodo usa system call ed è quindi più lento, ma è semplice da utilizzare e non nascono conflitti.

Pipe

Questo metodo di IPC si divide in due categorie.

- **Ordinary Pipe:** anche note come anonymous pipe, sono canali di comunicazione **unidirezionali** in cui si richiede una gerarchia **padre-figlio**.
  Il padre è colui che scrive (**produttore**), il figlio è quello che legge (**consumatore**).
  Si crea la pipe nel processo padre che poi si divide tramite un fork() creando il figlio. Il padre chiuderà il read-end (non leggerà), il figlio chiuderà il write-end (non scriverà).

- **Named Pipe:** sono sistemi di IPC **bidirezionali**. Non richiedono un rapporto padre-figlio tra i processi comunicanti. Possono essere usate da più processi contemporaneamente.

Le pipe **sono flussi continui di dati.**
Non è possibile distinguere se i dati contenuti nella pipe siano stati scritti insieme o in più operazioni separate.
La politica delle pipe è **FIFO**, il primo byte che entra è anche il primo ad uscire.

Le **ordinary pipe** sono in memoria solo fino a che ci rimangono i due processi che la usano sono “vivi”. E’ **volatile**.
Le **named pipe rimangono in memoria** fino a che non vengono chiuse.

Mailboxes

Sono metodi di IPC **simili alle named pipe**, tuttavia il loro funzionamento è più orientato verso lo **scambio di messaggi** invece che al flusso continuo di dati.

Ogni mailbox è dotata di un **ID unico**.
Perché due processi possano comunicare, devono essere assegnati alla stessa mailbox (si crea un **link**).
Ogni processo può comunicare su più mailbox.
Un **link** tra due processi può essere **unidirezionale o bidirezionale**.

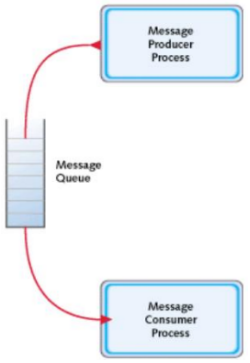
Le operazioni legate alle mailboxes sono

- **Create**
- **Send / Receive**: la forma è send(A, message) per mandare il messaggio alla mailbox A.
- **Delete**

La **comunicazione** tramite mailbox può essere **indiretta**, ovvero con **più di un ricevitore.**
Uno solo di essi può ricevere effettivamente il messaggio.
Si sceglie un'**opportuna politica** per stabilire **chi debba ricevere** il messaggio (o si limita a 2 il numero di processi per mailbox).

Lo scambio di messaggi può essere **bloccante** o no.
Se è bloccante, il mittente si blocca fino a messaggio ricevuto, mentre il ricevitore si blocca fino a che un messaggio non è disponibile.

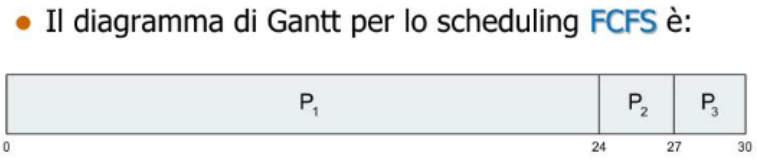
I messaggi transitano in una **coda** per passare da mittente a ricevitore.
La coda può avere un solo posto (**bufferizzazione assente**) oppure avere un buffer finito o illimitato di messaggi (**bufferizzazione automatica**).
Se la coda è piena, il mittente si blocca.



First come first served (FCFS)

La **CPU** viene assegnata **al primo processo che la richiede**.
Si basa su un'implementazione tramite coda **FIFO**.
Quando un processo viene messo in coda, si collega il suo PCB all'ultimo elemento.
Quando la CPU si libera, viene assegnata al primo processo della **ready queue**.

Per rappresentare lo scheduling si usa un **diagramma di Gantt**, che sarebbe un istogramma che mostra la durata dei processi e i loro istanti di inizio e fine.



Questo scheduling ha tempi di attesa lunghi per processi molto corti (**effetto convoglio**).
Uno scheduling più efficiente sarebbe



Priorità / Shortest job first (SJF)

Si associa a ciascun processo un **valore di priorità**.
La CPU viene **allocata** al processo con **priorità più alta**.
Questo metodo soffre del problema della **starvation**, ovvero i processi a bassa priorità potrebbero non essere mai eseguiti.
La **soluzione** a questo problema è l'**aging**, ovvero l'aumento graduale della priorità di un processo mentre questo si trova in attesa.

Lo scheduling **SJF** assegna i valori di **priorità a seconda del tempo di burst** di ogni processo (il **tempo** di CPU **necessario per terminare**). Vengono eseguiti prima i processi più corti.

Round Robin

A ciascun processo viene allocata la CPU per un **quanto di tempo q**, di solito tra i 10 e i 100 ms.
Se sono presenti n processi in ready queue, ciascuno occupa la CPU per un tempo 1/n espresso in funzione di un numero di quanti di durata q.

Le **prestazioni** per **q grande** sono quelle di un **FCFS**.
Per **q piccolo**, si deve far attenzione ad avere un quanto q molto maggiore del tempo di context switch, altrimenti si aggiunge troppo overhead.

In genere, il round robin ha **tempi medi di attesa maggiori** rispetto ad un approccio SJF.
Tuttavia, si ottiene il **miglior tempo medio di risposta**.

Shortest process next (SPN) e Shortest remaining time (SRT)

SPN assegna la CPU al processo con tempo di esecuzione minore nella ready queue.
A parità di valore, usa una strategia FCFS. **Corrisponde alla SJF**.

SRT valuta il tempo di esecuzione rimanente. Si usano **algoritmi predittivi** per determinare il tempo di esecuzione di ogni processo.
Si tratta semplicemente del caso di **SJF con prelazione**. Infatti di default si considera SJF come senza prelazione.

Code multiple

La ready queue si divide in **foreground** (interattiva) e **background** (batch).
Ogni coda ha il suo algoritmo di scheduling, **rispettivamente Round Robin e FCFS**.

Lo scheduling tra le due code può essere di due tipi:

- **Priorità fissa:** si servono prima tutti i processi fg, poi quelli in bg
- **Time slice:** ciascuna coda ha una percentuale di tempo di CPU, ad esempio 80% fg e 20% bg.

I **processi** possono essere **fissi** tra le code **oppure possono spostarsi** tra esse (eventualmente implementando anche l'**aging**).

Scheduling

Esistono varie **tipologie** di scheduling:

Priorità proporzionale alla frequenza: La priorità (statica) è inversamente proporzionale al periodo, ovvero i processi che usano più di frequente la CPU sono i più prioritari.
Questo algoritmo di scheduling è ottimale, ovvero se non è in grado di pianificare l'esecuzione di una serie di processi, non ci riuscirà nessun altro algoritmo che assegni priorità statiche.

In generale, nel caso di eventi periodici, se si hanno m eventi che richiedono C secondi con un periodo P, il carico può essere gestito solo se

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Se viene rispettato questo requisito, il sistema si dice **schedulabile** (**limite teorico**).
Tuttavia il **limite reale** per sistemi con processi periodici è

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq m \cdot (2^{1/m} - 1)$$

Scheduling EDF (Earliest Deadline First): Scheduling dei processi a seconda delle scadenze (tempo entro cui devono terminare l'esecuzione).
Quando un processo diventa eseguibile, deve annunciare la sua prossima scadenza allo scheduler.
Con il valore di questa scadenza si calcola la priorità.
Si può applicare questo metodo anche a processi non periodici e con tempo di elaborazione variabile.
L'algoritmo è ottimale e porta l'utilizzo della CPU al 100%.

Scelta dell'algoritmo di scheduling

Per selezionare l'algoritmo di scheduling si usano dei **modelli deterministici**, che considerano un **carico di lavoro** e scelgono quale sia la **prestazione di ogni algoritmo** per quel carico.
Per ciascun algoritmo si calcola il **tempo medio di attesa**.

Usando il metodo delle **reti di code** si descrive il sistema di calcolo come una **rete di server**, ciascuno con una coda di attesa.
La CPU è un server con la propria ready queue, il sistema di I/O ha le sue code per i dispositivi.
Conoscendo l'andamento degli arrivi e dei servizi, è possibile **fare una stima** dell'**utilizzo della CPU**, della lunghezza delle **code**, del **tempo di risposta**, etc...

Sia **n** la lunghezza media di una coda, **W** il tempo medio di attesa e **λ** l'andamento medio di arrivo di nuovi processi.
La **formula di Little** è un'equazione valida per ogni algoritmo di scheduling

$$n = \lambda \times W$$

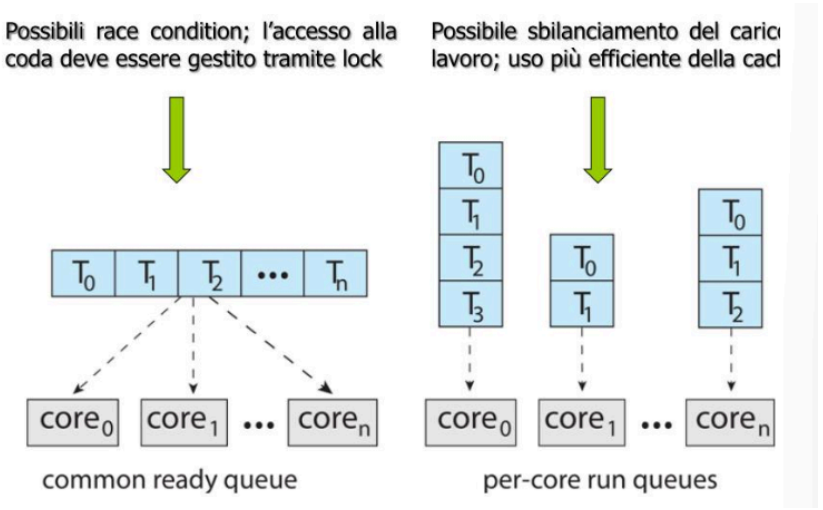
E' anche possibile **simulare** il sistema di calcolo tramite software, sia tramite dati generati artificialmente che tramite dati registrati, in modo da studiarne le prestazioni.

Scheduling multiprocessore

Lo scheduling diventa complesso quando nel sistema sono presenti più CPU.

Multielaborazione asimmetrica: lo scheduling, l'elaborazione dell' I/O e le altre attività sono affidate ad un solo processore detto **master server**.
Si riduce la necessità di condividere dati in quanto solo un processore deve accedere alle strutture dati di sistema.

Multielaborazione simmetrica (SMP): i processi pronti vanno a formare una **coda comune**, oppure c'è una **coda per ogni processore**.
Ciascun processore ha uno **scheduler** che esamina l'opportuna coda per capire quale processo eseguire.
Questo metodo ha una complessità maggiore data la necessità di più processori di accedere ad una struttura dati comune. (Usato da Windows, MacOs, Linux).



Predilezione del processore: mantenere un processo in esecuzione sempre sullo stesso core permette di riutilizzare il contenuto della cache per burst successivi. Linux implementa la **predilezione forte**, ovvero esistono specifiche chiamate di sistema che garantiscono che un processo non lascerà una data cpu.

Bilanciamento del carico: ripartire il carico uniformemente tra i vari processori.
La **migrazione guidata** prevede un processo dedicato che controlla periodicamente il carico delle cpu.
La **migrazione spontanea** prevede che un processore inattivo sottragga un processo in attesa da uno carico.

Multicore

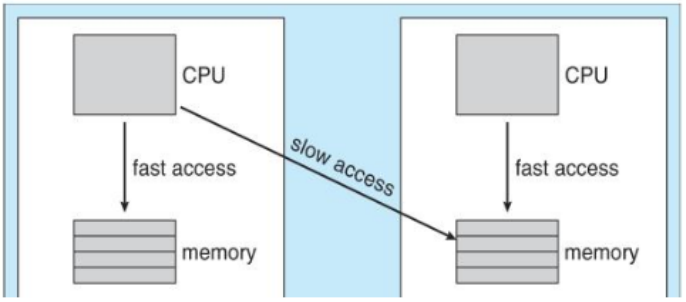
I sistemi **SMP** (Symmetric Multiprocessing) rendono possibile la **concorrenza fra thread** con l'utilizzo di più core fisici.
I sistemi SMP su processori multicore sono più rapidi e consumano meno energia.

Quando un processore accede alla memoria, la maggior parte del tempo è spesa in attesa dei dati: **stallo della memoria**.
Per ovviare a questo problema nasce l'**hyperthreading**, ovvero ad ogni core fisico sono assegnati più thread logici, in modo che l'unità di calcolo del core non rimanga inutilizzata durante l'attesa della memoria ma possa eseguire istruzioni per un altro thread.

NUMA

Non Uniform Memory Access. Si tratta di una situazione comune in sistemi costituiti da **diverse schede**, ognuna della quali con una o più CPU e memoria.
L'accesso da parte di una CPU alla memoria di un'altra scheda è molto lento rispetto ad un accesso alla memoria locale.

Per questo, ogni **thread dovrebbe risiedere nella memoria locale** del processore su cui viene eseguito.



Scheduling Linux

Nelle **prime versioni**, si usava una variante dell'algoritmo tradizionale di UNIX con code multiple gestite **principalmente** tramite **Round Robin**.

Dalla versione 2.26.23 **Completely Fair Scheduler** (**CFS**).
Si effettua una divisione in **classi di scheduling**, ciascuna con la propria priorità.

Lo scheduler assegna una percentuale del tempo della CPU. Tale percentuale viene calcolata sulla base di **valori nice** (da -20 a +19) assegnati ad ogni processo.

CFS non assegna una priorità, ma tiene conto del **tempo di esecuzione virtuale**, che viene salvato in una variabile **vruntime**.
Questo valore dipende dal valore nice. Nel caso di nice = 0, allora coincide col tempo effettivo.
La formula per aggiornare vruntime è

$$vruntime+ = \Delta t \cdot \frac{1024}{peso}$$

Il valore del peso dipende dal valore nice. 1024 è il valore per nice = 0.
Viene **eseguito** il task col **vruntime minore**.

L'algoritmo CFS non utilizza quanti di tempo discreti, ma definisce a seconda del numero di task in esecuzione una **latenza obiettivo** (tempo entro il quale ogni processo deve aver ricevuto la CPU).
Si determina quindi il **tempo ideale** per ogni processo:

$$tempo_ideale_i = target_latency \cdot \frac{peso_i}{\sum pesi}$$

Considerare solo il tempo ideale potrebbe provocare un numero troppo elevato di **context switch**.

Per evitare ciò, il sistema impone un **tempo minimo** di esecuzione.
Se un processo ha superato il tempo minimo di esecuzione ma non il tempo ideale, può essere soggetto a prelazione.
Se un processo supera il tempo ideale, è probabile che venga sospeso per garantire l'esecuzione di altri processi.

Lo scheduler CFS è **compatibile con NUMA** e riduce al minimo la migrazione dei thread.

Scheduling Windows

In **Windows 10** i processi sono schedulati a seconda della loro **priorità**.
I valori di priorità variano da 0 a 31, dove lo 0 è riservato allo zero-page, un thread responsabile di azzerare pagine libere se non ci sono thread da mandare in esecuzione.
La CPU è assegnata ai processi tramite **RR** ai processi con più alta priorità. Se un processo di priorità maggiore arriva, ne viene sospeso uno di priorità minore per poterlo eseguire al più presto.
Esiste una **priorità di classe** del processo e una **priorità all'interno della classe**.
La **priorità** è **dinamica** per processi con valore tra 0 e 15 per evitare starvation. Per tutti gli altri è statica.

In **Windows 11** si sfrutta inoltre la tecnologia Intel Alder Lake (divisione tra P-Core e E-Core).
Un microcontroller all'interno della CPU (**Intel Thread Detector**) controlla l'intero chip, registrando dati sull'uso di ogni processore, e comunicando al sistema operativo quali thread possano essere mandati ai P-Core e quali invece possano essere eliminati per fare spazio in caso di necessità.
Il thread detector **comunica con lo scheduler**, che può tecnicamente prendere decisioni diverse da quelle suggerite in casi specifici.
L'integrazione di questa tecnologia rende lo scheduling di Windows 11 **molto più efficiente** di quanto non fosse su Windows 10.

Sistemi real-time

Sistemi hard real-time: richiedono il completamento dei processi critici entro un intervallo di tempo predeterminato.

Sistemi soft real-time: richiedono solo che ai processi critici sia assegnata una maggiore priorità rispetto a quelli di routine.

I sistemi real-time sono per natura **guidati dagli eventi**.
La **latenza** di un evento è il tempo trascorso tra l'occorrenza di tale evento e la sua gestione da parte del sistema operativo.

- **Latenza delle interruzioni:** tempo tra la notifica di un'interruzione e l'avvio della routine per la sua gestione.
- **Latenza di dispatch**: tempo necessario al dispatcher per interrompere un processo e avviarne un altro.
  La **fase di conflitto** nella latenza di dispatch prevede la prelazione di ogni processo nel kernel e la cessione delle risorse da parte dei processi a bassa priorità.

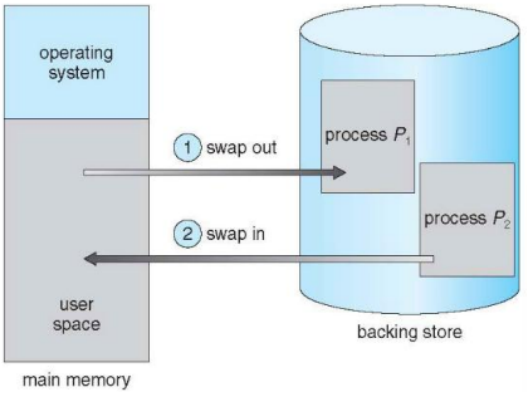
Gestione memoria principale

I file generati dal compilatore contengono **indirizzi rilocabili**, ovvero relativi all'inizio del modulo. Ciò è valido anche per i file prodotti dal linker.

E' compito del sistema operativo collocare in memoria i processi, evitando di lasciare aree libere di piccole dimensioni che non sono sufficienti per allocare un processo (**frammentazione esterna**).

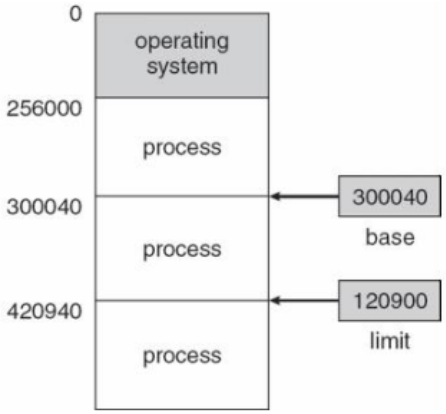
Per liberare spazio il sistema può scegliere di scaricare sul disco parte o l'intera immagine di un processo (**swap out**).

Nel caso di **librerie dinamiche**, il processo richiede il caricamento in memoria del modulo della libreria necessario. Se questo è già in memoria per un altro processo, non è necessario ricaricarlo.



Nel caso in cui un processo debba essere caricato in memoria ma questa è piena, occorre eseguire lo swap out di uno dei processi già allocati.
Il **costo di context switch** può essere **molto elevato**.

Ogni processo deve avere uno **spazio di memoria separato**.
Una possibile soluzione è avere un **registro base** e un **registro limite**, che definiscono l'inizio e la fine dello spazio del processo.



Se si prova ad accedere ad un indirizzo superiore al limite, il SO interrompe il processo (**segmentation fault**).

Un'altra soluzione è usare il **registro limite** per memorizzare l'**estensione** dello spazio di memoria del processo, non l'indirizzo finale.
Allora il limite effettivo sarà base + limite.

File di paging

Di default, Windows utilizza speciali file di paging per **allocare memoria extra sul disco** invece che sulla ram. Anche quando non necessario, il SO può scrivere dati sul disco.
Questa funzione è utile per programmi che utilizzano enormi quantità di ram (come programmi di rendering), ma **può rallentare il sistema** nel caso di utilizzo improprio.

Un'altra funzione è chiamata **cache**. Questa non si riferisce alla memoria interna al processore, ma a **pagine di ram** che, se non necessarie ad altri processi, vengono **conservate** in caso di riallocazione degli stessi processi.

File binari

I programmi risiedono sul disco sotto forma di file binari eseguibili.
L'insieme dei file binari in attesa di essere caricati in memoria per l'esecuzione forma la **input queue**.

Prima di essere eseguiti, i file attraversano diversi stadi:

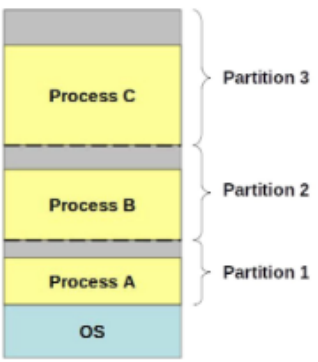
- Gli **indirizzi del programma** sono **simbolici**: es. nome di una variabile
- Gli **indirizzi generati dal compilatore** sono **rilocabili**: relativi alla posizione della prima istruzione
- Il **linker o il loader** generano gli indirizzi **assoluti**.

Questo genera **problemi** quando un processo viene spostato in memoria (es. swap out e swap in).
I **SO moderni** usano **codice dinamicamente rilocabile**: gli indirizzi assoluti non vengono determinati fino a che non sono necessari.
Questo richiede un supporto hardware per il mapping degli indirizzi in real-time (**tabella delle pagine**).

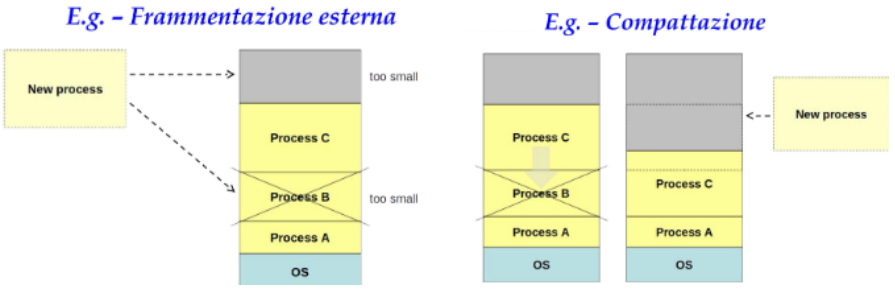
Frammentazione

Esistono due possibili tipi di frammentazione in un sistema:

- **Frammentazione interna:** La memoria allocata può essere leggermente superiore a quella richiesta. La differenza è una partizione di memoria che non viene impiegata completamente.



- **Frammentazione esterna:** E' disponibile lo spazio per soddisfare una richiesta, ma non è contiguo. Si può rimediare tramite **compattazione**.



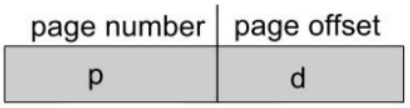
Paginazione

Una soluzione alla frammentazione esterna. Si ottiene consentendo la **non contiguità dello spazio degli indirizzi fisici**, permettendo di allocare la memoria ovunque essa sia disponibile.

Si **divide la memoria in pagine fisiche di dimensione fissa**.
Per eseguire un processo che occupa N pagine, basta trovare N frame liberi per caricare il programma, anche se in posizioni non contigue.
Si usa la **tabella delle pagine** per tradurre gli indirizzi logici in indirizzi fisici.

Si genera **frammentazione interna** relativa solo all'**ultima pagina**, in quanto potrebbe non essere completamente occupata dal processo.

Ogni **indirizzo** viene diviso in **numero della pagina** e **offset nella pagina**.



La frammentazione interna nel **caso peggiore** corrisponde ad avere una pagina finale occupata da un solo byte.
Nel **caso medio** si ha una frammentazione di mezza pagina.

Sembrerebbe che siano preferibili pagine molto piccole, ma in realtà avere più pagine richiede di avere molta memoria per tenerne traccia con la page table.

Esiste un **PTBR** (**Page Table Base Register**), un registro che punta sempre all'inizio della page table.
Il registro **PTLR** (**Page Table Length Register**) indica invece la dimensione della tabella.

Questo approccio richiede **due accessi in memoria**: uno alla page table e uno all'indirizzo richiesto.

Questo si risolve tramite l'uso di registri associativi **TLB** (**Translational Look-aside Buffer**) attraverso i quali si effettua una ricerca parallela su una tabella ristretta (di solito tra 64 e 1024 elementi).
Questi registri si trovano vicino alla cache ma non ne fanno parte.

Se si prova ad accedere ad una pagina che non si trova in ram, avviene un **page fault**.
Il processo viene sospeso fino a che la pagina mancante non sarà in memoria.
Se non c'è memoria libera per il caricamento, si deve eseguire uno swap out di alcune delle pagine allocate.
Per capire quali pagine rimuovere, si tiene conto di

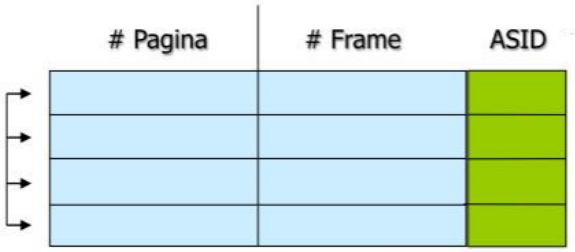
- Numero di accessi
- Tempo dall'ultimo accesso
- Se la pagina è stata modificata

Queste informazioni sono salvate in specifici bit per ogni elemento della page table.

Memoria associativa

Alcune TLB memorizzano un **address space identifier** (**ASID**) per ciascun elemento che permette di identificare univocamente ogni processo e garantire la protezione del suo spazio di indirizzi.

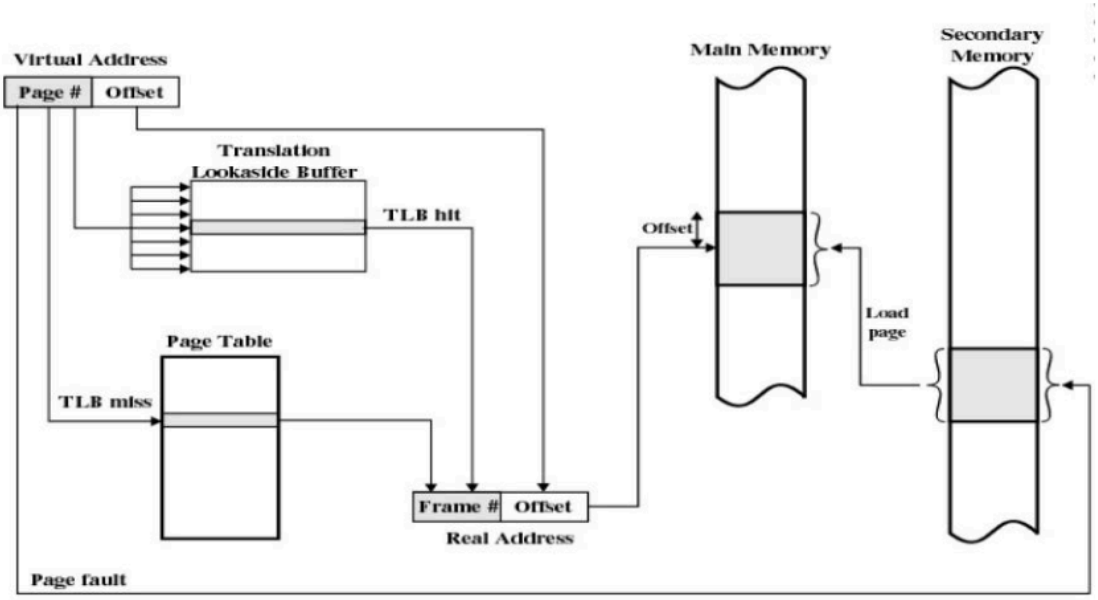
Prima di calcolare l'indirizzo fisico, si controlla la corrispondenza tra l'ASID in esecuzione e quello associato alla pagina virtuale. Se **non corrisponde**, si parla di **TLB miss**.



Usare ASID consente alla TLB di contenere nello stesso momento **elementi di processi diversi**.
In caso contrario, ad ogni cambio di contesto la TLB dovrebbe essere svuotata (**flush**) per evitare che il processo attualmente in esecuzione acceda a informazioni a cui non ha il permesso di accedere.

La conseguenza di un TLB miss è che l'informazione debba essere spostata nella TLB per poterne garantire un accesso rapido in futuro.
Si applicano varie **politiche di sostituzione** degli elementi della TLB.
Alcuni degli elementi sono **vincolati** per garantire sempre un accesso rapido.

Nelle architetture attuali, la **ricerca nella TLB** è **parte della pipeline delle istruzioni**, dunque non causa cali di prestazioni.
La TLB deve però essere piccola (64-1024 voci).
Esistono **TLB separate per istruzioni e dati**, in quanto la loro ricerca viene effettuata in fasi diverse della pipeline.



Protezione della memoria

Si implementa la protezione della memoria tramite alcuni **bit di protezione** associati a **ciascun frame** della tabella delle pagine.
Questi bit determinano se la pagina è accessibile in lettura / scrittura / esecuzione.
Un tentativo di accedere ad una pagina per cui non si hanno i permessi provoca una **trap** al SO

Un altro controllo è il **bit di validità**, che indica se la pagina è effettivamente parte degli indirizzi logici del processo, oppure se l'indirizzo non è legale per quello specifico processo.

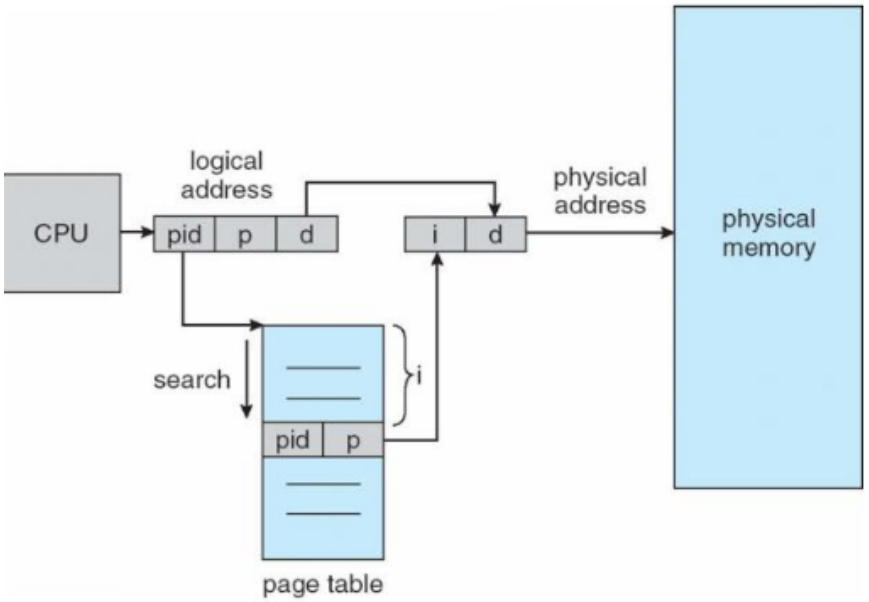
Problema tabella delle pagine

Si associa **una page table ad ogni processo**, che contiene un elemento per ogni pagina virtuale che questo utilizza.
Ogni tabella delle pagine **può occupare molta memoria** (milioni di elementi).

E' quindi **preferibile evitare di collocare la tabella delle pagine in modo contiguo** in memoria centrale.

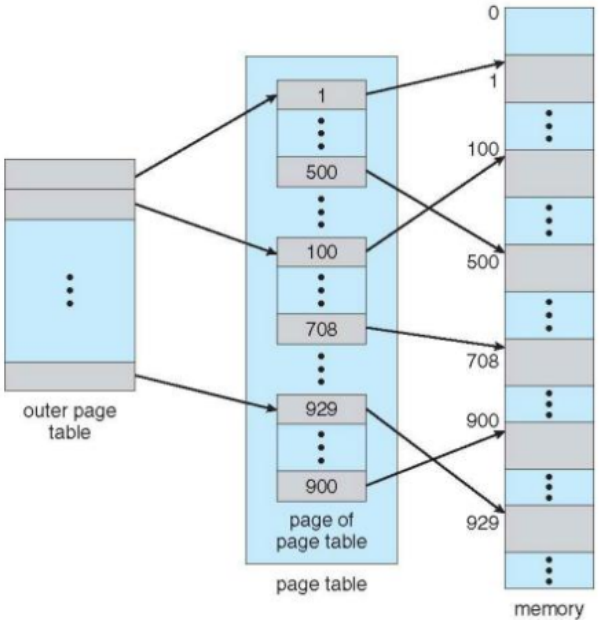
Tabella delle pagine invertita

Una prima **soluzione** al problema della memoria.
Si ha **un elemento** della tabella **per ogni frame** di memoria.
Gli elementi della tabella contengono l'indirizzo della pagina fisica e le informazioni sul processo che possiede tale pagina (PID).
Si **riduce** di molto la **memoria** utilizzata, **a discapito del tempo** di ricerca dell'indirizzo.
E' infatti necessario ricercare su tutta la tabella la corrispondenza <PID, pagina>.

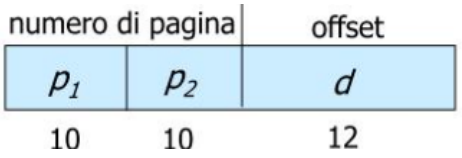


Paginazione gerarchica

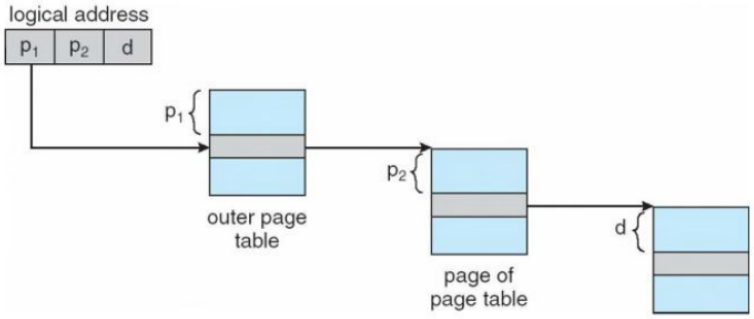
Si divide lo spazio degli indirizzi logici in **più tabelle delle pagine**.
Si usa un algoritmo di **paginazione a due livelli**.
La tabella delle pagine viene essa stessa paginata.



Gli indirizzi saranno allora divisi in tre sezioni:
pagina della tabella esterna → pagina della tabella interna corrispondente → offset.



Per arrivare al dato desiderato, si usano quindi **3 accessi alla memoria** per tabella esterna, tabella interna ed indirizzo effettivo.



Nonostante questo, questa soluzione mantiene prestazioni accettabili data la velocità delle memorie cache moderne.

E' possibile avere **più livelli di paginazione**.
Nelle architetture moderne, si va **dai 3 ai 7 livelli** di paginazione.
L'indirizzo verrà ulteriormente diviso come visto sopra nel caso a 2 livelli.

Per le **architetture moderne a 64 bit**, la paginazione gerarchica è da considerarsi **inadeguata** a causa dei **costi di accesso alla memoria** dati dai molti livelli di paginazione.

Tabella delle pagine hash

E' una soluzione comune per **sistemi a più di 32 bit**.
L'argomento della funzione hash è il numero della pagina virtuale.
Ogni **elemento** della **page table** ha un **bucket** per la gestione delle collisioni.
Ogni **elemento** del **bucket** è composto da

- numero della pagina
- indirizzo del frame fisico corrispondente
- puntatore al successivo elemento del bucket

Si confrontano tutti gli elementi della catena per trovare il frame richiesto.

Per spazi a **64 bit** si usa la **tabella delle pagine a gruppi**.
E' simile ad una tabella hash, ma ciascun elemento della tabella contiene riferimenti alle pagine fisiche corrispondenti ad un **gruppo di pagine virtuali contigue** (**cluster**).
Si riduce la memoria richiesta e si migliorano le prestazioni dato che lo spazio di indirizzi di un programma è contiguo.

Segmentazione

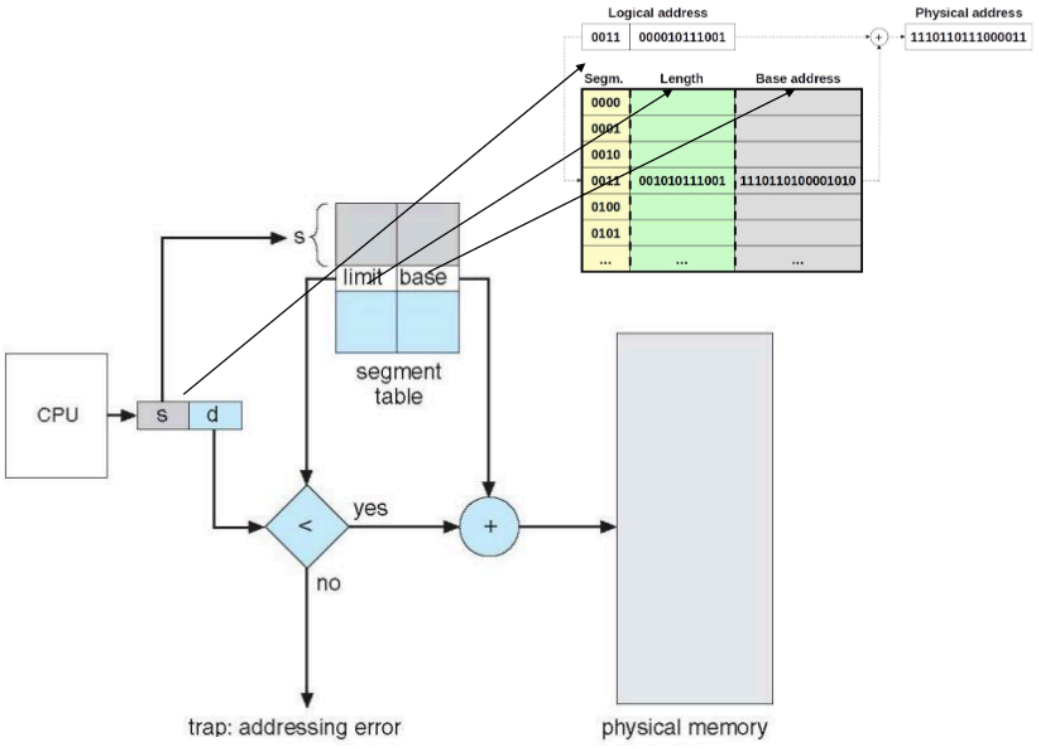
Si divide la memoria in **segmenti di lunghezza variabile**.
Gli elementi si identificano tramite un **offset dall'inizio** del segmento.
Gli indirizzi sono quindi del tipo <numero del segmento, offset>.

Si usa una **tabella dei segmenti** per mappare gli indirizzi.
Ogni elemento contiene:

- **base:** indirizzo iniziale della memoria contenente il segmento.
- **limite:** lunghezza del segmento.

Si usano due registri:

- **Segment Table Base Register (STBR):** punta alla locazione in memoria della tabella dei segmenti.
- **Segment Table Length Register (STLR):** indica il numero di segmenti usati dal programma. Un indirizzo è legale solo se il numero di segmento è inferiore a questo valore.

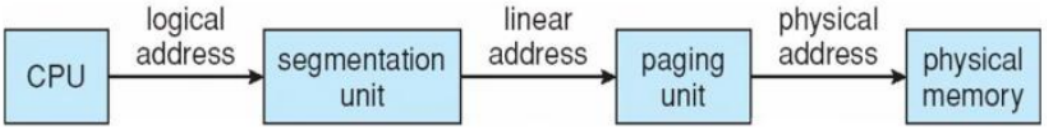


La tecnica della **segmentazione** è **obsoleta** e non più in uso nei sistemi moderni.
Continua però ad essere supportata per ragioni di retrocompatibilità di alcuni software datati.

Intel - segmentazione

La **vecchia** architettura intel era la **Pentium IA-32** a 32 bit.
La **nuova** architettura è invece la **IA-64** da 64 bit.

L'architettura Pentium supportava la segmentazione pura e la segmentazione mista a paginazione.

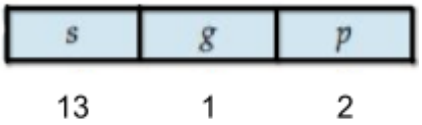


Le **prime versioni** supportavano segmenti da 64KB al massimo.
Al più erano assegnati ad ogni processo:

- **8 mila segmenti privati** nella Local Descriptor Table (LDT).
- **8 mila segmenti condivisi** nella Global Descriptor Table (GDT).

L'**indirizzo** è una coppia **<selettore, offset>**.
Il **selettore** è composto da 16 bit così divisi:

- **s** è il numero del segmento.
- **g** è il bit che indica se si tratta di LDT o GDT.
- **p** sono bit di protezione.



Per garantire la compatibilità fino a 4GB a segmento erano necessari tutti i 32 bit per l'offset.
Dunque l'indirizzo logico era diviso su 2 registri: uno per il selettore e uno per l'offset.
Un'unità di segmentazione interna al processore si occupava di trovare l'indirizzo del segmento tramite il selettore e poi sommarci l'offset desiderato.

Intel - paginazione

L'architettura Pentium prevede 2 taglie di **pagina**: **4KB** o **4MB**.
Per pagine da 4KB, si usa una paginazione a due livelli.
Per pagine da 4MB si usa la paginazione diretta.

Il **limite degli indirizzi a 32 bit** portò Intel a sviluppare il Page Address Extension (**PAE**).

- Paginazione a **3 livelli**.
- I 2 bit più significativi si riferiscono a una tabella dei puntatori alla directory delle pagine.
- Gli elementi della page directory e della page table sono da 64 bit.

Il sistema PAE estendeva lo spazio per l'indirizzo fisico del frame da 20 a 24 bit, permettendo l'utilizzo di fino a 64GB di ram.

Le **architetture moderne a 64 bit** usano indirizzamento a 48 bit.
La dimensione delle pagine può essere di 4KB, 2MB, 1GB.
Si può utilizzare la PAE per estendere gli indirizzi fisici da 48 a 52 bit (fino a 4PB, ovvero 4096 TB).

Memoria virtuale

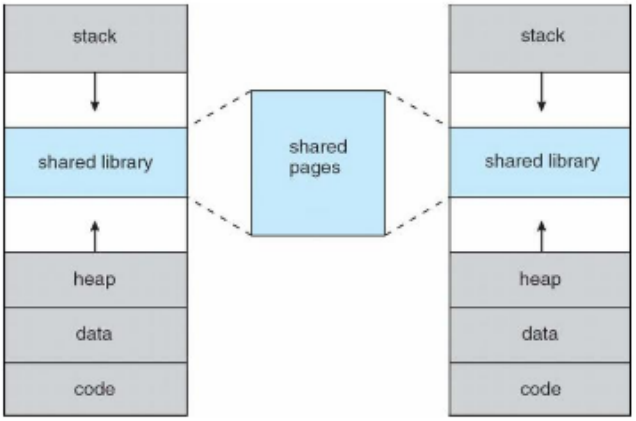
Raramente un programma necessita di risiedere per intero nella memoria centrale.
Molte porzioni di codice non vengono raggiunte dall'esecuzione, o alcuni array sono sovradimensionati rispetto all'utilizzo reale.

La **memoria virtuale** permette di separare la memoria logica da quella fisica.
I programmi non sono quindi vincolati dalla quantità di memoria fisica disponibile.
Inoltre non avendo interi programmi caricati in memoria, ma solo parte di essi, aumenta la produttività del sistema.

Lo **spazio di indirizzi virtuali** di un processo **inizia** all'**indirizzo logico 0** e **finisce** ad un indirizzo logico **max**.
La **MMU** si occupa di effettuare il mapping tra indirizzi logici e fisici.

Lo **heap** ospita la memoria allocata dinamicamente. **cresce dal basso verso l'alto**.
Lo **stack** conserva i vari parametri locali. Inizia da Max e **cresce verso indirizzi bassi.**

Le **librerie dinamiche** occupano **pagine condivise** a più spazi di indirizzo logici, e sono accessibili solo in lettura.



Pager

I processi vengono spostati in ram per intero oppure si può decidere di caricare solo le pagine necessarie quando vengono richieste.
Il modulo del SO che si occupa della paginazione su richiesta è il **pager**.

Normalmente, si usa un **lazy pager** che carica in memoria solo le pagine strettamente necessarie.

Se la memoria è piena, si esegue lo **swap** tra la memoria e il disco.
Questo può essere eseguito tramite **file di swap o partizioni di swap**. In termini di prestazioni conviene la partizione.

Quando un processo viene caricato in memoria, solo le pagine che il pager ritiene necessarie vengono effettivamente caricate.
Quando una nuova pagina è richiesta, è necessario trasferirla velocemente in memoria.
Occorre un meccanismo per distinguere le pagine presenti in memoria da quelle che risiedono ancora nel disco.

Tabella delle pagine

La tabella delle pagine contiene **bit di validità**.
Il bit di validità indica se una pagina risiede o no in memoria centrale.
Inizialmente i bit sono tutti impostati a "invalid".

Il primo riferimento ad una pagina causa un **page fault**.
Il SO consulta una tabella interna delle pagine (indicata nel PCB) per capire se si tratta di:

- riferimento non valido (fuori dallo spazio di indirizzi di quel processo)→ abort.
- pagina non in memoria → caricamento della pagina, aggiornamento del bit di validità.

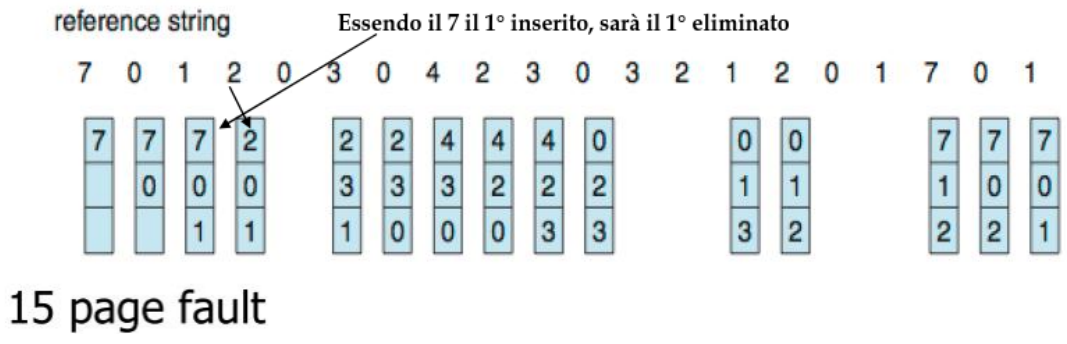
Si utilizza anche un **dirty bit** (**bit di modifica**) per ridurre il sovraccarico dei trasferimenti delle pagine. Solo le pagine modificate vengono riscritte sul disco.
Quando il **pager è inattivo**, si sceglie una **pagina modificata** e la **si copia sul disco**, azzerandone il dirty bit. In questo modo si riduce la probabilità che in caso di page fault sia necessario effettuare copie.

Per garantire l'efficienza del sistema è necessario trovare **algoritmi di sostituzione** che limitino quanto più possibile i page fault in caso di **swapping**.

Per valutare gli algoritmi di sostituzione delle pagine, si usa una **reference string**.
E' una stringa costituita dai numeri di pagina.
I risultati dei test dipendono dal numero di frame a disposizione.

Algoritmo FIFO

Si implementa una coda FIFO relativa alle pagine residenti in memoria.
Si consideri il seguente esempio per 3 frame (3 pagine in memoria per ogni processo):



15 page fault

Questo algoritmo soffre dell'**anomalia di Belady**.
Quando si effettuano richieste cicliche con dimensioni superiori a quella dei frame, si va incontro a numeri di page fault sempre più elevati tanto più alto è il numero di frame.



Algoritmo ottimo (OPT)

E' un algoritmo puramente teorico usato per misurare le prestazioni dei vari algoritmi al confronto con il caso perfetto.

Consiste nel **sostituire la pagina che non verrà utilizzata per il periodo di tempo più lungo**.

Least Recently Used (LRU)

Si **sostituisce** la pagina che è **inutilizzata da più tempo**.
Occorre associare ad ogni pagina il tempo dall'ultimo accesso.



12 page fault: migliore di FIFO ma peggiore di OPT

NOTA: Gli algoritmi OPT e LRU **non soffrono dell'anomalia di belady**.
Sono detti **algoritmi a pila**.
In un algoritmo a pila, l'**insieme delle pagine in memoria per n frame** è un **sottoinsieme dell'insieme delle pagine in memoria per n+1 frame**.
Se consideriamo LRU con n frame, le pagine in memoria in un dato istante saranno le stesse che si avrebbero usando n+1 frame, con l'aggiunta di una pagina extra.

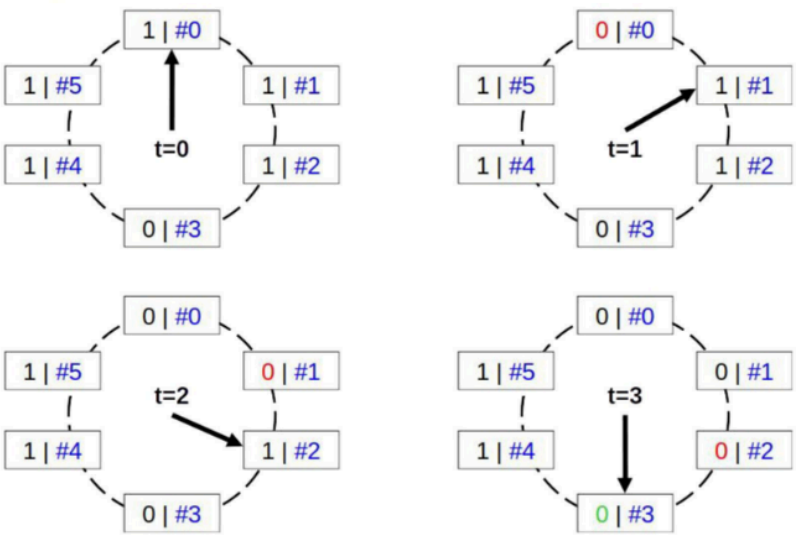
Esistono diversi modi per implementare LRU:

- **Contatore:** Ogni pagina ha un contatore. Quando si fa riferimento ad una pagina, si copia l'orologio nel suo contatore. Per sostituire, si effettua una ricerca lineare sulla tabella delle pagine.
- **Stack:** Si mantiene uno stack con i numeri di pagina utilizzati. Ogni riferimento sposta il numero corrispondente in cima.
- **Bit di riferimento:** Ciascuna pagina ha un bit inizialmente a 0. Fare un riferimento cambia il bit a 1. Si registrano i valori del bit di riferimento a intervalli regolari in un registro a scorrimento.
  La pagina "1101001" è stata usata più recentemente di quella "0110010".

Algoritmo seconda chance

Si parte da un algoritmo di base FIFO.
Quando una pagina viene scelta per essere sostituita, si guarda il suo **bit di riferimento**:

- Se è **1**, viene data una seconda chance (il bit viene azzerato) e si passa alla successiva scelta per la sostituzione.
- Se è **0**, la pagina viene effettivamente sostituita.



Una **versione migliore** di questo algoritmo guarda la **coppia <bit riferimento, bit modifica>.**

Si ottengono **4 classi**:

- (0,0): la pagina non è stata recentemente usata o modificata. Ideale per la sostituzione.
- (0,1): la pagina non è stata usata di recente ma necessita di essere copiata sul disco prima di sostituirla.
- (1,0): la pagina è stata usata di recente ma non necessita di copia sul disco.
- (1,1): la pagina è stata usata di recente e necessita la copia sul disco.

Si seleziona per la sostituzione la prima pagina della classe minima disponibile.

Algoritmi con conteggio

Si tiene traccia del numero di accessi fatti su ciascuna pagina.
Possono essere del tipo LFU o MFU (Most Frequently Used).
L'**implementazione** è **costosa** e le **prestazioni** sono **scadenti**.

Pool di frame liberi

Si mantiene un pool di frame liberi da usare come "cache".
Quando si verifica un page fault, si sposta la pagina dal disco ad uno dei frame liberi, senza aspettare che il frame vittima sia stato copiato sul disco.

Nel caso di page fault, il sistema controlla se il frame richiesto è già presente nel pool di frame liberi. In tal caso si salta la procedura di swap in e non sono richieste operazioni di I/O.

Altri metodi di sostituzione

Allocazione con priorità: Se un processo genera un page fault, viene sostituito uno dei suoi frame oppure il frame di un processo con priorità minore.

Sostituzione globale: Il SO seleziona un frame da sostituire tra tutti i frame presenti. E' il sistema usato dalla maggior parte dei SO.

Sostituzione locale: Il frama da sostituire deve essere tra i frame del processo stesso. E' possibile un sottoutilizzo della memoria.

Working set

Una **località** è un insieme di pagine che vengono accedute insieme, e sono quindi in uso contemporaneamente.
Una **finestra di working set** è un numero fissato di riferimenti a pagine.
Un **working set** è il numero di pagine referenziate nella finestra più recente.

- Se la finestra è troppo piccola non comprende l'intera località.
- Se la finestra è troppo grande comprende più località
- Se è infinita comprende tutto il programma.

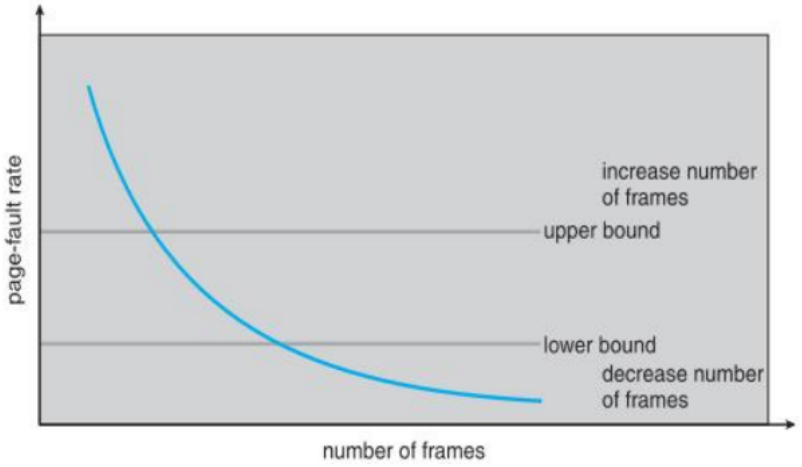
Se il numero di pagine richieste (la somma di tutti i working set) è maggiore del numero di frame disponibili, il sistema va incontro a **thrashing**, ovvero è costantemente occupato a eseguire swapping di pagine.
E' necessario sospendere o swappare uno o più processi per fare spazio.

Frequenza di page fault

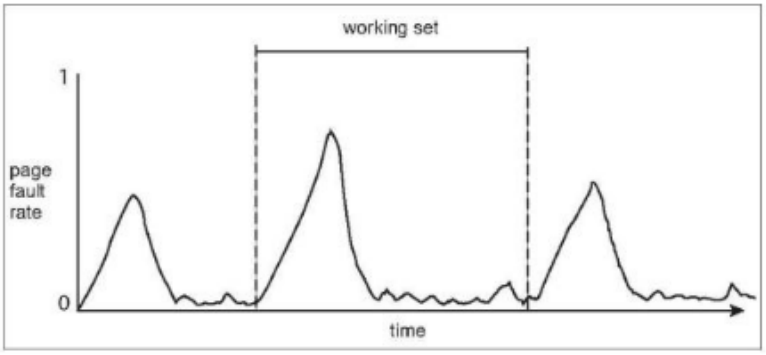
Si stabilisce una **frequenza accettabile di page fault.**

Se un processo è sopra la frequenza, prende altri frame.

Se è sotto la frequenza, cede alcuni frame ad altri processi.



Esiste una **connessione** tra la **frequenza di page fault** di un processo i il suo **working-set**.
Quando si verifica un cambio di località, si nota un picco nel grafo della frequenza.



Mappatura di file

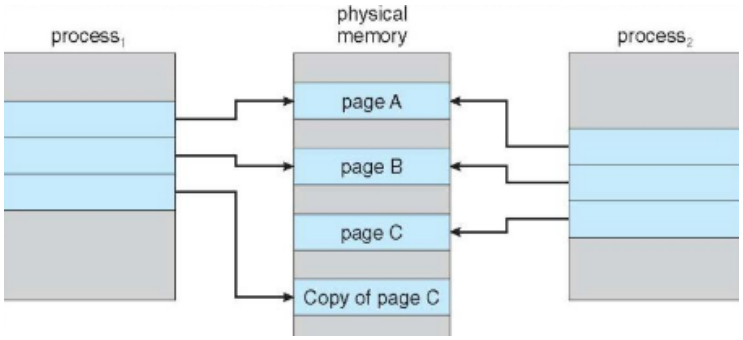
Si tratta di una procedura che permette di **trattare l'accesso ai file come un normale accesso alla memoria**.
Si realizza associando un blocco del disco ad una serie di pagine in memoria.

L’accesso iniziale provoca un page fault.
A questo punto, la pagina corrispondente alla porzione di file richiesta viene caricata in una pagina di memoria.
Le successive operazioni di lettura / scrittura verranno trattate come ordinari accessi in memoria.

Questo permette di **risparmiare moltissime system call**.

Copy on write

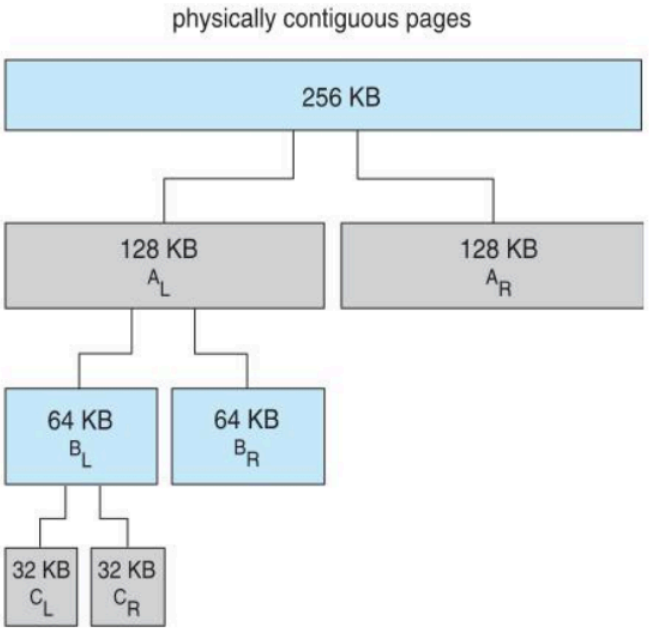
Questo metodo consiste nel posticipare quanto più possibile la copia di una pagina della memoria.
Se dopo un fork() il padre e il figlio non modificano una pagina, essa rimane condivisa tra i due processi.
Nel momento in cui uno dei due esegue una modifica, essa verrà copiata.



Memoria del kernel

Il kernel non utilizza la stessa memoria che viene riservata ai processi utente.
Il kernel richiede memoria per strutture dati di dimensioni variabili.
Spesso il codice e i dati del kernel non sono soggetti a paginazione.
Alcune parti della memoria del kernel devono essere contigue, in quanto alcuni dispositivi accedono direttamente all’interfaccia fisica, senza passare per indirizzi logici.

Esistono vari modi per allocare memoria per il kernel:
Allocatore potenza di 2: Alloca la memoria in blocchi contigui di dimensione pari ad una potenza di 2. Quando il kernel richiede un quantitativo di memoria, ne viene allocata la potenza di 2 più vicina.



Allocazione a lastra:

- Una **lastra** è composta da uno o più frame contigui.
- Una **cache** è composta da più lastre contigue (nelle nuove versioni di Linux questa contiguità non è richiesta).
- Ogni struttura dati del kernel ha una sua cache assegnata (PCB, semafori, ...).
- Ciascuna cache è popolata da istanze della relativa struttura dati.

Il kernel crea e distrugge spesso questi oggetti. Per ridurre i costi in termini di CPU, il sistema mantiene in memoria questi oggetti invece di distruggerli, mantenendoli pronti per future richieste.

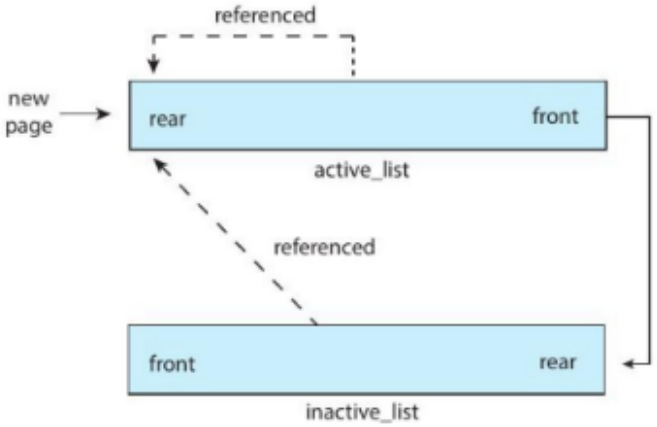
Gestione delle pagine in Linux

Attualmente, Linux adotta l'**allocazione a lastre** per la **memoria del kernel** (e per alcune strutture dati utente).
Alcune distribuzioni recenti usano ulteriori allocatori di memoria:

- **SLOB (Simple List Of Blocks):** mantiene tre liste per oggetti piccoli, medi, grandi. Usato per sistemi embedded con poca memoria.
- **SLUB:** nuova versione dell'allocatore a lastre (SLAB), ottimizzata per multiprocessore.

Per quanto riguarda la **memoria utente**, si mantengono **2 liste: active_list e inactive_list**.
Ogni pagina ha un **bit di accesso** che viene impostato a 1 ad ogni accesso.
Periodicamente, i bit di accesso vengono resettati.

La pagina utilizzata meno di recente finisce all'inizio della active list, da cui può essere spostata nell'inactive list.
Le due liste vengono mantenute bilanciate. Se quella attiva diventa troppo grande, le pagine in testa vengono declassate alla lista inattiva, e quindi rese disponibili per il recupero della memoria.



Per ridurre il numero di page fault necessari allo startup di un processo si usa il **prepaging**: si portano in memoria alcune o tutte le pagine necessarie al processo prima che esse vengano referenziate.
Questa tecnica è utile se le pagine effettivamente utilizzate tra quelle caricate sono molte, altrimenti si spreca memoria.

Dimensione delle pagine e pagine bloccate

Si seguono alcuni criteri per decidere la dimensione delle pagine:

- **Dimensione** della tabelle delle pagine.
- **Overhead** di I/O.
- Numero di **page fault**.
- **Dimensione** della TLB.
- Frammentazione.

Si dice **TLB reach** la quantità di memoria accessibile tramite TLB.
Idealmente, il **working set** di un processo dovrebbe essere contenuto interamente nella TLB per evitare page fault.

Aumentare la dimensione delle pagine provoca **frammentazione**, perché non tutte le applicazioni richiedono grandi pagine.
Prevedere **pagine di diversa dimensione** permette alle applicazioni che le richiedono di usare pagine molto grandi, senza provocare frammentazione per le altre.

Talvolta, occorre avere alcune pagine **bloccate in memoria**, in modo che esse non siano accidentalmente rimosse da un algoritmo di sostituzione.
Il kernel blocca in memoria principale tutte le sue funzioni vitali.

Solitamente si implementa tramite un bit di controllo, il **lock bit**.

Gestione della memoria in WIndows

La memoria kernel di divide in due categorie:

- **Paged pool:** memoria paginabile, può essere salvata in un file di paging sul disco. Usata per dati del kernel che non devono essere immediatamente disponibili.
- **Nonpaged pool:** memoria non paginabile, usata per le parti più importanti del kernel che non possono mai essere rimosse dalla memoria principale.

Windows impiega la **paginazione su richiesta via clustering**, ovvero cerca in memoria non solo la pagina richiesta, ma anche quelle ad essa adiacenti.

Ad ogni processo viene associata una dimensione minima ed una massima del working set.
Il **working set minimo** è la quantità di pagine che sono sempre caricate in memoria dal SO.
In caso di disponibilità in memoria, possono essere allocate altre pagine fino alla dimensione del **working set massimo**.

SI utilizza anche la **compressione della memoria** per ridurre la dimensione di un programma inutilizzato in memoria. Questo aggiunge overhead per la decompressione, ma comunque meno del file di paging.

Gestione file - Interfaccia del file system

Il sottosistema di gestione dei file è la parte più visibile di un sistema operativo.
Fornisce i meccanismi per la **registrazione**, l'**accesso** e la **protezione** di dati e programmi del SO e dei suoi utenti.

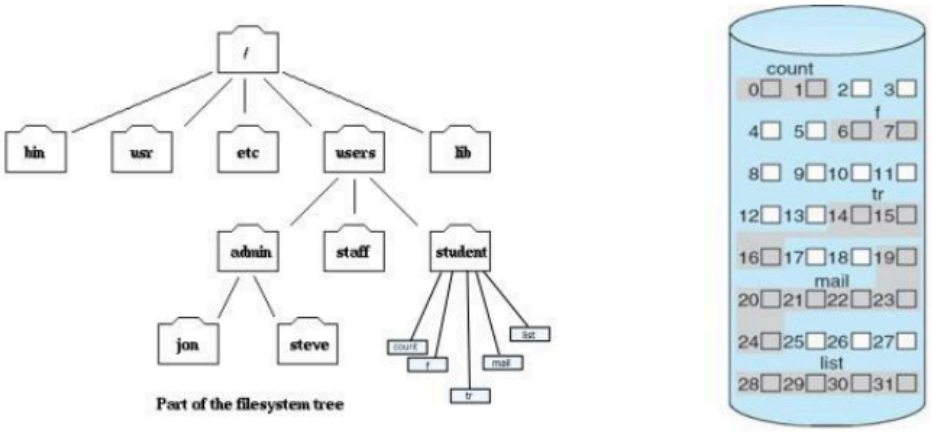
Anche il file system **virtualizza** i dispositivi di memorizzazione permanente.
Ci sono due livelli:

- **Visione utente:** interfaccia, ciò che offre la macchina virtuale.
- **Implementazione:** principi realizzativi architetturali.

Un sistema di calcolo può usare **diversi media** per registrare le informazioni, come dischi rigidi, dischi ottici, ssd, memorie flash, etc...
Il compito del SO è di fornire una visione logica e un'interfaccia uniforme per l'accesso a tali dispositivi.

File system

La struttura logica con cui il SO gestisce i file prende il nome di **file system**.
Tutti i SO moderni gestiscono l'archiviazione file in **modo gerarchico** (**HFS, Hierarchical File System**).
Questa organizzazione non ha alcun legame con la posizione fisica effettiva del file in memoria di massa.



Il file system è responsabile della gestione dei file in memoria di massa.
Fornisce all'utente una serie di **funzioni** per operare su di essi.
Il file system garantisce una **gestione** dei file **indipendente dalle caratteristiche fisiche** dei dispositivi che vengono usati per immagazzinarli.
Questo è vantaggioso sia per l'utente che per i programmi.

Attributi dei file

Nome: identificativo del file per l'utente.
Identificativo: etichetta unica che identifica il file all'interno del file system.
Tipo: necessario per sistemi che supportano tipi differenti.
Locazione: puntatore al dispositivo e alla posizione del file in tale dispositivo.
Dimensione.
Protezione: parametri di controllo per l'accesso in lettura / scrittura / esecuzione.
Ora, data, utente: dati necessari alla sicurezza del sistema per il controllo d'uso.

Alcuni file system più recenti supportano anche **attributi estesi**, ovvero alcune informazioni aggiuntive associate ad un file. Possono essere di qualsiasi tipo e fornire informazioni aggiuntive ad un'applicazione, al SO o all'utente.
Queste informazioni non sono parte del file ma sono salvate separatamente.
Ad esempio, un attributo aggiuntivo è il **checksum**, una stringa alfanumerica utilizzata per verificare l'integrità dei dati di un file.

Operazioni sui file

Un file è un tipo di dato astratto su cui devono essere definite alcune operazioni:

- **Creazione:** Reperire lo spazio in memoria per memorizzare il file, creare un nuovo elemento nella directory per memorizzare il nome, la posizione e tutti gli altri attributi.
- **Scrittura:** Chiamata al sistema con nome del file e dati (o puntatore ad essi) da inserire come parametri. Il file viene reperito dal file system e i nuovi dati vengono scritti nella posizione indicata dal **puntatore di scrittura**, che viene successivamente aggiornato.
- **Lettura:** Chiamata al sistema con nome del file e indirizzo in memoria. Viene reperito nel file system e vengono prelevati i dati nella posizione indicata dal **puntatore di lettura,** che viene poi aggiornato.

Solitamente, il SO mantiene un **puntatore unico per lettura e scrittura**.

- **Cancellazione:** Reperimento del file dal file system, eliminazione dell'elemento dalla directory e rilascio della memoria allocata.
- **Troncamento:** Cancellazione del contenuto del file, che mantiene immutati tutti gli attributi tranne la sua dimensione.
- **Impostazione degli attributi:** Reperimento e modifica dell'elemento della directory.

Altre operazioni si ottengono mediante la **combinazione di operazioni di base**.
Ad esempio, la copia di un file viene effettuata tramite creazione, lettura e scrittura su file.

Quasi ogni operazione su file richiede di individuarlo e aprirlo.
Il SO mantiene una tabella in memoria centrale contenente informazioni su tutti i file aperti: la **tabella dei file aperti**.

Tabella dei file aperti

Quando si richiede una qualsiasi operazione su un file, si ricercano le relative informazioni tramite un puntatore in questa tabella.
Quando un file non è in uso attivo, deve essere chiuso e il suo elemento viene rimosso dalla tabella.

Esistono due **chiamate a sistema** per aprire e chiudere i file: **Open(F)** e **Close(F)**.

Nei **sistemi multiutente** esistono due livelli di tabella:

- **Tabella di sistema:** contiene i riferimenti ad ogni file aperto nel sistema, come posizione, dimensione, date di accesso e modifica, etc...
- **Tabella associata al processo:** contiene i riferimenti a tutti i file aperti da un processo e un puntatore alla posizione corrente.

Sia WIndows che Linux utilizzano una struttura a due livelli per la gestione dei file aperti.
Questo serve per garantire la **condivisione dei file tra processi** in maniera flessibile.
Se due processi aprono lo stesso file in lettura, ci si aspetta che l'avanzamento del puntatore in lettura A non provochi anche l'avanzamento di quello di B, motivo per cui esistono due voci diverse nella tabella di sistema.
Nel caso di un fork(), però, solitamente se il padre legge, anche il puntatore del figlio deve avanzare. E' quindi possibile condividere il puntatore.

Sono memorizzati nella tabella:

- **Contatore di aperture:** tiene conto di quanti processi hanno aperto il file. Questo permette l'opportuna rimozione del file dalla tabella dopo la chiusura da parte dell'ultimo processo.
- **Locazione del file su disco:** cache delle informazioni di accesso ai dati permanenti.
- **Puntatore alla posizione corrente del file:** puntatore alla locazione dell'ultima operazione di lettura / scrittura per ogni processo che ha aperto il file.
- **Diritti di accesso:** controllati dal SO per permettere o negare le operazioni sul file.

I file possono essere associati ad un **lock**.
Il lock può essere **shared** se il file può essere aperto contemporaneamente da più processi, o **exclusive** altrimenti.
Tale lock può essere **obbligatorio o consigliato**.
Se è obbligatorio (Windows), l'accesso al file viene negato se il lock è già acquisito.
Se è consigliato (UNIX), i processi vedono che lo stato del file è bloccato e decidono sul da farsi. E' compito del programmatore garantire la corretta acquisizione e cessione del lock.

Tipi di file

Il tipo è un attributo del file che ne indica la **struttura logica interna** e permette di interpretarne correttamente il contenuto.

Si usano diversi meccanismi per identificare i tipi di file:

- **Estensione:** il tipo è indicato dal suffisso (DOS)

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com-pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

- Attributo associato al file nella directory (MAC OS).
- **Magic number:** il tipo è rappresentato da un valore posto all'inizio del file (UNIX).

Il tipo di file e la corrispondente struttura logica possono essere riconosciuti e gestiti in modi diversi dal SO.
Un sistema che gestisce **molti formati** ha un **codice molto complesso**, ma una gestione efficiente. Al contrario, un sistema che ne gestisce **pochi** ha un **codice più snello**, ma la gestione è affidata ai programmatori.

Un problema legato ai file è l'**abuso di posizione dominante**.
Se grandi aziende inserissero all'interno dei SO più popolari i codec multimediali (software per la gestione di video, audio e immagini), si rischierebbe l'esclusione dal mercato di alcune aziende minori (VLC, etc...)

Record logici

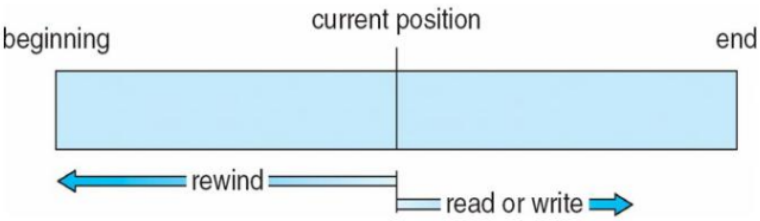
La **dimensione dei blocchi** del disco, detti record fisici, è **fissata**.
Occorre risolvere il problema della corrispondenza tra record logici e fisici (**packing**).

Una parte dell'ultimo blocco fisico che contiene un file rimane vuoto: **frammentazione interna.**

Accesso ai file

Esistono due modalità di accesso:

- **Accesso sequenziale:** E' un modello di accesso ispirato ai nastri. Lo utilizzano compilatori e editor. Impossibile leggere oltre l'ultima posizione scritta. La scrittura aggiunge informazioni in fondo al file.
  Viene trattato un gruppo di byte alla volta. Il puntatore indirizza il record corrente e avanza ad ogni lettura / scrittura.
  La lettura può avvenire in ogni posizione del file, purché sia raggiunta sequenzialmente.
  La scrittura avviene solo in fondo al file (append).

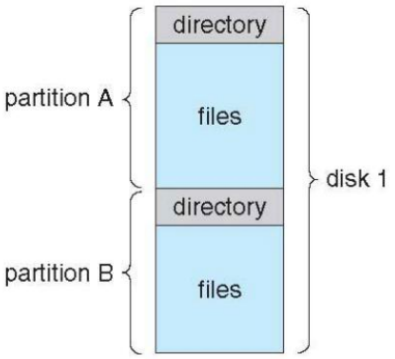


- **Accesso diretto:** E' un modello che si ispira al disco. Si opera su un record di dati in posizione arbitraria nel file.
- **Accesso indicizzato:** Può essere realizzato sulla base del metodo di accesso diretto. Implica la costruzione di un **file indice** per l'accesso al file.

L'indice contiene i puntatori ai blocchi del file. Per reperire un elemento, occorre cercare nell'indice il blocco necessario e usare il puntatore trovato per accedere ai dati.
Questo metodo è ispirato ai database.

Struttura del disco

Il disco è suddiviso in **partizioni** o **minidischi**.
Ciascuna partizione contenente un file system è detta **volume**. Ha quindi una directory di dispositivo che contiene informazioni su tutti i file contenuti.
Il volume può essere un pezzo di disco, un disco intero, o addirittura più dischi collegati in RAID.



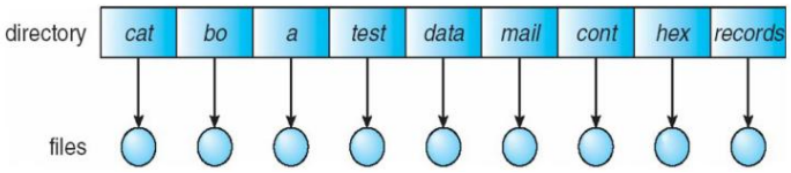
Operazioni sulle directory

- **Ricerca:** Possibilità di scorrere una directory per reperire l'elemento associato ad un file.
- **Creazione di un file:** Aggiunta del record descrittivo del nuovo file alla directory.
- **Cancellazione di un file:** Rimozione del record descrittivo di un file nella directory.
- **Elenco dei contenuti di una directory:** Possibilità di elencare tutti i file nella directory e il contenuto associato ad ogni puntatore ad essi.
- **Attraversamento del file system:** Possibilità di accedere ad ogni directory e ad ogni file in esse contenuto.

L'organizzazione della struttura delle directory deve garantire **efficienza** nel reperire file rapidamente, **nominazione** conveniente per gli utenti (ad esempio due utenti possono avere un file con lo stesso nome) e **grouping**, ovvero raggruppamento dei file sulla base di proprietà logiche.

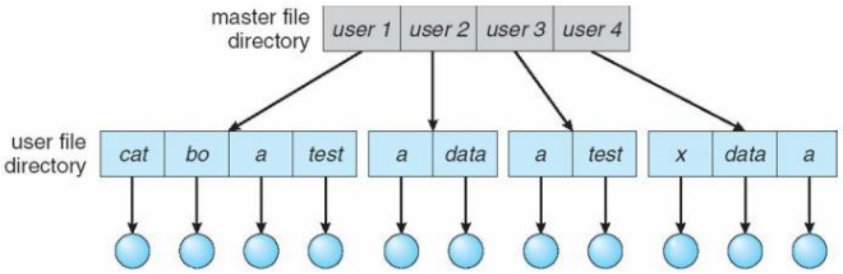
Directory monolivello

Si usa una **directory unica** per tutti gli utenti.
Questo metodo ha problemi di nominazione e nessun raggruppamento logico.



Directory a due livelli

Si ha **una directory per ogni utente**.
Sono ammessi nomi uguali per file di utenti diversi.
La ricerca è efficiente.
Non c'è ancora la capacità di raggruppamento logico.



Sistemi multiutente

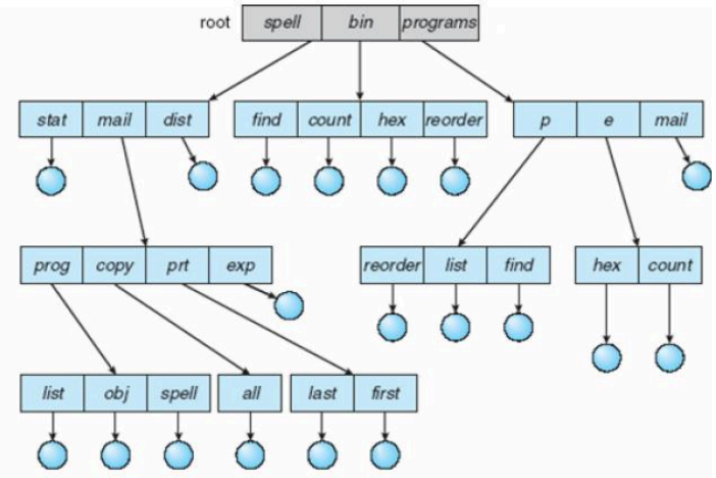
Quando si apre la sessione di lavoro, si ricerca l'identificativo dell'utente nella **MFD** (**Master File Directory**). L'utente viene quindi ammesso al sistema all'interno della propria **UFD** (**User File Directory**).

Ogni riferimento a file da parte dell'utente viene interpretato dal SO come esclusivamente correlato ai file presenti nella relativa UFD.
Per riferirsi ai file di altri utenti, se l'accesso è autorizzato, è necessario usare il pathname completo.
I file di sistema sono raggruppati in directory universalmente accessibili.

Directory con struttura ad albero

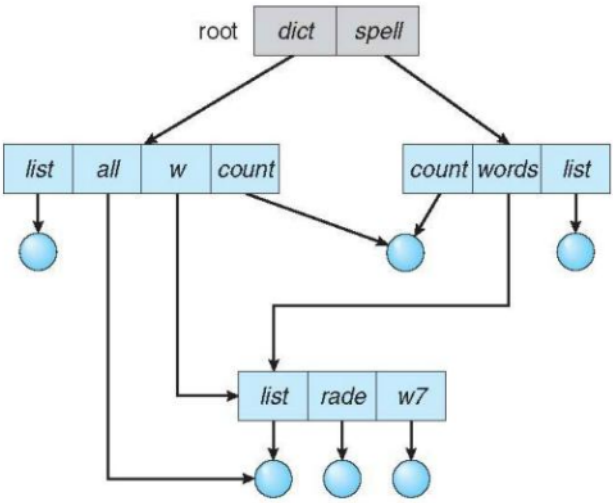
La ricerca è efficiente, esiste la capacità di raggruppamento logico e si introduce il concetto di **working directory**, ovvero la directory di lavoro attuale.

I riferimenti ai file vengono considerati nella directory corrente (**path relativo**). Per operare su file in altre directory è necessario usare il pathname completo (**path assoluto**).



Directory a grafo aciclico

Sono presenti file e sottodirectory condivise.
Si introduce il concetto di **aliasing**, ovvero nomi diversi possono essere usati per riferirsi ad uno stesso oggetto.
La condivisione avviene tramite **soft link** (collegamenti) o **hard link** (duplicazione dell'elemento di directory).

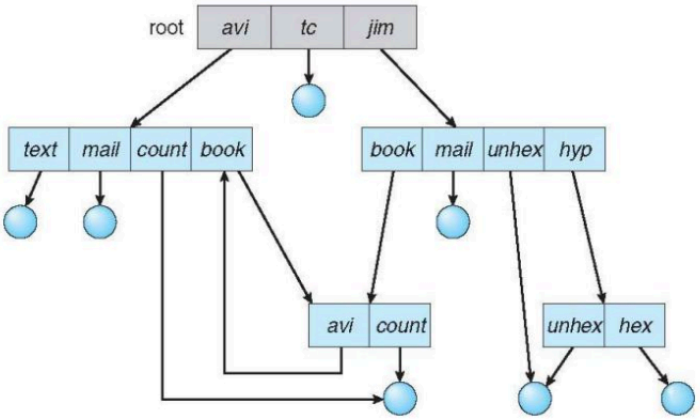


Emergono **problemi** nel momento dell'**eliminazione** di un file.
Esistono due soluzioni:

- Puntatori all'indietro che permettono di reperire tutti i link al file eliminato e rimuoverli.
- Conservazione del file fino all'eliminazione di tutti i link ad esso associati.

Directory a grafo generale

Sono ammessi link tra directory che generano un ciclo.



Mantenere un grafo aciclico garantisce la semplicità degli algoritmi di ricerca.

Si ottiene l'aciclicità ammettendo link ai file ma non alle sottodirectory.
Alternativamente, si usa un algoritmo di individuazione di cicli (molto costoso).

Se un grafo è un **DAG** (Grafo diretto aciclico), allora ha sicuramente almeno un ordinamento topologico.
E' possibile verificare la presenza di un ordinamento topologico anche se il grafo non è un DAG. Se l'algoritmo non individua alcun ordinamento, il grafo di partenza conteneva almeno un ciclo diretto.

Gestione cicli

Se si ammette la presenza di cicli, occorre marcare ciò che è raggiungibile ed è già stato visitato per evitare copie multiple o problemi simili.

In caso di cicli il contatore dei riferimenti ad un file può essere non nullo anche se il file non ha più alcun riferimento.
Il sistema di **garbage collection** esplora il file system per controllare quali file siano accessibili. Tutto ciò che non viene marcato viene spostato nell'elenco dei blocchi liberi.

Montaggio del file system

Prima di accedervi, è necessario **montare** il file system.
Ogni file system unmounted può essere montato in un **mount point** prescelto.

La **procedura di montaggio** è la seguente:

- Si fornisce al SO il nome del dispositivo da montare e la locazione che dovrà assumere nel file system.
  Solitamente, il punto di montaggio è una directory vuota a cui sarà agganciato il file system da montare.
- Una volta montato, il file system risulta accessibile a programmi e utenti in modo trasparente ed entra a far parte del grafo.
- La directory in cui si effettua il mount può essere non vuota, ma in quel caso i suoi contenuti rimarranno inaccessibili fino all'unmount.

I sistemi Windows e MacOS rilevano e montano in automatico tutti i file system.
Nei sistemi UNIX-like, i file system vanno montati manualmente.
In Linux, il file /etc/fstab contiene le configurazioni di montaggio.

Condivisione di file

La **condivisione** di file è particolarmente utile in **sistemi multiutente**.
Tuttavia è richiesto uno schema di protezione che garantisca un controllo di accesso ai file mediato dal SO.

Un metodo molto diffuso per la condivisione dei file è il **Network File System** (**NFS**).

Si usa di solito un modello legato al concetto di **proprietario** del file e di un **gruppo** di utenti a cui il proprietario è affiliato.
Il proprietario è colui che ha creato il file e può modificarne gli attributi.
Il gruppo si usa per definire un sottoinsieme di utenti autorizzati a condividere l'accesso al file.

Gli ID di proprietario e gruppo sono contenuti tra gli attributi del file.

File system remoti

Si sfrutta la **rete** per ottenere l'accesso a file residenti su sistemi remoti.
Esistono diversi modi per implementarli:

- Il **trasferimento** viene richiesto esplicitamente via **protocollo FTP**.
- Si usa un **file system distribuito** che permette l'accesso automatico del calcolatore locale a directory remote.
- Si effettua l'accesso tramite un **browser** attraverso il **world wide web**.

Nel caso del **modello client-server** si ha un server centrale che mette a disposizione risorse (file o directory) ai client che ne fanno richiesta.
Tali modelli sono solitamente **molti-a-molti**, ovvero un server può gestire più client e un client può accedere a più server.

Tale soluzione soffre di alcuni **problemi** legati all'**autenticazione.** Protocolli insicuri possono condurre a **spoofing** (falsificazione dell'identità).
La sicurezza si ottiene tramite chiavi di cifratura per l'autenticazione reciproca tra server e client.

I **file system locali** possono essere soggetti a **malfunzionamenti**:

- Problemi hardware ai dischi.
- Alterazione dei metadati.
- Problemi ai controllori dei dischi.
- Problemi ai cavi di connessione.
- Errore umano.

Nei **file system remoti** i **malfunzionamenti** possono dipendere dalla caduta della rete o dei server remoti.
Per **ripristinare** tali malfunzionamenti è necessario mantenere alcune **informazioni di stato** sia dal lato client che da quello server.

Tuttavia, alcuni protocolli moderni, come NFS, non mantengono informazioni di stato.
Il server dimentica il client subito dopo aver risposto alla richiesta.
La responsabilità del ripristino passa totalmente dal server al client.

Semantica della coerenza

La **semantica della coerenza** specifica quando le modifiche effettuate ad un file da un utente possano essere osservate dagli altri utenti.

Tale semantica è correlata agli algoritmi di **sincronizzazione tra processi**, pur essendo meno critica a causa delle lunghe latenze e bassa velocità di trasferimento dei dischi e della rete.

Nei sistemi UNIX, le scritture in un file aperto devono essere visibili immediatamente agli altri utenti.

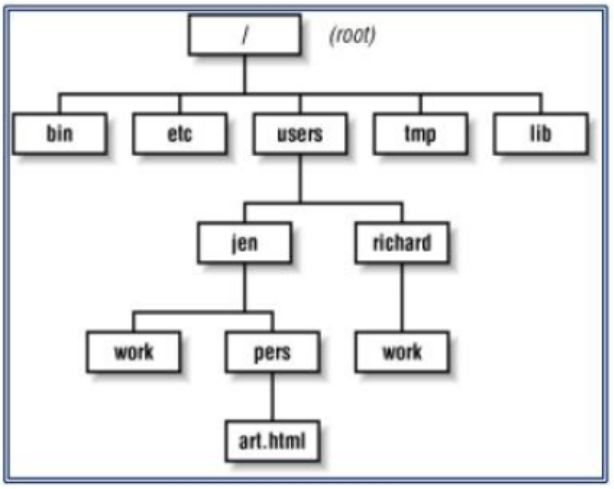
Unix / Linux

I file e le directory possono appartenere a più directory, dunque la struttura è a **grafo generale**.

Varie **directory** ospitano **programmi di sistema**, ad esempio dev per i driver, bin per il codice eseguibile, include per le librerie del sistema, etc...

A **ciascun utente** è associata una **home directory**, generalmente sottodirectory di usr. I file creati da un utente sono contenuti nella sua home.

Si usa un sistema di **Copy On Write** (**COW**), che permette a due processi padre e figlio di condividere alcune pagine di memoria comune. Nel momento in cui uno di essi ne modifica una, questa viene effettivamente copiata.



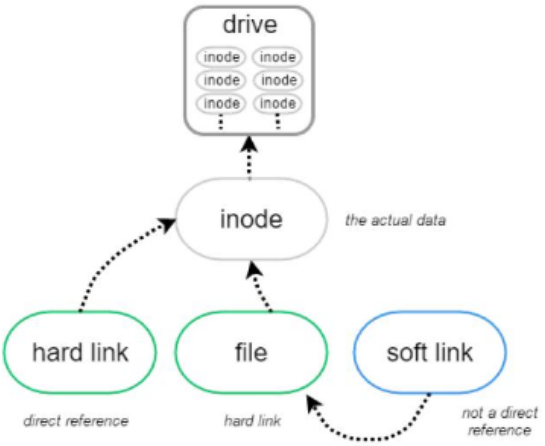
Ciascun file viene identificato univocamente da un **pathname** che include l'intero cammino dalla radice dell'albero al file.
Poiché il pathname è **unico**, file e sottodirectory presenti in una stessa directory devono avere nomi distinti.

Un utente che interagisce con il file system ha un proprio **contesto**, ovvero una posizione nel file system (**working directory**).
Al collegamento nel sistema, l'utente si trova nella sua home directory.
Il contesto può essere variato muovendosi nel grafo (comando cd).
Quando ci si riferisce ad un **file nella working directory**, si omette il suo percorso. Si parla allora di **path relativo**.

Si definisce **hard link** un'etichetta o un nome alternativo assegnato ad un file.

Si possono avere nomi distinti che identificano lo stesso file.
I comandi eseguiti su uno qualsiasi di essi agiscono sullo stesso contenuto.
La rimozione può essere effettuata anche agendo sull'hard link.

Un **soft link** è invece un tipo di file che punta ad un altro file.
Non contiene i dati relativi al target, ma punta ad un elemento di directory da un'altra parte nel file system.
Quando si rimuove un file, i soft link diventano inutilizzabili, mentre gli hard link preservano il file fino a che ne esiste almeno uno.



Quando si crea un file, si specifica la **protezione** da applicare ad esso. In UNIX, si sceglie quali permessi tra lettura / scrittura / esecuzione assegnare a **user, group, others**.
Si usa il comando **chmod** per modificarli.
Questi sono i valori possibili per chmod (777 equivale a ogni permesso dato a tutti).

User			Group			Others		
R	W	X	R	W	X	R	W	X
4	2	1	4	2	1	4	2	1

--- → nessun accesso
r-- → puoi vedere i nomi dei file, ma non puoi entrare e non puoi accedere ai file
--x → puoi entrare nella directory: accesso ai file se conosci i nomi ma non puoi fare ls
rw- → quasi inutile: vedi i nomi, ma non puoi entrare né modificare
r-x -> puoi vedere i file e puoi entrare e accedere
-wx → puoi modificare se conosci i nomi ma non puoi fare ls

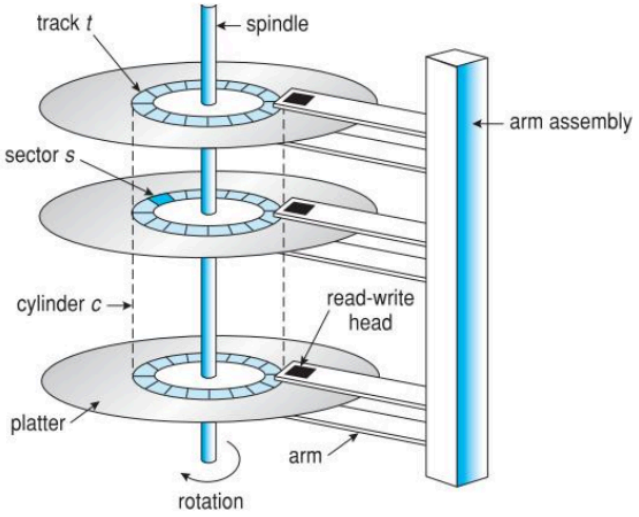
Memoria secondaria - HDD

Negli attuali sistemi di calcolo, i **dischi ottici** rappresentano il mezzo principale per la memorizzazione di dati.

Si dice **velocità di trasferimento** la velocità con cui i dati fluiscono dal disco alla ram.

Si dice **tempo di posizionamento** il tempo necessario a spostare il braccio del disco in corrispondenza del cilindro desiderato (**seek time**) + il tempo necessario affinché la testina raggiunga il settore desiderato (**latenza di rotazione**).

Il **crollo della testina**, che normalmente è sospesa su un cuscinetto d'aria di pochi micron, corrisponde all'impatto della stessa sulla superficie del disco.
L'ultima tecnologia per l'**HDD** è la tecnologia **HAMR**, che utilizza il **calore** per superare alcune limitazioni fisiche della registrazione magnetica tradizionale.
Riscaldando temporaneamente il supporto durante la scrittura si riduce la coercitività del materiale e si riesce ad inserire dati in aree più piccole aumentando la densità di archiviazione.
Questa tecnologia prevede l'inserimento di un **diodo laser** all'interno della testina.
Un altro componente fondamentale è il **trasduttore a campo** che si occupa di concentrare il calore in un'area ridotta.
L'intero **ciclo di funzionamento**, composto da riscaldamento, scrittura e raffreddamento, dura pochissimo. La durata è nell'arco del **nanosecondo**.



Segue la descrizione delle varie componenti del disco:

- **Piatto:** Un disco rigido si compone di uno o più dischi paralleli, in cui ogni superficie è detta piatto ed è identificata da un numero univoco.
- **Traccia:** Su ogni piatto, vi sono numerosi anelli concentrici, detti tracce, ciascuno identificato da un numero univoco.

- **Cilindro:** L'insieme di tracce poste alla stessa distanza dal centro si dice cilindro. Corrisponde ad una serie di tracce con lo stesso numero ma su piatti diversi.
- **Settore:** Ogni traccia è suddivisa in settori circolari, ognuno identificato da un numero univoco.
- **Blocco:** L'insieme dei settori posti nella stessa posizione su tutti i piatti.
- **Testina:** Su ogni piatto è presente una testina di lettura/scrittura. La posizione di tutte le testine è solidale sui diversi piatti. Sono quindi tutte sullo stesso cilindro.

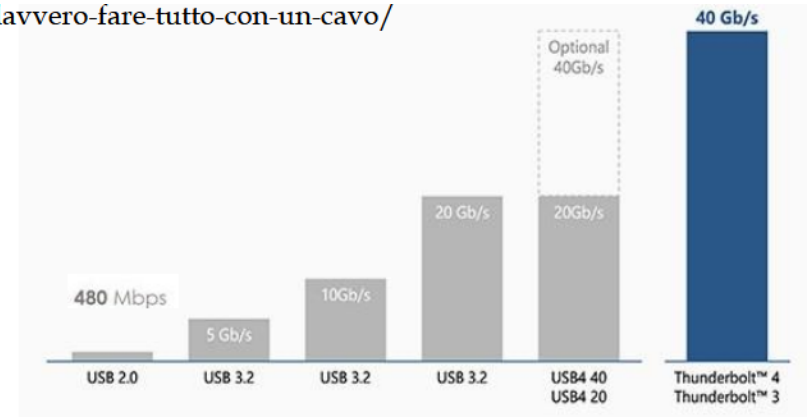
L'efficienza e la capacità della memoria secondaria è sempre in aumento:

Parameter	Started with (1957)	Improved to	Improvement
Capacity (formatted)	3.75 megabytes[18]	32 terabytes (as of 2024)[19][20]	8.5-million-to-one[c]
Physical volume	68 cubic feet (1.9 m^3)[d][5]	2.1 cubic inches (34 cm^3)[21][e]	56,000-to-one[f]
Weight	2,000 pounds (910 kg)[5]	2.2 ounces (62 g)[21]	15,000-to-one[g]
Average access time	approx. 600 milliseconds[5]	2.5 ms to 10 ms; RW RAM dependent	about 200-to-one[h]
Price	US$9,200 per megabyte (1961;[22] US$97,500 in 2022)	US$14.4 per terabyte by end of 2022[23]	6.8-billion-to-one[i]
Data density	2,000 bits per square inch[24]	1.4 terabits per square inch in 2023[25]	700-million-to-one[j]
Average lifespan	c. 2000 hrs MTBF[*citation needed*]	c. 2,500,000 hrs (~285 years) MTBF[26]	1250-to-one[k]

Miglioramento delle connessioni

Le connessioni continuano ad essere migliorate ogni anno, aumentando le capacità di trasferimento dati.
PCI Express 7.0 offre un trasferimento di dati a 128 GT/s e una banda bidirezionale fino a 512 GB/s.
Ad esempio, **Thunderbolt** 5 avrà una larghezza di banda di 120 Gb/s in trasmissione e 80 Gb/s bidirezionali.
Lo stesso vale per **USB** 4 version 2.0, che offre una velocità fino a 80Gb/s.
Entrambe offrono anche capacità di ricarica fino a 240W.

[2] davvero-fare-tutto-con-un-cavo/



Da notare che si misurano i trasferimenti anche in **GT/s**, ovvero **gigatransfer al secondo**.
La differenza con unità come il Gb/s sta nel fatto che PCIe è un bus seriale con un clock incorporato nei dati. Ad esempio, nel caso di una codifica 8b/10b usata da PCIe 1.0, si trasferiscono 10 bit di cui solo 8 sono di dati effettivi e 2 sono necessari a far capire al ricevitore quando prelevare i dati.

Operazione di I/O

Per eseguire un'operazione I/O, si inserisce il comando opportuno in **porte di I/O mappate in memoria**.

L'**adattatore** invia il comando al controllore del disco, che **agisce sugli elementi elettromeccanici** per portare a termine il compito richiesto.
Il trasferimento avviene tra la superficie del disco e la cache incorporata nel controllore.

Tempo di accesso

Si usa questa formula per stimare il **tempo medio di accesso** ai dati:

Tempo di accesso medio = seek time medio + latenza media

- Per i dischi più veloci ⇨ 3msec + 2msec = 5msec
- Per dischi lenti ⇨ 9msec + 5.55msec = 14.55msec

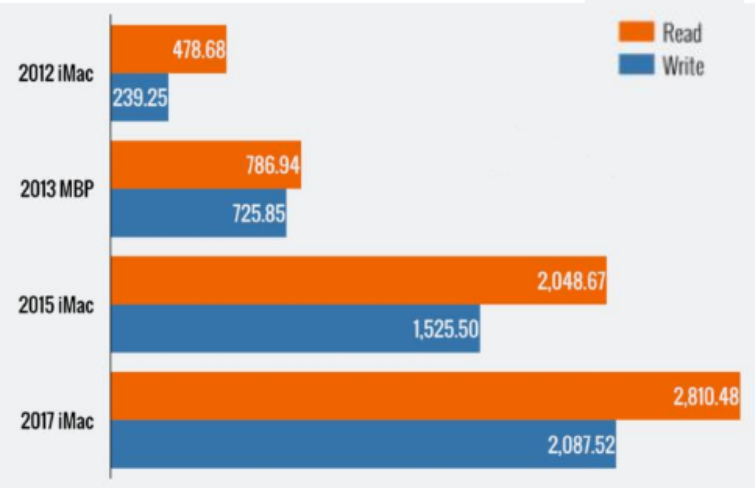
Il **tempo medio di I/O** vale invece:

Tempo medio di I/O = tempo medio di accesso + quantità di dati da trasferire/velocità di trasferimento + overhead

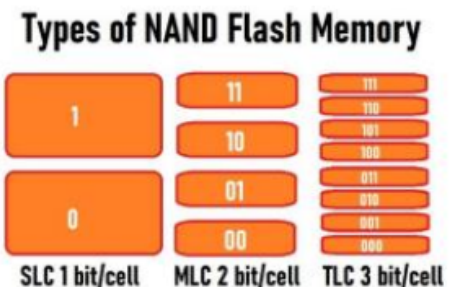
SSD

Le SSD sono memorie allo stato solido (EEPROM).
In una memoria flash le informazioni vengono registrate in un array di **FLoating Gate MOSFET**, un tipo di transistor in grado di mantenere la carica elettrica per un lungo periodo.
Ogni **transistor** costituisce una **cella di memoria** che contiene un bit di informazioni *(o più, vedi sotto)*.
Queste memorie non presentano parti mobili e per questo sono più **resistenti agli urti** rispetto a dischi HDD.
Sono anche **più piccole** e quindi portatili.

I **miglioramenti** visti nel caso degli HDD coinvolgono anche gli **SSD**.



Le **SSD moderne** sfruttano la tecnologia **NVMe** (**Non-Volatile Memory Express**), che sfrutta la banda larga offerta dalla tecnologia PCIe.
Rispetto alla **precedente tecnologia SATA**, che era in grado di gestire una sola coda alla volta con un massimo di 256 comandi, PCIe può gestirne fino a 64000, ciascuna da 64000 comandi.
Questa grande quantità di code permette di evitare l'**I/O lock** (applicazioni bloccate in attesa della coda di I/O), in quanto ogni applicazione ha la propria coda.



Le memorie SSD **TLC** usano **3 bit di dati per cella**.
Le QLC ne usano 4.
Le SSD **Single Level Cell** (**SLC**) usano invece un solo bit per cella.
Queste ultime sono usate solo in ambito aziendale a causa del maggior costo a parità di capacità.
Sono però molto più durature.

La maggior parte delle SSD moderne utilizza la tecnologia delle memorie flash **NAND**, che garantisce uniformità nella distribuzione dei dati e nell'usura dell'unità.

Inizialmente le SSD avevano un **ciclo di vita minore** di un HDD. Questo problema è ormai stato risolto. Attualmente, si parla di 1000 anni di vita con 2TB di scritture all'anno.

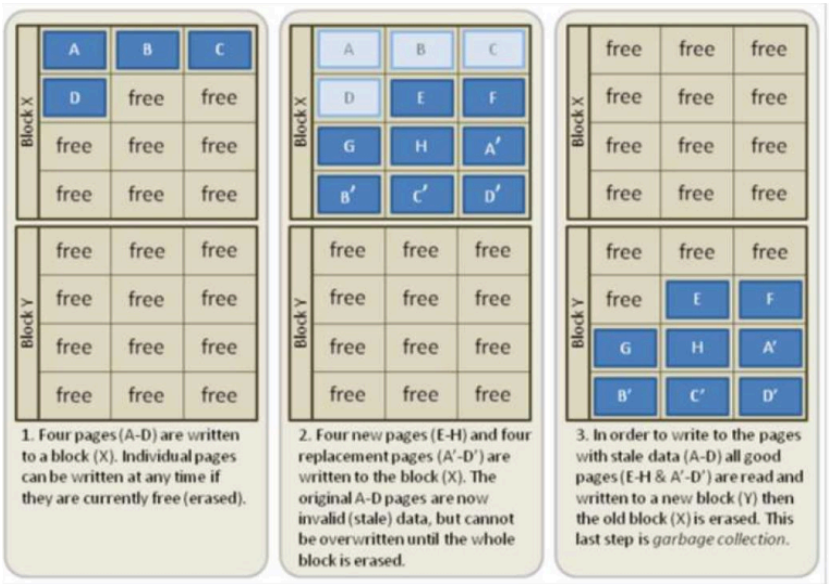
Il **costo per terabyte** di dati è **più elevato** rispetto a quello degli HDD.

La **frammentazione** nelle SSD non influisce sulle sue prestazioni in quanto il tempo di accesso ad ogni dato è identico.
Gli SSD utilizzano blocchi molto superiore a quelli dei dischi HDD (4 KiB e 256 KiB).
Le **celle non possono essere riscritte**, bisogna **prima cancellarle**.
Si ricorre quindi ad alcuni metodi per l'individuazione di pagine non valide e si fa ricorso al garbage collection.

Garbage collection

Ogni riscrittura, anche parziale, di un blocco richiede di cancellarlo e riscriverlo interamente. Questo comporta una **rapida degradazione dei blocchi**. Per questo i produttori sono costretti ad inserirne di più per sostituire quelli che durante la vita del disco diventano inutilizzabili.

Nelle nuove versioni dello standard NVMe 2.0, gli SSD saranno divisi in zone anziché emulare i blocchi degli HDD. Questo metodo consentirà di ottimizzare l'utilizzo della tecnologia SSD e ridurre al minimo scritture parziali o riscritture dello stesso blocco.



Pages are written into blocks until they become full. Then the pages with current data are moved to a new block and the old block is erased.

Tape

Sono stati i primi supporti di memorizzazione secondaria.
Inizialmente erano sotto forma di **bobine**, mentre ora sono racchiusi in **cartucce**.
Hanno una **durata** molto **lunga** e sono capaci di immagazzinare **dati in quantità enormi** a **costi bassissimi**.

Il **problema** è il **tempo di accesso lunghissimo**, circa 1000 volte più lento di un HDD.

Sono generalmente usati per **backup**, registrazione di dati poco usati.

Il nastro è avvolto in bobine e scorre su una testina. Una volta raggiunta la posizione desiderata, la velocità è paragonabile a quella di un disco.

	2006	2010	2014	2015	2017	2020
Areal Density (bits per sq inch)	6.67 Billion	29.5 Billion	85.9 Billion	123 Billion	201 Billion	317 Billion
Cartridge Capacity (Terabytes)	8	35	154	220	330	580
# of Books Stored*	8 Million	35 Million	154 Million	220 Million	330 Million	580 Million
Track Width	1.5 µm	0.45 µm	0.177 µm	0.140 µm	103 nm	56.2 nm
Linear Density (bits per inch)	400'000	518'000	600'000	680'000	818'000	702'000
Tape Material	Barium Ferrite	Barium Ferrite	Barium Ferrite	Barium Ferrite	Sputtered Media	Strontium Ferrite
Tape Thickness (micrometers - µm)	6.1	5.9	4.3	4.3	4.7	4.3
Tape Length (meters)	890	917	1255	1255	1098	1255

© Copyright IBM Corporation 2020. IBM and the IBM logo are trademarks of IBM Corp. registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml

* assumes 1MB of text data per book

IBM

Il futuro - DNA

L'uso del DNA come supporto di archiviazione è qualcosa a cui molte aziende (Microsoft compresa) stanno lavorando da tempo.

Si usano le **basi del DNA** (A, G, C, T) e tramite un algoritmo (**fountain code**) si generano alcune "**gocce**" che contengono parte dell'informazione binaria del file.
Ad ogni goccia si associa un codice a barre per il riassemblaggio del file.

Un file salvato in DNA può essere **copiato un numero potenzialmente infinito di volte** senza errori nel recupero del file.

La **densità** è **eccezionale**: 215 petabyte di dati in un solo grammo.

Il **costo** è **altissimo**: 7000 dollari per archiviare 2 MB e altri 2000 per leggerli.

Blocchi logici

Le unità a disco vengono indirizzate come giganteschi **array di blocchi logici**.
Il blocco logico è la **minima unità di trasferimento**.

La **mappatura** dei blocchi logici avviene **in modo sequenziale**:

- Il settore 0 è il primo settore della traccia del primo disco del cilindro più esterno.
- Si prosegue lungo la traccia, poi lungo le altre tracce dello stesso cilindro.
- Si continua cilindro per cilindro dall'esterno all'interno.

Eventuali **settori danneggiati** vengono **esclusi** dalla mappatura.

CLV (**Constant Linear Velocity**): La densità di bit per traccia è uniforme.
Le tracce più esterne hanno circonferenza maggiore, per cui contengono più settori.
Per mantenere costante la quantità di dati che passa sotto la testina, è necessario aumentare la velocità di rotazione spostandosi verso l'interno.

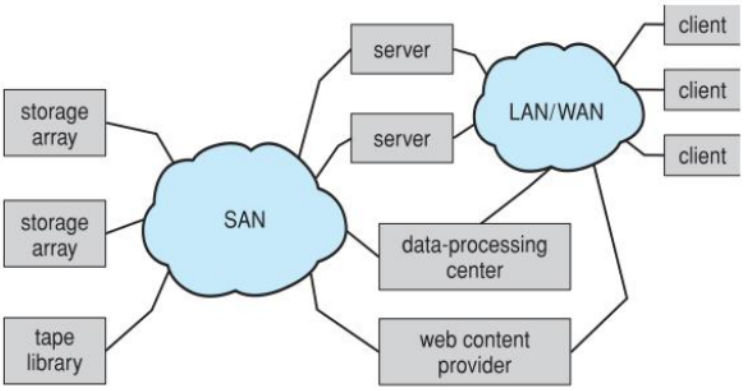
CAV (**Constant Angular Velocity**): La velocità di rotazione è costante.
La densità di bit delle tracce esterne diventa man mano minore, in modo da garantire una quantità costante di dati sotto la testina.

SCSI e FC

Si accede alla memoria secondaria tramite porte locali di I/O collegate al bus.

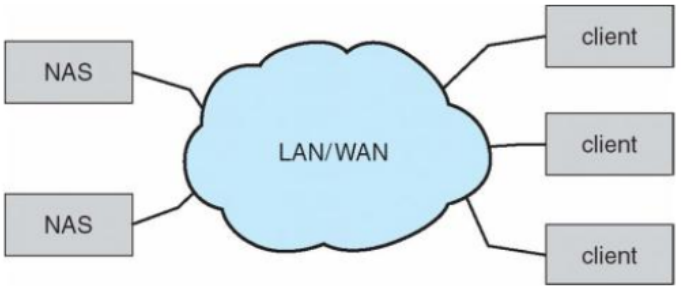
La tecnologia **SCSI** è un'interfaccia standard per il trasferimento dati.
Prevede uno **SCSI initiator** che **richiede** le operazioni e uno **SCSI target** che **realizza** il compito richiesto.
Ciascun target può avere al massimo 8 unità collegate (quindi in totale 16 dischi).

La tecnologia **FC** (**Fiber Channel**) è invece un'architettura seriale ad alta velocità che sfrutta uno spazio di indirizzi a 24 bit. E' usata nelle **Storage Area Network** (**SAN**), nelle quali molti host sono connessi a molte unità di memorizzazione comune.



NAS

Sta per **Network Attached Storage**, si tratta di un dispositivo di archiviazione connesso alla rete.
I client accedono alla memoria connessa alla rete tramite un'interfaccia RPC, supportata da protocolli come NFS (Linux) e CIFS (Windows).
Le chiamate di procedura remota sono implementate per mezzo di TCP / UDP sopra una rete IP.



Scheduling del disco

Il SO è responsabile dell'uso efficiente dell'hardware.
Per i dischi, questo significa garantire **tempi di accesso** brevi e una buona **larghezza di banda**.

Il tempo di accesso si divide in **tempo di ricerca (seek)** e **latenza di rotazione**, come visto sopra.
L'unico su cui si può intervenire è il tempo di ricerca. Si cerca quindi di minimizzarlo.

L'ampiezza di banda è il numero totale di byte trasferiti, diviso per il tempo totale di trasferimento (dalla prima richiesta all'ultimo trasferimento).

Una **richiesta** comprende:

- **TIpo** di operazione: immissione/emissione di dati.

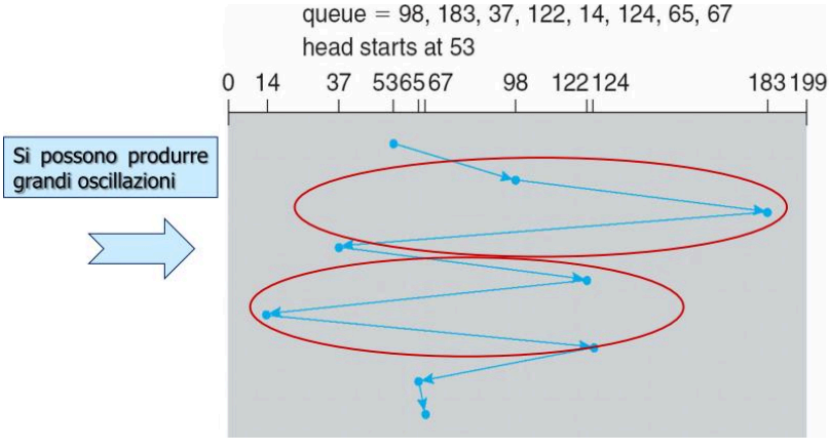
- **Indirizzo sul disco** relativo al quale effettuare il trasferimento.
- **Indirizzo in memoria** relativo al quale effettuare il trasferimento.
- **Numero di byte** da trasferire.

Se unità a disco e controller sono disponibili, la richiesta può essere soddisfatta all'istante, altrimenti deve essere aggiunta alla **coda delle richieste** di quell'unità.

Il SO sceglie quale delle richieste in coda servire per prima tramite un **algoritmo di scheduling del disco**.

First Come First Served (FCFS)

Come già visto per lo scheduling della CPU, si esegue sempre la prima richiesta in coda.
Il **problema** di questo algoritmo è che si producono grandi **oscillazioni**.

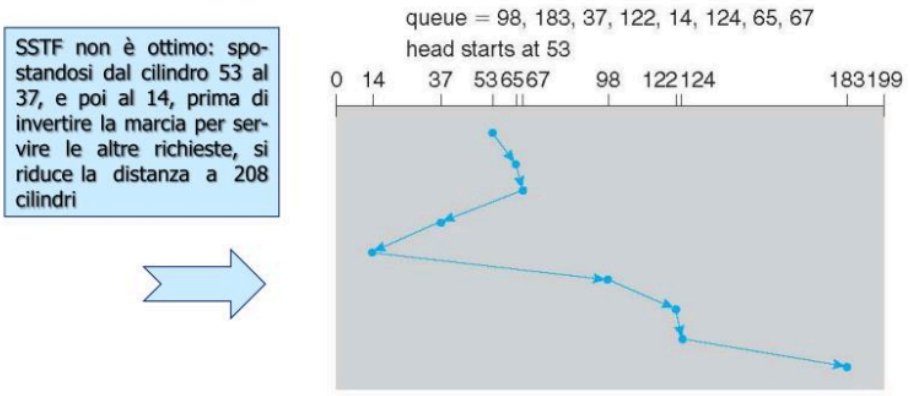


In questo caso il movimento totale della testina è di 640 cilindri.

Shortest Seek Time First (SSTF)

Viene selezionata per prima la richiesta con **seek time minimo** relativo alla posizione corrente della testina.

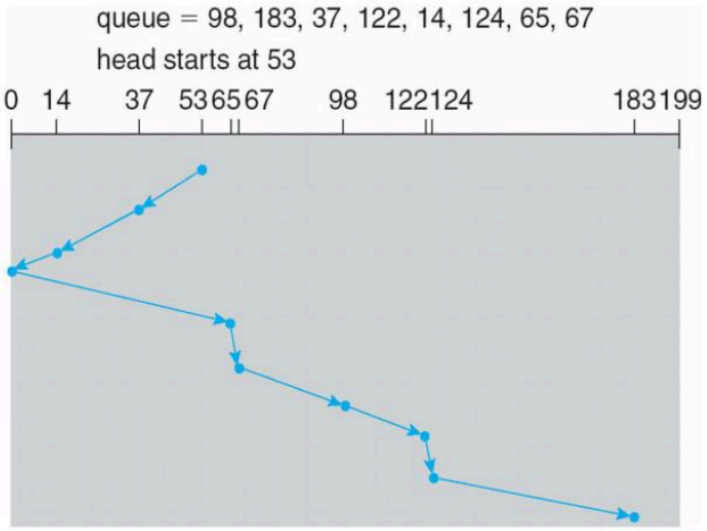
E' di fatto una forma di Shortest Job First (SJF), e soffre dello stesso **problema**, ovvero l'**attesa indefinita** di alcune richieste.
In questo caso si producono 236 cilindri di spostamento.



Scan, C-Scan, C-Look

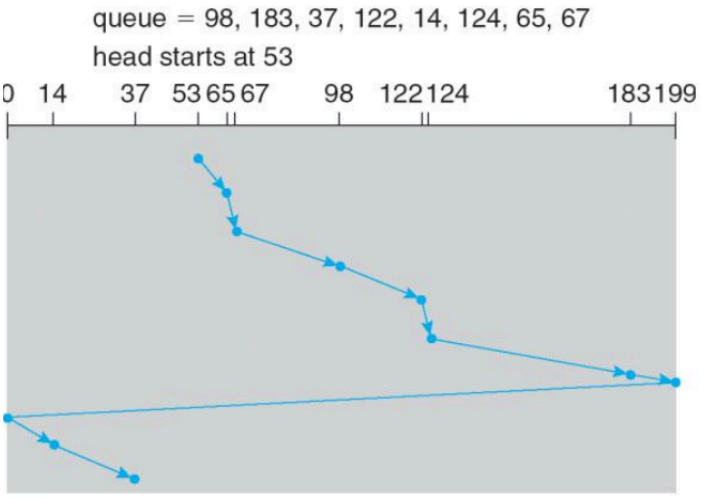
Il braccio della testina si muove da un estremo all'altro del disco, servendo le richieste in modo **sequenziale**.
Giunto ad un estremo, inverte la direzione di marcia e prosegue.
E' anche detto **algoritmo dell'ascensore**.

Se le richieste sono distribuite uniformemente, quando la testina inverte il senso di marcia la maggior parte delle richieste si trova nei pressi dell'estremo opposto.
Tali richieste avranno **tempo di attesa più lungo**.



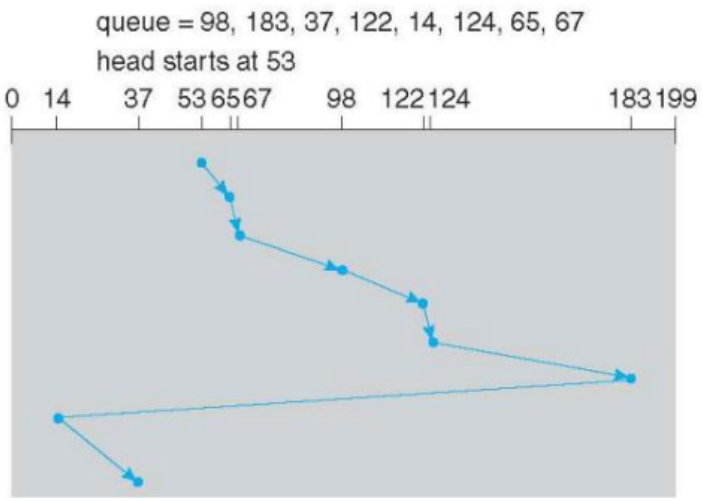
In questo si produce un movimento di 236 cilindri.

Esiste anche la variante **C-Scan**, che una volta arrivato ad un estremo, ritorna subito al primo cilindro.



In questo caso produrrebbe un movimento di 383 cilindri.

Un'ultima versione è il **C-Look**, che serve l'ultima richiesta in una direzione e poi inverte il movimento senza arrivare alla fine del disco.



Produce un movimento dei 353 cilindri.

Riepilogo Scheduling disco

- **SSTF** è molto comune e ha un comportamento naturale.
- **LOOK e C-LOOK** forniscono prestazioni migliori per sistemi che usano intensamente i dischi (riducono l'attesa indefinita).

L'algoritmo di scheduling del disco deve essere un modulo separato del SO, in grado di essere sostituito senza problemi nel caso cambiassero le caratteristiche del sistema.

Linux implementa un sistema di **deadline**: le richieste che superano un certo tempo di attesa vengono servite subito.

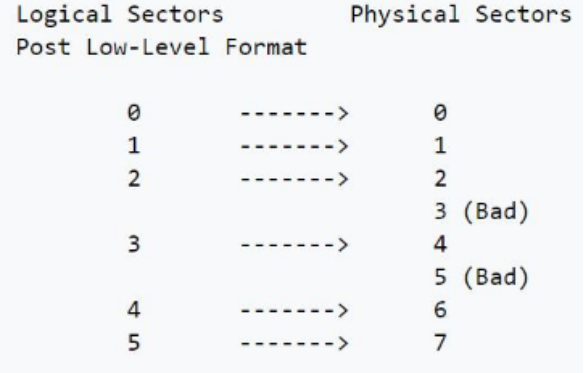
Formattazione

Formattazione di basso livello o fisica: Si suddivide il disco in settori che possono essere letti e scritti dal controllore del disco.
Si salva sul disco una struttura dati per ogni settore.
La dimensione standard è di 512 byte.

Per poter usare il disco, il SO deve mantenere le proprie strutture dati sul disco.

- Il disco viene partizionato in uno o più gruppi di cilindri detti **dischi logici**.
- Viene creato un file system (**formattazione logica**).
- Per migliorare le prestazioni, la maggior parte dei file system accorpa i blocchi in gruppi (**cluster**).

Una parte dei settori rimane nascosta al SO. Questi servono per sostituire i settori una volta danneggiati. Se ne occupa il controllore.
Si parla di **accantonamento dei settori**.



Correzione degli errori

Le memorie elettroniche o magnetiche memorizzano i bit usando meccanismi che possono occasionalmente generare **errori**.

Formalizziamo il concetto di errore in una codifica a n bit:

- **C** codifica corretta, **C′** codifica errata.
- **Distanza di Hamming:** H(C,C′), è il numero di cifre binarie diverse a parità di posizione.

Per scoprire gli **errori singoli**, ovvero H(C,C′)=1, aggiungiamo un **bit di parità** alla codifica.
Tale bit viene posto a 1 o 0.
Allora il numero di “1” in entrambi i dati è pari (o dispari).
Se in C′ si verifica un errore singolo, allora il numero di “1” non sarà più pari (o dispari).
Non è possibile scoprire numeri di errori pari.

In generale, possiamo avere **più di un bit** per correggere o scoprire possibili errori multipli.
Si hanno

- **n** bit della codifica minimale.
- **m > n** bit della codifica estesa.
- **r = m - n** bit che estendono la codifica minimale.
- solo 2^n su 2^m codifiche sono valide.

Per cui per m=n+1, solo metà delle codifiche sono valide.

La **distanza minima tra parole legali** del codice è la **distanza di Hamming del codice**.
La distanza di Hamming indica **quanti errori si possono rilevare o correggere.**

Per **rilevare d errori** serve una **distanza minima d+1**.
Questo perché le parole valide sono tra loro ad un multiplo della distanza minima, per cui su una distanza di 3 si potrebbero rilevare al massimo 2 errori. A distanza=3 potrebbe essere un'altra parola valida (non viene riconosciuto alcun errore), oppure una parola non valida (viene rilevato un errore). Non avrei quindi alcuna garanzia.

Per **correggere d errori** serve una **distanza minima 2d+1 .**
Questo perché è necessario conoscere la parola più vicina per correggere gli errori individuati. A distanza minima = 3, se avessi una H(C,C′)=1, allora sarebbe facile risalire a C, in quanto è la sola parola valida a quella distanza, le altre sono tutte più lontane. Se avessi invece H(C,C′) = 2, è possibile che esista una parola che dista 1 da C′, rendendola la scelta del correttore rispetto alla soluzione reale C.

Vediamo il caso d=1:
I bit aggiunti sono di parità calcolati su sottoinsiemi di bit della parola del codice: se si numerano i bit da sinistra i bit di parità vengono inseriti nelle posizioni che sono potenze di 2 {1,2,4,8,16,...}. Gli altri sono i bit di dati.

Ogni bit di parità viene calcolato su un **sottoinsieme di bit**.
Ogni bit di dati può influire su diversi bit di parità.
Per sapere su quali bit influisce, si calcola la sua **espansione**, ovvero lo si riscrive come somma di potenze di 2.
ESEMPIO: il bit 11=1+2+8 influisce su questi bit di parità.

Al momento del **controllo** vengono ricalcolati i bit di parità per ogni posizione k, con k = 1,2,4,8,etc...
Se la parità non è corretta, si aggiunge a k un contatore inizializzato a 0.
Al termine, il contatore conterrà il valore del bit errato.
ESEMPIO: se il contatore è 1+2+8=11 il bit in tale posizione è sbagliato.
NOTA: in questo esempio assumo d=1, per cui l'errore può essere corretto.

Partendo dalla sequenza:
0 1 1 0
si calcolano e inseriscono i bit di parità nelle posizioni 1, 2 e 4:
_ _ 0 _ 1 1 0
quindi i bit di dati occupano le posizioni 3, 5, 6 e 7.
Il bit 3 influenza i bit di parità 1 e 2 (3=1+2).
Il bit 5 influenza i bit di parità 1 e 4 (5=1+4).
Il bit 6 influenza i bit di parità 2 e 4 (6=2+4).
Il bit 7 influenza i bit di parità 1, 2 e 4 (7=1+2+4).

Il bit di parità 1 è calcolato sui bit di dati 3, 5 e 7 (0 1 0) e quindi vale 1.
Il bit di parità 2 è calcolato sui bit di dati 3, 6 e 7 (0 1 0) e quindi vale 1.
Il bit di parità 4 è calcolato sui bit di dati 5, 6 e 7 (1 1 0) e quindi vale 0.

Esiste un **numero minimo di check bit per correggere errori singoli d=1**.

Tali bit devono essere configurati in modo che la distanza minima di Hamming sia 2d+1=3.

Word size	Check bits	Total size	Percent overhead
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

i bit di parità vengono inseriti nelle posizioni che sono potenze di 2 (1, 2, 4, 8, 16 …)

Boot

Nei sistemi **UNIX/Linux** il boot loader più conosciuto è GRUB (GRand Unified Bootloader).
Inizialmente, un piccolo file system viene caricato in RAM per contenere drivers e altri moduli del kernel necessari ad avviare il file system reale.

Una volta inizializzato il file system, Linux crea un primo processo (**systemd**) e successivamente tutti gli altri.

In **Android**, si usa il bootloader è **LK** (Little Kernel).

In **Windows**, il firmware del sistema **esegue** il codice nella **MBR** (Master Boot Record) che, con l'ausilio della tabella delle partizioni, identifica quella dove si trovano kernel e drivers.

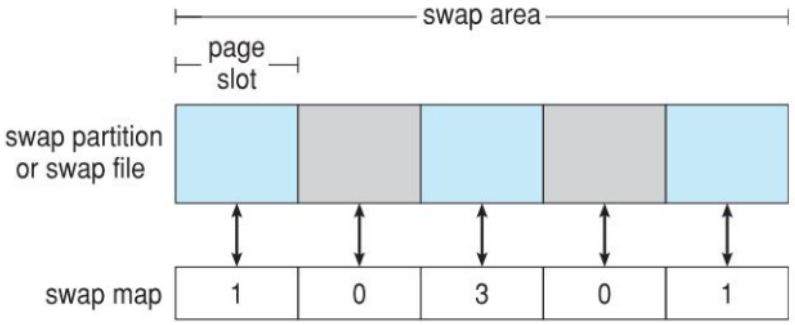
Gestione swap

La memoria virtuale impiega una **parte del disco come** se fosse un'**estensione della memoria centrale**.

L'obiettivo è fornire al sistema una migliore produttività.
Lo spazio di swap di trova generalmente in una partizione separata del disco, ma può anche trovarsi direttamente nel file system.

In **Linux**, la swap è utilizzata solo per la **memoria anonima**, ovvero dati che non corrispondono a file.
Linux permette l'istituzione di una o più **aree di avvicendamento**.
Un'area di avvicendamento è una serie di **moduli da 4 KB**, detti **slot delle pagine**, che conservano le pagine avvicendate.

Ogni area dispone di una **mappa di avvicendamento**, un array di contatori interi a cui corrispondono slot dell'area.
Se un contatore è 0, allora la pagina è disponibile.
Se il valore è maggiore, allora lo slot è occupato da una delle pagine avvicendate.
Il valore del contatore rappresenta il numero di collegamenti alla pagina: se è 3, allora la pagina appartiene allo spazio di indirizzi di 3 processi.



Connessione dei dispositivi di memoria

I calcolatori possono accedere alla memoria tramite:

- Un dispositivo collegato alla macchina.
- Un dispositivo connesso alla rete.
- Il cloud.

Alla memoria secondaria connessa alla macchina si accede dalle porte locali di I/O che sono collegate al bus (Nei pc, interfaccia ATA o SATA).

Si usano tecnologie come SCSI e FC per realizzare il trasferimento di dati dalla memoria secondaria.

RAID

Redundant Array of Independent Disks, l'affidabilità del sistema viene garantita tramite la ridondanza.
Aumenta il **tempo medio di guasto**.
Sono spesso affiancati da **NVRAM** (Non-Volatile Random Access Memory) per garantire la consistenza dei dati scritti contemporaneamente su dischi multipli.

E' consigliato usare **un solo tipo di disco** per un sistema RAID, in quanto funzionerà in ogni caso alla velocità del disco più lento.

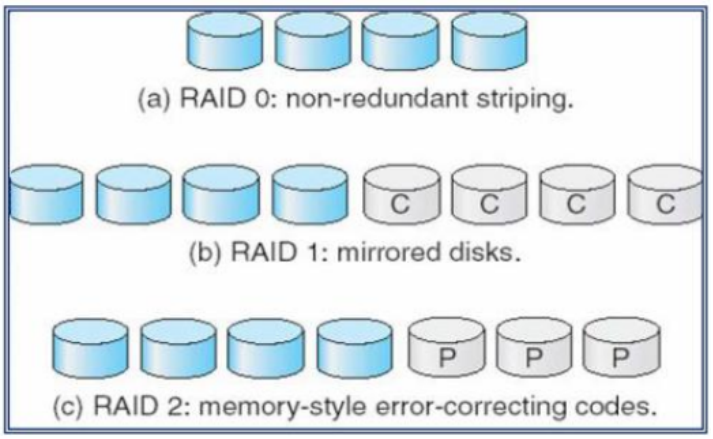
Un **vantaggio** è che **richieste indipendenti** possono essere **servite in parallelo** da dischi diversi.
Anche una sola richiesta di un elevato numero di blocchi può essere servita da più dischi che operano in maniera coordinata.

Si distinguono due livelli di **granularità** di sistemi RAID:

- **Grana fine:** I dati vengono inseriti in blocchi relativamente piccoli, le richieste di I/O accedono a tutti i dischi del gruppo. **data rate elevato, I/O rate basso** (serve richieste pesanti molto velocemente, ma serve poche richieste alla volta).
- **Grana grossa:** I dati vengono inseriti in blocchi relativamente grandi, le piccole richieste vengono servite da un piccolo numero di dischi. SI ha un **data rate basso, I/O rate alto**. Il contrario di prima.

Esistono diversi tipi di RAID:

- **RAID 0, sezionamento del disco o data striping**: Tratta un gruppo di dischi come un'unica unità di memorizzazione. Ogni blocco di dati viene diviso in sottoblocchi memorizzati su dischi distinti. Il tempo di trasferimento aumenta proporzionalmente al numero di dischi del gruppo.
- **RAID 1, mirroring o shadowing**: Conserva duplicati di ciscun disco.
- **RAID 2, struttura a blocchi di parità**: Utilizza un minor grado di ridondanza, mantenendo buone caratteristiche di affidabilità. Usa uno striping a livello di bit e usa codici di Hamming per la correzione degli errori.



Il **RAID 0** non è una vera architettura RAID, in quanto non mantiene una ridondanza.
Presenta il **minor costo** ma anche la **minore affidabilità**.
Presenta la **miglior prestazione in scrittura**, ma non in lettura.
Usato spesso in applicazioni di supercomputing.

Esistono **combinazioni di livelli RAID**:

- **RAID 0+1, stripe of mirrors:** I dati vengono distribuiti su un insieme di dischi in duplice copia.
- **RAID 1+0, mirror of stripes:** I dati sono distribuiti su un array di dischi che viene duplicato su un altro array di dischi.

Altri livelli di RAID comprendono

- **RAID 3**: Usa uno striping a livello di bit, il movimento è unico per tutte le testine. Usa un solo disco di parità. I disk controller identificano il disco guasto, permettendo di correggere gli errori usando un solo disco di parità.
  Si accede in scrittura a tutti i dischi dati e al disco di parità, si accede in lettura solo ai dischi dati.
- **RAID 4**: Striping a livello di blocco, testine indipendenti. Si usa un solo disco di parità. Piccole letture richiedono l'accesso ad un solo disco di dati.
  Accessi in scrittura richiedono 4 I/O: due per leggere il disco dati e parità e due per aggiornarli.
  Il collo di bottiglia rimane il disco di parità.
  I risultati in termini di affidabilità, costo e capacità sono uguali a quelli del RAID 3.
- **RAID 5**: Striping a livello di blocco e parità distribuita su tutti i dischi, testine indipendenti. MIgliori prestazioni rispetto a RAID 3 in quanto scritture indipendenti possono non dover accedere allo stesso disco per la parità. Ottime prestazioni per grandi o piccole letture e grandi scritture. Le piccole scritture sono meno efficienti del sistema di mirroring.
- **RAID 6:** Striping a livello di blocco, parità distribuita su tutti i dischi, testine indipendenti. Tolleranza fino a due guasti tramite codici di Reed-Solomon. Le prestazioni sono quelle di RAID 5, ma piccole scritture richiedono 6 operazioni di I/O.
  L'affidabilità è superiore al RAID 5, ma anche il costo.

NON SERVE CONOSCERE IL CODICE DI REED-SOLOMON. IN OGNI CASO E' A PAGINA 575 DELLE SLIDE.

Il RAID può essere implementato via hardware o software.

- **RAID hardware:** le unità sono collegate alla scheda madre tramite hardware speciale chiamato **controller** RAID. E' un dispositivo utilizzato per gestire le unità disco in un array di archiviazione. Può essere usato come livello di astrazione tra il SO e i dischi fisici. Tale controller può essere anche integrato nella scheda madre. Il SO interagisce come se il sistema fosse un singolo disco logico, delegando la gestione al controller.
- **RAID software**: Il controller usa le risorse di sistema, come processore e memoria. Svolge le stesse funzioni di un controller hardware, ma può influenzare le prestazioni di altre applicazioni sul server.
  Su WIndows 10 Pro è inclusa la gestione del RAID. Lo stesso vale per Windows 11. E' supportato il RAID per Windows 11 home, ma in versione molto limitata.

Il RAID 6 risulta vantaggioso soprattutto quando non ci si può permettere downtime in caso di secondo guasto.
E' particolarmente usato in contesti come satelliti, centrali nucleari, moduli spaziali, etc... in cui le singole particelle cariche possono causare bit flip.

NOTA: mancano alcune informazioni sul RAID meno importanti (slide 576).

File system

Struttura del file:

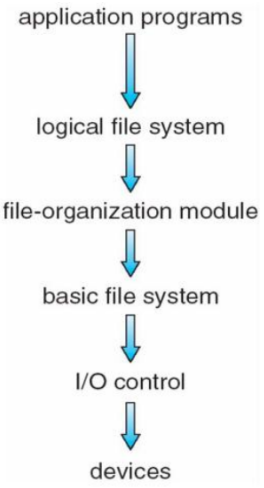
- Unità di memorizzazione logica
- Collezione di informazioni correlate
- **File Control Block**: struttura dati del kernel che descrive il file.

Il file system risiede nella memoria secondaria.
Fornisce una semplice interfaccia verso i dispositivi di memorizzazione, realizzando il mapping tra indirizzi logici e fisici.
Fornisce la possibilità di accedere al disco in maniera efficiente per memorizzare e recuperare rapidamente le informazioni.

I dischi sono mezzi convenienti per la memorizzazione dei file perché:

- Si possono riscrivere localmente.
- E' possibile accedere direttamente a qualsiasi blocco di informazioni del disco, e quindi a qualsiasi file, sia in modo sequenziale che diretto.

Le operazioni di I/O su disco avvengono con granularità di blocco.
I blocchi attuali sono da 4 KB.
I dispositivi fisici vengono controllati tramite **device drivers**.



Per l'**utente**, il file system è la definizione di file e directory e le loro operazioni.
Per il **SO**, è la scelta di algoritmi e strutture dati che mettono in corrispondenza il file system logico con i dispositivi fisici.

Il file system è **stratificato**, ovvero organizzato in livelli. Ognuno di questi livelli usa funzioni di quelli inferiori per crearne di nuove, usate dai livelli superiori.

Controllo I/O

I **driver dei dispositivi** traducono comandi di alto livello in sequenze di bit che guidano l'hardware di I/O a compiere una specifica operazione.
Scrivono quindi **configurazioni di bit** in **specifiche locazioni di memoria** per indicare al dispositivo le operazioni.

Il file system invia comandi generici al driver di dispositivo per leggere / scrivere blocchi fisici sul disco.
Gestisce il buffer del dispositivo e la cache che conserva i metadati.

File system logico

Il **modulo di organizzazione dei file** traduce gli indirizzi logici di blocco in indirizzi fisici e contiene il modulo per la gestione dello spazio libero.

Il **file system logico** gestisce i **metadati**, ovvero tutte le strutture del file system eccetto i dati veri e propri memorizzati nel file.
Mantiene le strutture di file tramite **file control block**, che contengono informazioni come proprietario, permessi, posizione del contenuto.
Gestisce le directory, la protezione e la sicurezza.

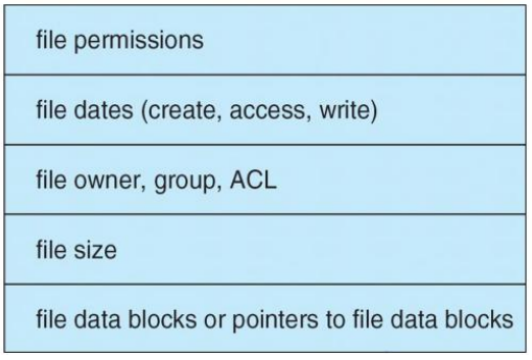
Esistono diversi **tipi di file system**:

- **ISO 9660** per CD-ROM.
- **UFS** per UNIX.
- **FAT, FAT32, FAT64, NTFS** per Windows.
- **ext2, ext3, ext4** (extended file system) per Linux (ma ne supporta molti altri).
- **FUSE**, un progetto open source volto alla realizzazione di un modulo per il kernel Linux che permetta agli utenti di creare un proprio file system senza scrivere codice a livello kernel.

Le funzioni del file system vengono realizzate tramite chiamate di sistema invocate tramite la API, che utilizzano dati gestiti dal kernel, residenti su disco o memoria.

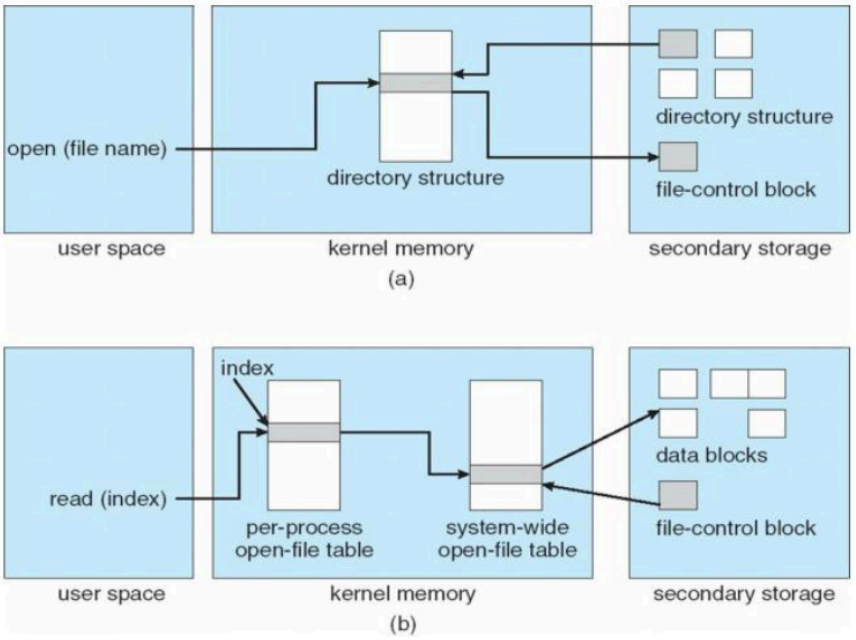
Risiedono **nel disco:**

- **Blocco di controllo di avviamento**: contiene le informazioni per l'avviamento di un SO da quel volume. Si dice **boot block** per UFS, **partition boot manager** per Windows. Di solito è il primo blocco del volume.
- **Blocchi di controllo dei volumi**: contengono dettagli riguardanti la partizione, come il numero totale di blocchi, la loro dimensione, il contatore dei blocchi liberi e i puntatori ad essi.
- **Strutture delle directory:** usate per organizzare i file .
- **Blocchi di controllo dei file:** contengono dettagli sul file, come il numero di FCB, permessi, dimensione, date di creazione/modifica, puntatori ai dati.
  NTFS usa una **master file table** (struttura in stile DB relazionale).



Risiedono **nella memoria:**

- **Tabella di montaggio:** contiene informazioni relative a ciascun volume montato.
- **Struttura della directory:** contiene informazioni relative alle directory referenziate di recente.
- **Tabella dei file aperti**: contiene una copia dell'FCB per tutti i file attualmente aperti nel sistema.
- **Tabella dei file aperti per ciascun processo:** contiene un puntatore all'elemento corrispondente nella tabella generale dei file aperti, con aggiunte le informazioni di accesso specifiche del processo.



(a) Apertura di un file, (b) lettura da file

File system Google

Google ha progettato un file system proprietario per la gestione e il recupero di dati per le specifiche dell'azienda.

GoogleFS, o BigFiles, conserva i dati raccolti da Google utilizzando tecnologie non convenzionali, a causa della grandezza dei file.
Immagazzina dati di circa 100 GB ciascuno, raramente eliminati, sovrascritti o compressi.

Oracle ASM (Automatic Storage Manager) permette di gestire file, directory e volumi per mezzo di direttive SQL in ambiente DBMS (DataBase Management System).

Avviamento

Una partizione può essere un **volume**, ovvero contenere un sistema operativo, oppure avere solo una formattazione di basso livello (**raw**), ovvero una sequenza non strutturata di blocchi.

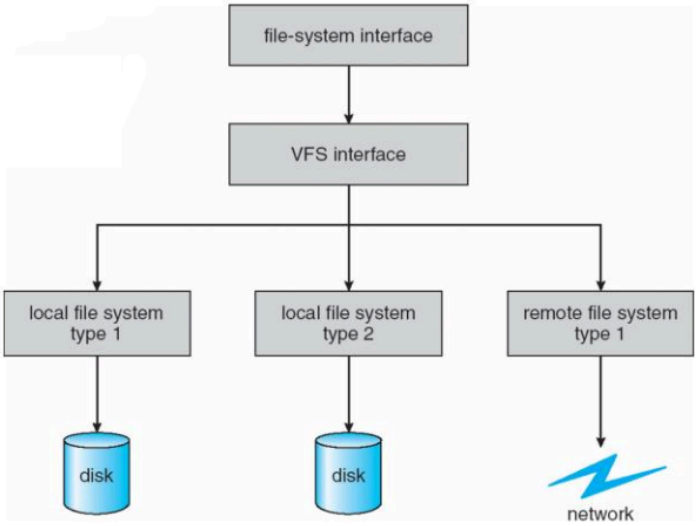
Le informazioni relative all'**avviamento** del SO si registrano in una **apposita partizione**, con formato proprio.
Il sistema in questa fase non ha ancora caricato i driver dei dispositivi e quindi non può usare i servizi del file system (nemmeno interpretarne il formato).
La partizione di avviamento può contenere anche informazioni per l'avvio di più SO (**dual boot**).

Nel momento del caricamento del SO, si esegue il montaggio della partizione radice (**root partition**) che contiene il kernel del SO ed eventuali altri file di sistema.
A seconda del SO, gli altri volumi possono essere montati in questa fase o successivamente.
Si verifica durante il montaggio la presenza nel volume di un file system valido.
Infine, il sistema annota nella **tabella di montaggio** residente in memoria che un file system è stato montato.

File system virtuali

I file system virtuali forniscono una modalità di implementazione di **file system object-oriented**.
Il VFS gestisce una **API di chiamate a sistema** valide per qualunque tipo di file system.

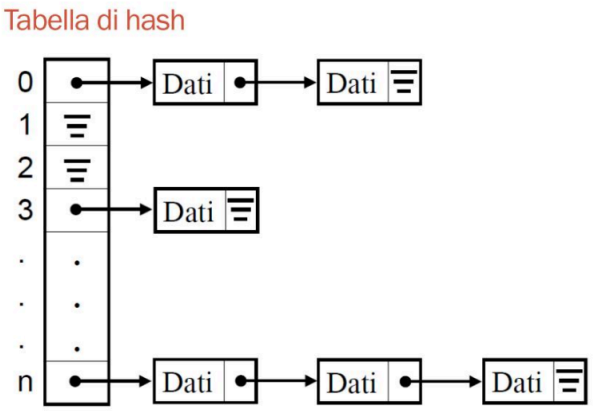
Si può vedere un VFS come un'interfaccia tra il file system e il kernel.
L'API interagisce con l'interfaccia dei file system virtuali, non con quelli fisici.



Realizzazione di una directory

E' possibile implementare directory in diversi modi:

- **Lista lineare** di nomi di file e puntatori a blocchi di dati. Questa soluzione è semplice da implementare tramite array o lista concatenata. L'esecuzione è onerosa per la ricerca. Una lista ordinata risolve il problema della ricerca, ma è onerosa da tenere in ordine.
- **Tabella hash**, sempre tramite lista lineare. Il tempo di ricerca migliora, l'inserimento e la rimozione sono O(1) eccetto collisioni.



Allocazione

Il problema nei dischi è allocare lo spazio per i file in modo da **ridurre** al minimo lo **spreco di memoria** e **garantire rapidità d'accesso**.

Esistono diversi metodi di allocazione:

- Allocazione contigua
- Allocazione concatenata

- Allocazione indicizzata

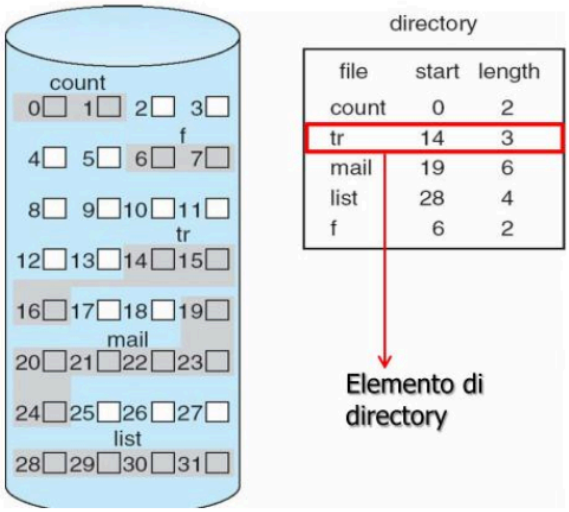
Allocazione contigua

Ciascun **file** occupa **blocchi contigui** sul disco.

Per **reperire** il file necessitiamo solo di **posizione** del file e **lunghezza** da leggere.
L'accesso è casuale, le prestazioni sono ottimali.

I **problemi** di questo metodo sono la **frammentazione esterna** e la necessità di **conoscere a priori la dimensione** del file.

Viene effettuato un **mapping** da blocchi logici a fisici.



Nei sistemi operativi moderni il disco si alloca con granularità maggiore della dimensione del blocco fisico.
Ciascun file consiste di uno o più **extent** di dimensione variabile.

Un **extent** è una porzione di spazio contiguo.

Inizialmente, per un file viene allocato un extent.
Se non dovesse bastare, ne vengono allocati altri.
Questo metodo soffre di **frammentazione interna** per extent grandi.

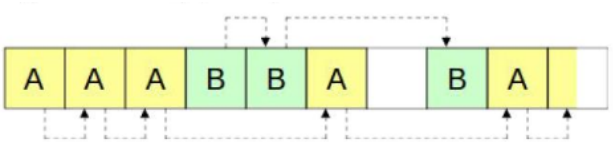
L'elemento della directory contiene l'indirizzo iniziale dell'extent, la sua dimensione e l'indirizzo dell'extent successivo.

Allocazione concatenata

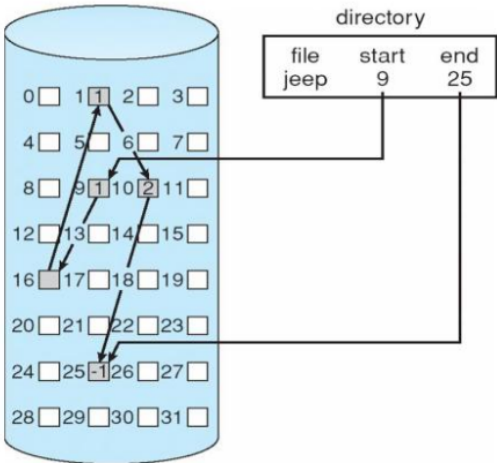
Ciascun **file** è una **lista concatenata di blocchi**.
I blocchi possono essere ovunque nel disco, senza alcun vincolo di contiguità.

Ciascun blocco contiene un puntatore al blocco successivo.
Quando si raggiunge un blocco con puntatore vuoto, si è raggiunta la fine del file.
Non c'è frammentazione esterna, quindi non si spreca spazio.

L'efficienza si può migliorare raccogliendo blocchi in **cluster**, aumentando però la frammentazione interna.



Nella directory, si mantiene l'indirizzo del blocco iniziale e di quello finale.



File allocation table (FAT)

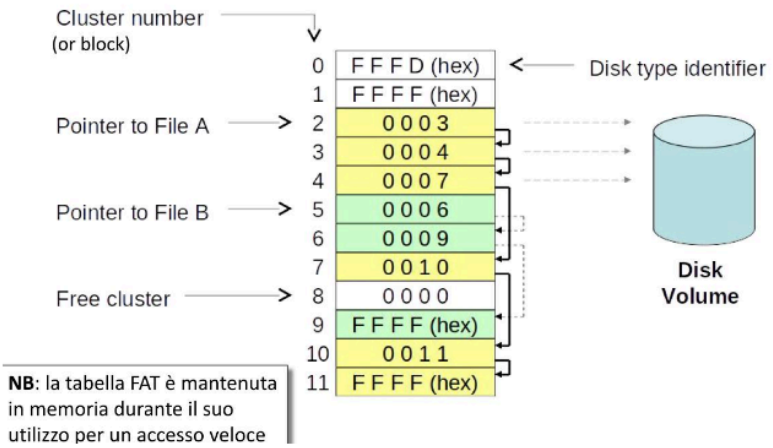
Si tratta di una **variante** dell'allocazione **concatenata** utilizzata da MS-DOS.

Si riserva una sezione all'inizio di ogni volume per la FAT.
La FAT ha un elemento per ogni blocco del disco.
L'elemento della directory contiene il numero del primo blocco del file.
L'elemento della FAT indicizzato da quel blocco contiene il numero del blocco successivo.
La fine del file è indicata da un elemento della tabella con valore speciale di fine file.

Solitamente si usano **cluster** al posto dei blocchi.

L'intera FAT può essere caricata in memoria per facilitare la ricerca di blocchi liberi.
IN FAT-32 si mantengono anche due valori

- FSI_Free_Count: il numero di blocchi liberi.
- FSI_Nxt_Free: Un suggerimento per il prossimo blocco libero da cercare.



La dimensione riservata alla FAT determina la massima lunghezza di una partizione.
Per FAT-32, Windows impone una massima lunghezza di 2TB per la partizione con blocchi da 32KB.
Alternativamente, con blocchi da 64KB si sale a 16TB indirizzabili.

Un file deve essere più piccolo di 4 GB: Infatti con 32 bit si possono gestire solo 2^32 byte = 4 GB.

Il formato **exFAT** estende il limite e permette di entrare nella gamma degli exabyte come dimensioni massime teoriche (usa 64 bit).

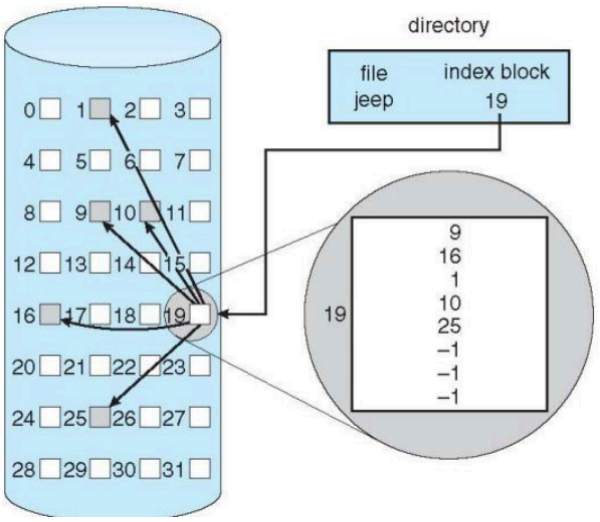
Attualmente, **Windows** utilizza il file system **NTFS**.
Le versioni Pro e Enterprise supportano l'utilizzo di **ReFS** (la Home solo in lettura), un particolare file system per le gestione e la memorizzazione di grandi file.

Funzionalità	ReFS	NTFS
Lunghezza massima del nome del file	255 caratteri Unicode	255 caratteri Unicode
Lunghezza massima del nome del percorso	32.000 caratteri Unicode	32.000 caratteri Unicode
Dimensione massima dei file	35 PB (petabyte)	256 TB
Dimensioni massime volume	35 PB	256 TB

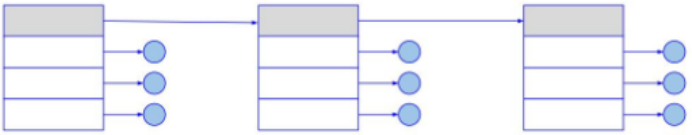
Allocazione indicizzata

Per ogni file esiste un **blocco indice** all'interno di una **tabella indice**, che contiene tutti i puntatori ai blocchi del file.

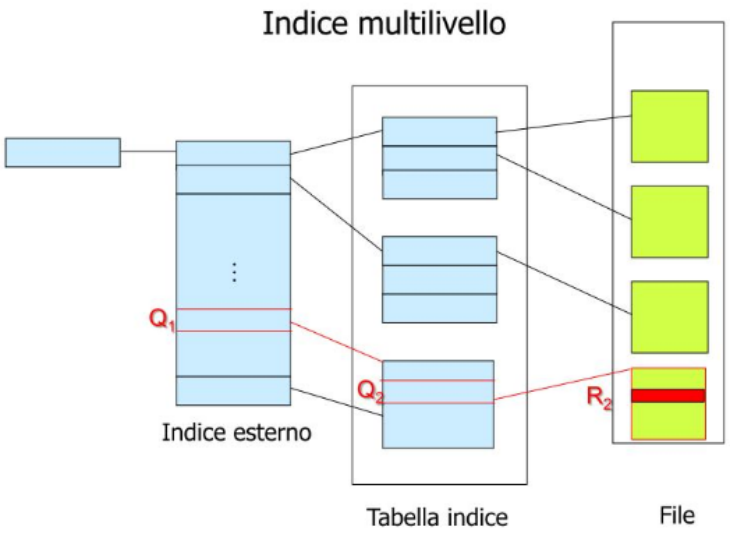
L’accesso è casuale, non c’è frammentazione interna.



Si può implementare un **sistema concatenato** per cui i vari blocchi indice sono tra loro collegati.
In questo caso, se un blocco indice contiene n elementi, n-1 di essi punteranno ai dati e 1 punterà al successivo blocco indice.
Allora la dimensione dei file non ha alcun limite.



Alternativamente, si può usare un **sistema multilivello**, simile a quanto visto per la tabella delle pagine.
Allora un primo livello avrà una tabella che indicizza tabelle, e così via.



UNIX utilizza un sistema combinato con indirizzi a 32 bit:

- I primi 10 indirizzi sono ad indirizzamento diretto
- L'undicesimo indirizzo è al primo livello di indirettezza.
- Il dodicesimo indirizzo è al secondo livello di indirettezza.
- Il tredicesimo indirizzo è al terzo livello di indirettezza.

Prestazioni

Il **miglior metodo** per l'allocazione dei file **dipende dal tipo di accesso**:

- L'**allocazione contigua** ha ottime prestazioni **sia per accesso sequenziale che casuale**.
- L'allocazione **concatenata** si presta naturalmente all'accesso **sequenziale**.
- L'allocazione **indicizzata** è più complessa e può richiedere più accessi per sistemi multilivello. Le tecniche di clustering possono migliorare il throughput e ridurre l'overhead sulla CPU.

L'allocazione concatenata risulta comunque meno efficiente di quella contigua per quanto riguarda l'accesso sequenziale.
L'unico limite di quella contigua è la necessità di individuare gli spazi liberi contigui sul disco.
Per risolvere questo problema, i moderni SO ricorrono periodicamente ad algoritmi di deframmentazione, che riorganizzano i file sul disco in modo da ristabilire l'adiacenza tra file e liberare porzioni contigue quanto più grandi possibile.

La **velocità di I/O dei dischi è molto lenta**: un processore moderno a 3 GHz può eseguire 317900 MIPS (Million Instructions Per Second) (Intel Core i7 6950x).
Un disco HDD invece si aggira sui 250 I/O al secondo.

Facendo un rapido calcolo si ha 317900/250 sono circa 1270 milioni di istruzioni durante un unico accesso al disco.

Spazio libero

Per tenere traccia dello spazio libero disponibile su un disco, il sistema conserva una **lista dello spazio libero**.
Per **creare un file** bisogna

- cercare nella lista dello spazio libero la quantità di spazio necessaria e allocarla al file.
- aggiornare il contenuto della lista.

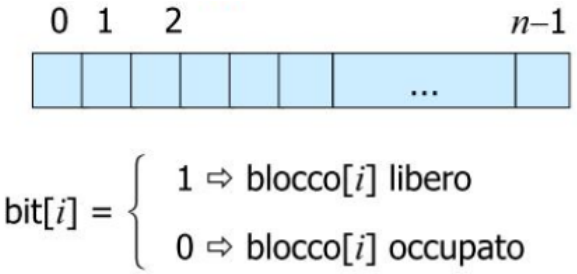
Per **rimuovere un file** bisogna

- aggiungere alla lista dello spazio libero i blocchi che erano assegnati al file.
- aggiornare il contenuto della lista.

La lista non sempre è veramente una lista.

Bitmap

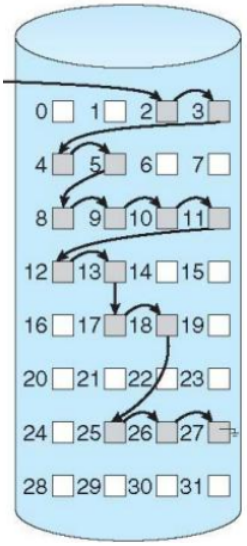
Il **vettore di bit** o **bitmap** è un vettore di n blocchi tale per cui



Si cerca quindi il primo byte diverso da 0.
Per avere buone prestazioni, la bitmap deve trovarsi in **memoria centrale**.
La bitmap è solitamente comoda da mantenere in memoria.

Lista concatenata

Si collegano tutti i blocchi liberi mediante puntatori e si mantiene un puntatore al primo blocco libero. Ogni blocco contiene il puntatore al prossimo blocco libero.



Non si spreca spazio e spesso non è necessario attraversare la lista.

Nella FAT, il conteggio dei blocchi liberi è incluso nella struttura dati per l'allocazione e non richiede un metodo di gestione separato.

Lista di blocchi liberi

Si realizza una **lista di blocchi** (**grouping**).
Sul primo blocco, si memorizzano gli indirizzi di n blocchi liberi, dei quali n-1 sono effettivamente liberi, mentre 1 contiene gli indirizzi di altri n blocchi.

Il **conteggio** è implementato tramite una lista che contiene un indirizzo del disco e un contatore che indicano rispettivamente un blocco libero e il numero di blocchi liberi contigui che lo seguono.

TRIM

TRIM è un comando ATA (Advanced Technology Attachment) che consente al **SO** di **informare un SSD su quali blocchi di dati può cancellare**, perché non sono più in uso.

TRIM è **complementare alla garbage collection**.

- Elimina la copiatura di pagine scartate o non valide durante il processo di garbage collection per risparmiare tempo e migliorare le prestazioni della SSD.
- L'SSD ha un minor numero di pagine da spostare durante la garbage collection, il che allunga la vita della SSD riducendo i cicli di programmazione/cancellazione.

Prestazioni

In UNIX, gli inode (FCB) sono preallocati e distribuiti nel disco, in modo da mantenere dati e metadati vicini e diminuire il tempo di seek durante le operazioni di I/O.

Se si aumenta la dimensione dei puntatori, aumenta anche la dimensione della memoria gestibile tramite file system, ma anche la dimensione delle strutture dati necessarie per l'allocazione.

Le prestazioni dipendono da

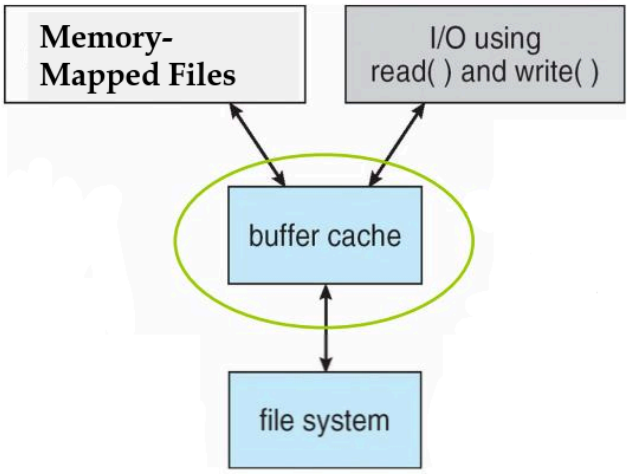
- Vicinanza di dati e metadati nel disco.
- Disponibilità di **buffer cache** per contenere i blocchi usati di frequente.
- Utilizzo di tecniche di ottimizzazione di tale cache per migliorare l'accesso sequenziale.
- Possibilità di avere **scritture asincrone** in modo da non dover aspettare il disco per proseguire.

Buffer cache

Una **cache delle pagine** impiega tecniche di memoria virtuale per gestire i dati dei file, trattando blocchi del disco come pagine di memoria.

L'I/O mappato in memoria impiega una **cache delle pagine**, mentre **I/O da file system** (con chiamate a sistema) utilizza la buffer **cache del disco** in memoria centrale.

I SO attuali usano una **buffer cache unificata** (**page cache**), che prevede l'utilizzo di un'unica cache per memorizzare sia i file mappati in memoria che i blocchi trasferiti dalle operazioni di I/O ordinario da file system, **evitando** il **double caching**.



LRU

L'algoritmo LRU è in generale ragionevole per la **sostituzione delle pagine e dei blocchi**.
Tuttavia

- Pagine relative ad un file da leggere in modo **sequenziale** non si dovrebbero sostituire secondo LRU, in quanto una pagina usata di recente non verrà probabilmente più utilizzata.
- Il **rilascio indietro** rimuove una pagina della cache non appena si verifica una richiesta della pagina successiva.
- Con la **lettura anticipata** si leggono e copiano nella cache la pagina richiesta e diverse pagine successive, che probabilmente verranno referenziate in sequenza.

Malfunzionamenti

Poiché i file e le directory sono mantenuti sia in memoria RAM (parzialmente) che nei dischi, è necessario assicurarsi che malfunzionamenti nel sistema non comportino la **perdita di dati** o l'**incoerenza**.

Si possono verificare anche incoerenze tra le strutture, ad esempio il contatore degli FCB può indicarne uno allocato, nonostante nella directory non contenga un puntatore all'elemento.

Coerenza

Il **verificatore di coerenza** (*fsck* in UNIX e *chkdsk* in DOS/Windows) confronta i dati della struttura di directory con quelli dei blocchi nel disco per accertarsi che non ci siano incongruenze.

Si possono effettuare **backup** su un altro dispositivo di memorizzazione ed eventualmente recuperare file persi o contenuti in dischi danneggiati da tale backup.

Network File System (NFS)

Consiste nella realizzazione di un **sistema di accesso a file remoti** attraverso LAN o WAN.

Nasce in ambiente UNIX (e supportato da Linux) ed usa protocolli UDP/IP o TCP/IP.

Nel contesto dell'NFS si considera un insieme di **stazioni di lavoro interconnesse** come un insieme di calcolatori indipendenti **con file system indipendenti**.
E' garantito un certo grado di **condivisione tra file system**, su richiesta esplicita e in modo trasparente.
Una directory remota viene montata su una directory del file system locale.
Per il montaggio della directory remota occorre fornire la locazione di tale directory nella rete.

Potenzialmente (nel rispetto dei diritti di accesso) **qualsiasi file system o directory** nel NFS **può essere montato** in modo remoto su qualsiasi directory locale.

L'NFS è progettato per operare in un **ambiente eterogeneo** di calcolatori, SO e architetture di rete. E' indipendente dall'ambiente hardware e software che gli fa da substrato.
Si ottiene questa indipendenza tramite l'utilizzo di **RPC** (Remote Procedure Call) costruite su un protocollo di rappresentazione esterna dei dati (**XDR, External Data Representation).**

La definizione di NFS distingue tra i servizi relativi al montaggio (protocollo di montaggio) e quelli relativi all'accesso ai file remoti (protocollo NFS).

Alcuni **vantaggi** del NFS sono

- Il **risparmio di spazio disco**, in quanto i dati devono essere salvati su una sola macchina ma sono accessibili anche a tutte le altre.
- Il fatto che non sia necessario avere **home directory** separate su ogni macchina, in quanto possono essere rese disponibili attraverso la rete.
- Si riduce la necessità di **unità per supporti rimovibili** come CD-ROM e USB in quanto possono essere usati da altri dispositivi nella rete.

Affidabilità

Si dice **affidabilità** di un qualsiasi dispositivo la **probabilità che esso funzioni** correttamente per un dato tempo.
Si tratta di una probabilità, quindi il suo valore varia tra 0 e 1.

I **guasti** possono essere di natura **permanente**, ovvero dopo essersi verificato rimane presente, oppure **temporaneo**, che si divide in **transitorio** (se accade una sola volta) o **intermittente** (se accade in modo imprevedibile).

Le **tipologie di errore** dello standard IEEE sono dovuti a
- Guasti componente
- Errore operatore
- Interferenza fisica esterna
- Errore di progettazione
- Errore di programmazione
- Errore di struttura dati

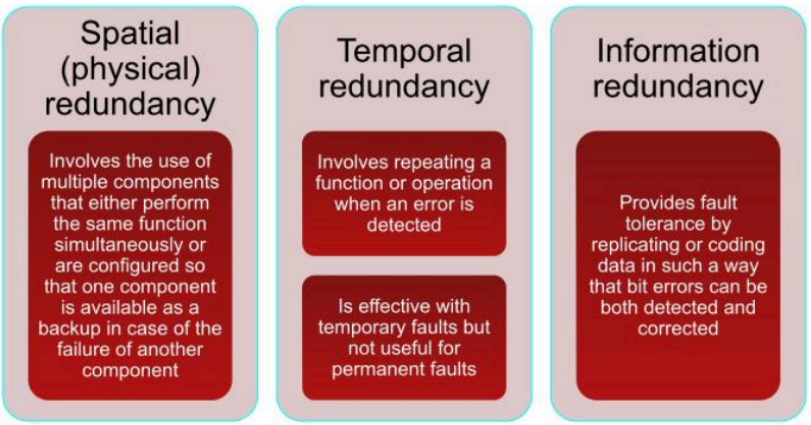
Definiamo **fault tolerance** l'abilità di un sistema o di un componente di continuare il normale funzionamento nonostante la presenza di guasti.
Tipicamente, include un qualche tipo di **ridondanza**.
Lo scopo è di aumentare l'affidabilità del sistema, a discapito di costi economici o di prestazioni.

Definiamo **affidabilità logistica** la probabilità che non si verifichi alcun guasto (di ogni genere)

L'**affidabilità di missione** è invece la probabilità che non si verifichino guasti con conseguenze gravi, ovvero tali da compromettere la funzionalità del sistema.
Si distingue tra **guasti significativi** (che degradano la funzionalità del sistema, ma non compromettono il completamento della missione) e **guasti maggiori** (che impediscono il completamento della missione).

La **sicurezza** è la probabilità che non si verifichino guasti con possibili conseguenze catastrofiche, tali da produrre danni a persone, cose o al sistema stesso.

Esistono diverse **soluzioni** da implementare nel SO per renderlo fault tolerant:
- Isolamento dei processi
- Controllo della concorrenza
- Virtual machines
- Checkpoints e rollbacks



NOTA: Guardare esempi slide da 633 per vedere un esercizio

Consideriamo una **funzione di guasto Q(t)**, ovvero la probabilità che un sistema si guasti per la prima volta nell'intervallo (0,t).
Se T rappresenta la variabile aleatoria "durata di vita del sistema", si ha
Q(t) = P{T <= t}

Q(t) è quindi una **funzione di distribuzione cumulativa**, che soddisfa le seguenti proprietà:

- Q(t) = 0 per t = 0.
- 0 <= Q(t) <= Q(t +Δt) per Δt >= 0.
- Q(t) = 1 per t = infinito.

Q(t) è una **funzione continua** e la **sua derivata** equivale alla **densità di probabilità di guasto**:
q(t) = dQ(t)/dt

Al contrario, la funzione **affidabilità R(t)** è la **probabilità che il sistema funzioni correttamente nell'intervallo (0,t)**.
Quindi, prendendo T "durata di vita del sistema", si ha
R(t) = P{T > t}

Come prima, valgono

- R(t) = 1 per t = 0.
- 1 >= R(t) >= R(t+Δt) per Δt >= 0
- R(t) = 0 per t = infinito

Come prima, la **derivata** rappresenta la **densità di probabilità di affidabilità**.
Si ha che
r(t) = dR(t)/dt = -q(t)

Se la frequenza di guasto è costante, allora il **tempo medio di guasto MTTF** coincide con il tempo medio tra due guasti (Mean Time To Failure).

Nel caso di sistemi riparabili, è necessario considerare anche l'evento di riparazione o sostituzione. Si definisce **disponibilità** la quantità

A = MTTF / (MTTF + MTTR)

In cui **MTTR** è il Mean Time To Repair (tempo medio di riparazione).

NOTA: Guardare slide 638 per esempi su esercizi con RAID.